Diseño de compiladores

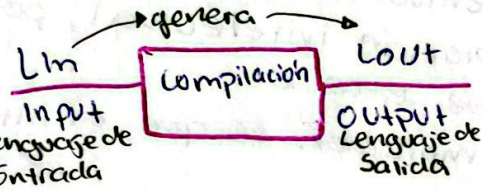
Procesadores de lenguaje → Bloque que trata la materia

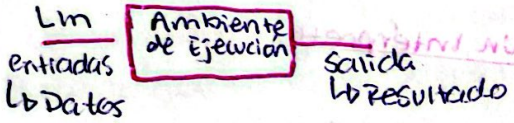
↳ cualquier herramienta que nos permita introducir un mensaje de entrada y genera una salida.

El proceso de compilación tiene dos escenarios

- Lenguaje de entrada: Donde se procesa el lenguaje de entrada.
- Lenguaje de salida: Puede ser o NO ejecutable

2↳ Compiladores → ciencia del lenguaje





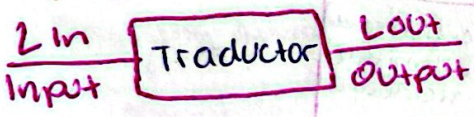
En un compilador el L In tiene un nivel de complejidad diferente al LOut

El problema de la compilación es entender. La capacidad de entender el problema tener la capacidad de crear el efecto necesario.

La razón de ser de un compilador es la de convertirse en un procesador de lenguaje

2↳ TRADUCTOR ↳ Procesador de lenguaje que acepta como entrada un lenguaje fuente y produce como salida uno destino

Ej. Trad. idiomas, n° romanos



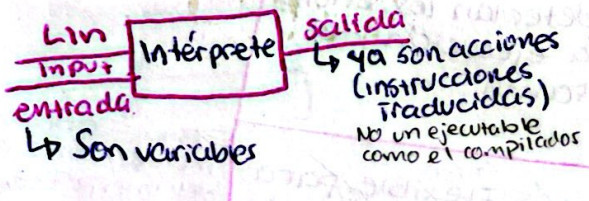
Su estructura es más simple y fácil de manejar

La entrada y salida del Traductor tienen un nivel de complejidad similar

$$\underset{\text{Input}}{LIn} \cong \underset{\text{Output}}{LOut}$$

es decir, en el contexto de las construcciones (estructura)

2↳ Intérprete: 2↳ Hace un análisis línea por línea



El Intérprete recibe el LIn (instrucciones) y los datos necesarios (combina ambas cosas) y genera un resultado.

Recibe una parte del LIn (una sentencia, función pero NO Todo a la vez)

Scanned with CamScanner

- El intérprete solo recibe instrucciones, sentencias, oraciones. Es decir, Unidades básicas que ya tienen sentido y ejecuta,
- El compilador puede recibir un libro completo y generar un ejecutable

Diferencia entre el intérprete y el proceso de compilación.

- No se puede afirmar que uno sea mejor que otro, depende el contexto
- En un compilador, la salida, depende de la arquitectura, en el Intérprete tiende a ser más portable e independiente de la arq.

→ La ventaja del compilador sobre el intérprete, es la seguridad. El compilador es más portable y eventualmente brinda mayor seguridad, todo esto en sentido de apropiación intelectual, porque hace una Ing. Inversa en el Amb. de Ejecución.

→ La ventaja del Intérprete sobre el compilador, es que el intérprete es más Fácil de mantener.

Diferencias de un compilador y Un Intérprete

Compilador	Intérprete
Tiene como salida un único lenguaje objeto	Tiene como salidas instrucciones traducidas
El rendimiento de la Ejecución del programa compilado (la salida) es más rápido que interpretado	El rendimiento (del intérprete) se somete al rendimiento de analizar la traducción una por una
No requiere del programa fuente porque el programa objeto es ejecutable	Se requiere del lenguaje Fuente para su ejecución
Los errores sintácticos y semánticos se detectan antes de la ejecución del programa objeto.	Se detectan los errores en la ejecución del programa.
Tiene menos Flexibilidad en el uso de la memoria para el programa obj.	Es más Flexible para que el programa pueda usar la memoria.

Está asociado a una memoria de ejecución

→ Está asociado a la memoria de ejecución del intérprete,

PROCESADORES de Lenguaje

Modelo Híbrido - Tendencia.

- Una app... ¿Qué modelo utiliza? - Trad, comp, Interp o modelo híbrido
  ↳ Tener claridad sobre esto.

Scanned with CamScanner

Proceso de compilación

Ej: posición (símbolo) = inicial (operador) + velocidad
(identificadora)

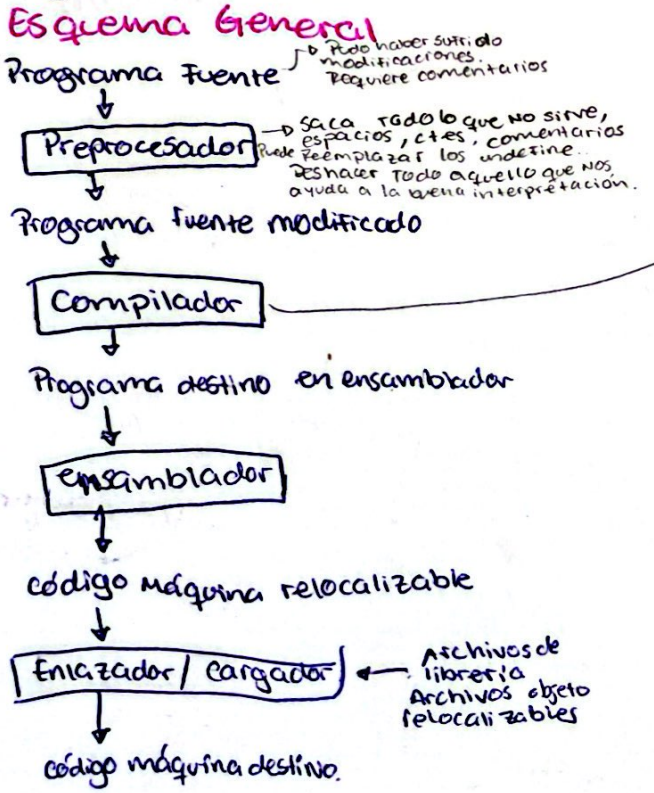
COMPILACIÓN

ANÁLISIS
- Léxico (Palabras) Roles que tienen los caracteres al formar palabras.
- sintáctico (Roles dentro de la oración) Análisis de instrucciones.
  Artículo sustantivo, verbo ....
  sujeto
- semántico (Tiene sentido?) Significado de la oración.

SÍNTESIS
- Generador de código intermedio
- Optimización
- Generador de código

\# La compilación: Es un proceso de todo lo que le va a pasar al lenguaje para convertirse en instrucciones o código máquina

Esquema General



Fases de un compilador

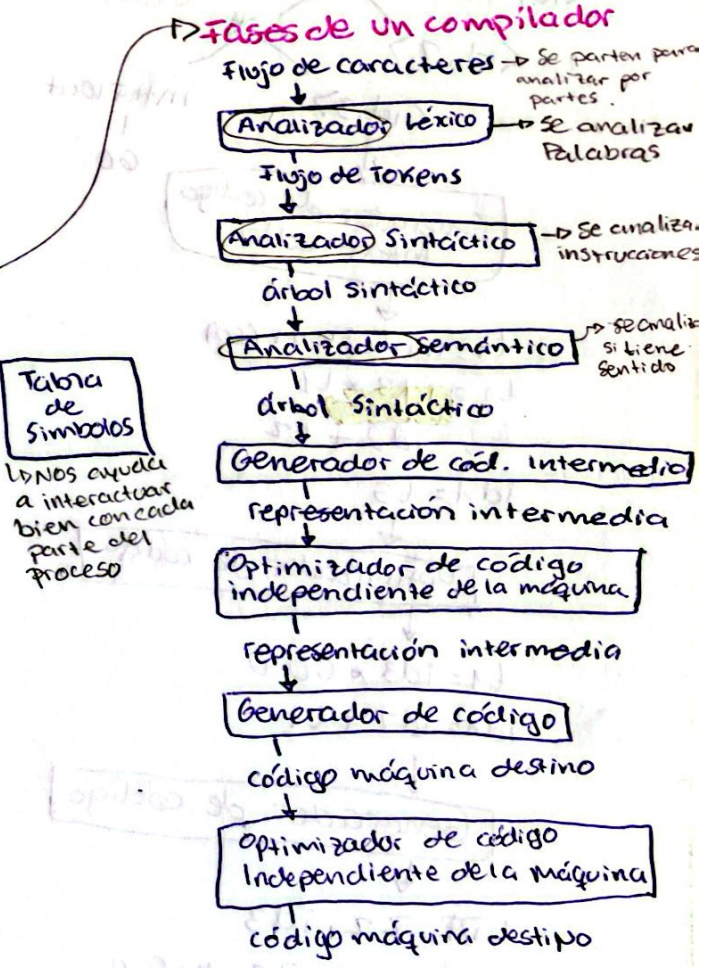


Tabla de símbolos
Nos ayuda a interactuar bien con cada parte del proceso

Fases de un compilador

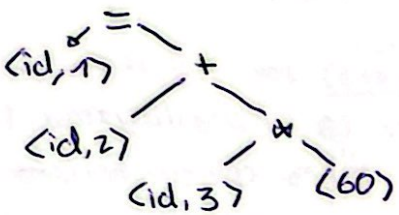
Ej: este ejemplo está mal → Según el Profe.

Posición = inicial + velocidad * 60

↓ Analizador léxico

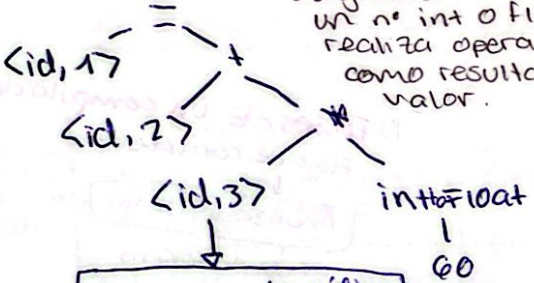
<id,1> <=> <id,2> <+> <id,3> <*> <60>
(1 identificador)

↓ Analizador sintáctico



↓ Analizador semántico

Dice si la operación de asignación en el que habrá un nº int o float que realiza operaciones y da como resultado un tipo de valor.



↓ Generador de código intermedio

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

↓ Optimizador de código

```
t1 = id3 * 60.0
id1 = id2 + t1
```

↓ Generador de código

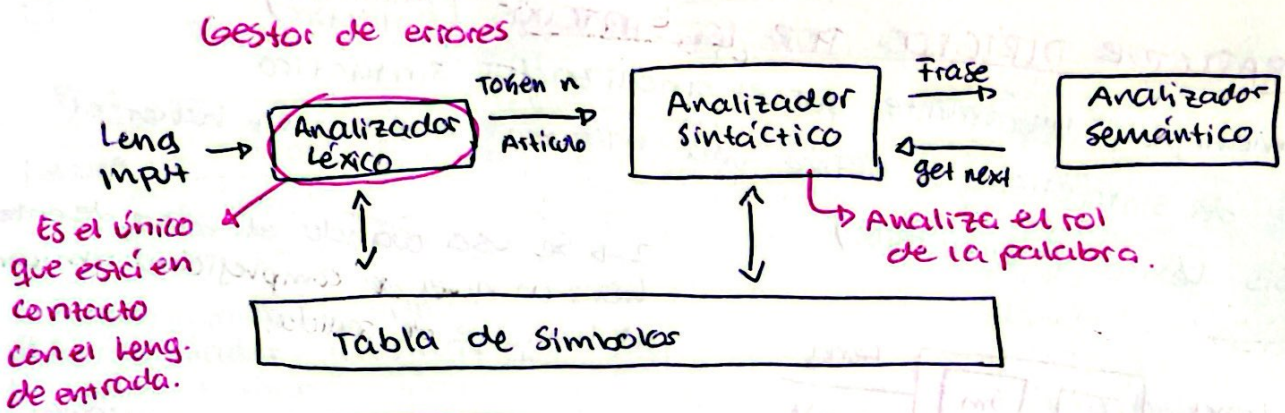
```
LDF R2, id3
MULF R2, R2, #60,0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```

Tabla de símbolos

1	posicion	....
	inicial	....
	velocidad	....

↳ También está en contacto con la tabla de errores.

Scanned with CamScanner



→ El analizador léxico es el que carga la tabla de símbolos, el resto va a consultar, modificar y eventualmente la va a utilizar para analizar errores.

→ El sintácticto forma las palabras, construye la oración y le pasa al semántico.

Es probable que la tabla de símbolos esté vacía o esté cargada con palabras reservadas (while, for, if ...) al principio.

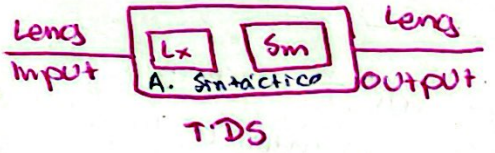
El que carga la tabla de símbolos es el analizador léxico.

El sintáctico y semántico sólo hacen consultas sobre la tabla.

Scanned with CamScanner

TRADUCTOR DIRIGIDO POR LA SINTAXIS (Analizador sintáctico)

Está manejado únicamente por el analizador sintáctico (dentro de sintáctico, se define una manera nueva para hacer el análisis léxico y semántico)



→ Se usa cuando el Leng de entrada tiene un nivel de complejidad similar al lenguaje de salida.

→ El analizador sintáctico interactúa con el lenguaje de entrada a diferencia de lo que ocurre en un compilador tradicional.

→ El analizador sintáctico cuenta con una NOTACIÓN que permite representar la sintaxis del lenguaje de entrada en forma de reglas sintácticas y la interacción con ellas. Se denomina BNF y está formado por: PPT

① Conjunto de símbolos terminales (aparecen explícitamente en una gramática o lenguaje de entrada.

② Conjunto de NO terminales (variables sintácticas)

Terminales forman parte de la entrada.

BNF – (Backus - Naur Form) → Lenguaje Natural

Define una jerarquía de una producción, es decir, un sistema jerárquico de producciones

Prod: no-terminal → no-terminal/terminal
↳ variable sintáctica

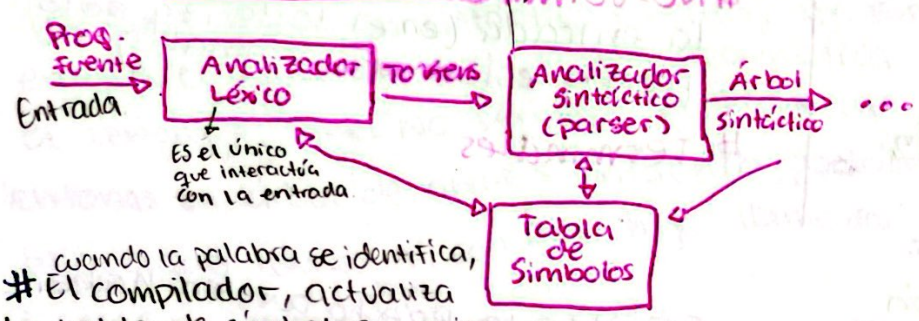
Top: Prod. inicial → designación de uno de los NO-terminales como símbolo inicial.

↳ con esto buscamos describir la gramática (lenguaje) formado por cadenas de sumas y restas de dígitos.

Scanned with CamScanner

PPT

Fases de un compilador tradicional



El compilador, cuando la palabra se identifica, actualiza la tabla de símbolos, y avisa al analizador sintáctico que identifica una palabra y le dice el rol que cumple esa palabra

Ej: La clase.

'La' pertenece a una clase denominado artículo definido

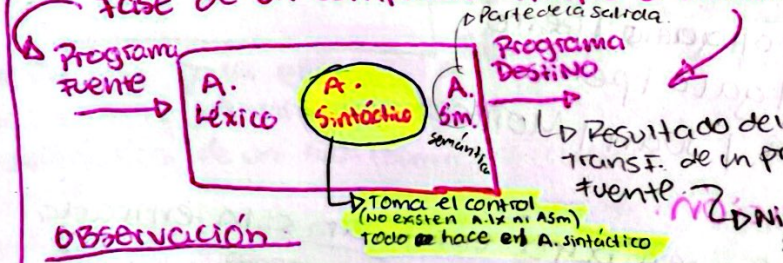
El A. Léx carga en la Tabla de símbolo y avisa al analizador sintáctico que encontró un 'artículo', osea, el rol que cumple.

Notar que...

- El Analizador Léxico genera e interactúa con el analizador sintáctico mediante Tokens.
- Sólo el analizador léxico es el único que interactúa con el programa fuente en un compilador trad.
- El analizador sintáctico interactúa con el analizador semántico através del árbol sintáctico generado.

TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS (Traductor un analizador sintáctico)

Fase de un compilador simple o traductor dirigido por la sintaxis



Puede ser que tengamos una entrada, una oración en español, está en presente y lo traducimos al pasado, ocurre una transformación de la entrada

Nivel de complejidad similar

OBSERVACIÓN

- En un TDS, el analizador sintáctico maneja el proceso.
- El analizador léxico y semántico se subordinan al sintáctico, y actúan como procesos que se ejecutan dentro del mismo.
- El analizador sintáctico interactúa con el lenguaje de entrada a diferencia de lo que ocurre en un compilador tradicional.

compilador simple

La parte de un compilador simple que interactua con un prog. fuente es el sintáctico.

TDS

- Existe una simplificación de los procesos

El lenguaje de entrada tiene un grado de ambigüedad.

Jen y Ptt

b) Dos herramientas de validación que permiten aplicar las reglas sintácticas a cualquier lenguaje de entrada de manera a validar su construcción e identificando errores cuando corresponda.

Las herramientas son:

- Árboles sintácticos
- Derivaciones.

Scanned with CamScanner

BNF

Ej: Entrada = 'la casa'

Una manera de describir la entrada es por: **extensión** se lista todas las posibles entradas que se pueda tener.

La otra es por: **Comprensión** es más estructurada que permite 'validar', 'matchear', 'parsear' la entrada

→ var. sintáctica

#NO-terminal: NO es una parte de la entrada (en el lado izq solo pueden estar NO terminales)

Terminales

En el lado derecho pueden haber terminales y NO-terminales

BNF:

Describir la entrada, si la entrada es: 'la casa'

Deriva en / Tiene la forma de

Los hijos son el lado derecho.

Sistema Serárquico → árboles

- Sujeto → artículo sustantivo | sustantivo_propio
- Artículo → artdefinido | artindefinido
- Artdefinido → el | la | los | las
- Artindefinido → un | una | unas | unos
- Sustantivo → casa | auto | gato | perro
- Sustantivo_propio → Joni | Juan | Moisés

Herramientas de validación.

- Árbol sintáctico
- Derivación

↳ Sirven para verificar si la entrada pertenece o NO al lenguaje

Lo ver gráficamente para detectar y manejar la entrada.

→ lenguaje definido por un conjunto de reglas

PPT - Libro

→ Una gramática → Lenguaje de entrada. independiente de contexto en su representación

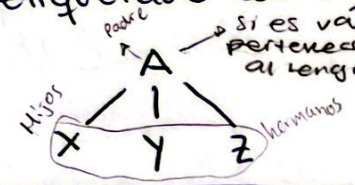
BNF está formado por:

1. Un conjunto de símbolos terminales, que son los símbolos que aparecen explícitamente en una gramática o lenguaje de entrada.
2. Un conjunto de NO_terminales a los que se le conoce como variables sintácticas → todo lo que aparece en el lado izq, NO debe aparecer en la entrada.
3. Un conjunto de producciones en forma jerárquica, formadas por:
   - un lado izq. donde se encuentra un NO-terminal
   - el símbolo → que se lee como "deriva en" o "tiene forma de".
   - Un lado derecho formado por un conjunto de terminales y NO-terminales.
4. Una designación de uno de los NO-terminales como símbolo inicial

Scanned with CamScanner

2 Árbol Sintáctico → Específico de una entrada.

Un árbol de análisis sintáctico muestra, en forma gráfica, la manera en que el símbolo inicial de una gramática deriva a una cadena en el lenguaje. Si el NO terminal A tiene una producción $A \to XYZ$, entonces un árbol de análisis sintáctico podría tener un Nodo interior etiquetado como A, con tres hijos llamados X, Y, Z, de izq a der:



El "ε", es un recurso que utilizaremos cuando necesitemos deshacernos de una rama.
Ej: Hacer que NO pueda existir
La hoja puede ser ε

Propiedades

1) La raíz se etiqueta con el símbolo inicial. vacío ↑
2) Cada hoja se etiqueta con un terminal o con ε
3) Cada Nodo interior se etiqueta con un NO-terminal
4) Si A es el NO terminal que etiqueta a cierto Nodo interior y $X_1, X_2 \dots X_n$ son etiquetas de los hijos de ese nodo de izq a derecha, entonces debe haber una producción $A \to X_1 X_2 \dots X_n$. Aquí, cada una de las etiquetas $X_1, X_2, \dots X_n$ representa un símbolo.

#Recurso para eliminar el efecto de una variable sintáctica de un NO-terminal → 'ε'

EL VACÍO NO forma parte del lenguaje de entrada

#El objetivo del árbol es, agarrar la cadena de entrada y tratar de apartir de la cadena de entrada, las hojas, construir un árbol que termine en una raíz que sea el padre. o a partir del padre, construir un árbol posible que refleje la entrada

Backtracking ocurre cuando tenemos una producción compuesta

Siempre que haya una prod. compuesta, tenemos la chance del Backtracking → ocupa mucho espacio de procesamiento, se vuelve ineficiente.

#El backtracking debe ser evitado

2 Derivaciones

Ejemplo BNF → El BNF define un lenguaje de entrada formado por suma y resta de dígitos

cadena → cadena + cadena
cadena → cadena - cadena
cadena → cadena
cadena → 0|1|2|...|9

Símbolos terminales: {0, 1, 2, ... 9, +, -}
Símbolos No-terminales: {cadena}
Prod Inicial: cadena.

Ejemplos de cadenas de entrada:
5+2-1 (cadena de entrada) → se puede encontrar un árbol sintáctico que reproduzca la entrada
9+10+3 (No es una cadena de entrada)

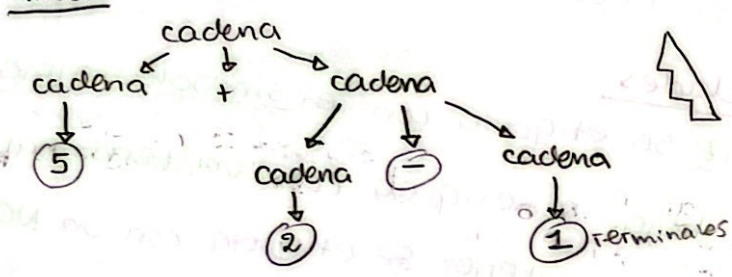
Scanned with CamScanner

b) Para el caso de una cadena de entrada 5+2-1 y el BNF cadena: **La derivación es de izq a derecha** 1° camino

cadena → cadena + cadena → 5 + cadena → 5 + cadena - cadena ...

... 5 + cadena - cadena → 5 + 2 - cadena → 5 + 2 - 1

Árbol



Este árbol tiene profundidad hacia la izq.

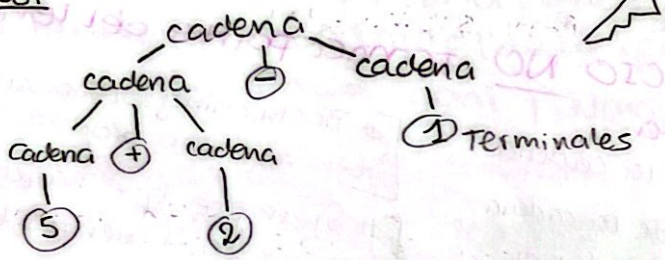
Derivación por la derecha (der a izq)

cadena → cadena - cadena → cadena - 1 → cadena + cadena - 1 ...

... cadena + 2 - 1 → 5 + 2 - 1

Tiene más profundidad hacia la derecha.

Árbol



Tenemos dos árboles para una misma entrada, eso significa que estamos en presencia de un lenguaje ambigüo → Se tiene dos caminos para llegar a la entrada.

La ambigüedad es mala.
↳ Produce backtraking
↳ El efecto es malo.

Si voy por un camino y no llego, la cadena NO pertenece al lenguaje.

→ Tenemos que buscar un mecanismo para que mi gramática NO sea ambigüa.
↳ Para que las producciones compuestas No me creen problemas.

Resumen !! obs.

Escribir una gramática que nos permita describir la entrada es lo ideal.

- Si hay 2 o más árboles sintáctico o derivaciones para la misma cadena de entrada.
  LA GRAMÁTICA ES AMBIGÜA (Es un problema) → Produce Backtracking.
- Que una entrada NO sea ambigua NO significa nada, ya que se debe probar todas las entradas.

Scanned with CamScanner

Clase 3 V.

Ejercicio (Tarea)

Determinar cual de las sgtes gramáticas son ambigüas. Justificar obteniendo el árbol sintáctico correspondiente.

1) $S \to 0S1 \mid 01$ → NO es Ambigüa

2) $S \to S(S)S \mid \varepsilon$

3) $S \to aSbS \mid bSaS \mid \varepsilon$

Para determinar si una gramática es ambigua, se debe obtener al menos dos árboles sintácticos para una cadena de entrada que pertenece al lenguaje.

Desarrollo

① $S \to 0S1 \mid 01$ → Esta gramática NO era ambigüa al NO encontrar dos o más árboles sintácticos o derivaciones para una misma cadena de entrada

Ej: Entrada: 0011

$S \Rightarrow 0\overset{S}{S}1 \Rightarrow 00\overset{\varepsilon}{1}1$

Derivación: Siempre de izquierda a derecha.

② $S \to S(S)S \mid \varepsilon$ → NO forma parte de la gramática

$\Sigma = \{(,)\}$ → son Terminales

→ El Alfabeto de una gramática son los terminales.

Ej: Entrada = ()()

$S \Rightarrow \overset{\text{deriva a vacío}}{S}(S)S \Rightarrow S(S)S(S)S \Rightarrow (\overset{\text{se consume y queda}}{S})S(S)S \Rightarrow ()S(S)S \Rightarrow ()(S)S \Rightarrow ()()S$ (ε, ε, ε)

$\ldots \Rightarrow ()()$ → DERIVACIÓN POR LA IZQ.

DERIVACIÓN POR LA DERECHA

$S \Rightarrow S(S)S \Rightarrow S(S)S(S)\underset{\varepsilon}{S} \Rightarrow S(S)S(\overset{\varepsilon}{S}) \Rightarrow S(S)\overset{\varepsilon}{S}() \Rightarrow S(S)() \ldots$

$\ldots \Rightarrow S()() \Rightarrow ()()$

Realizando la derivación por la derecha e izq. se pudo reproducir la cadena de entrada, por lo tanto, la gramática es ambigüa

③ $S \to aSbS \mid bSaS \mid \varepsilon$

Ej: Entrada: abba

Derivación por la izq.

$S \Rightarrow a\underset{\varepsilon}{S}bS \Rightarrow abS \Rightarrow abb\underset{\varepsilon}{S}aS \Rightarrow abba\underset{\varepsilon}{S} \Rightarrow abba$ //

Derivación por la derecha.

$S \Rightarrow aSb\underset{\varepsilon}{S} \Rightarrow aSbbS\underset{\varepsilon}{a}S \Rightarrow aSbb\underset{\varepsilon}{S}a \Rightarrow a\underset{\varepsilon}{S}bba \Rightarrow abba$ //

Se llega al resultado por la der e izq.

Por lo tanto, la gramática es ambigua.

Scanned with CamScanner

Para saber si una cadena de entrada pertenece a un lenguaje definido por esa NOTACIÓN (conjunto de reglas - BNF), entonces se ejecutan algunos pasos:

1. Eliminar el indeterminismo (↳ indefinición, ciclo infinito) que pudiera tener la gramática.
2. Eliminar la ambigüedad (↳ sintáctica, semántica) o descubrir la ambigüedad.

↳ Indeterminismo: ¿Tiene recursión por la izq?
↳ Si tiene, se debe eliminar la recursión x izq.

↳ Ambigüedad:

① Eliminación de recursión por la izq.

↳ Exige que sea compuesta $A \to A\alpha \mid \beta$ NO se puede $A \to A\alpha$

Para toda producción que tenga la forma $A \to A\alpha \mid \beta$ (= recursión), es posible reemplazarla por el sgte sistema de producciones equivalente:

$A \to A\alpha \mid \beta$
↳ ES rec x izq porque al menos un término del lado derecho que empieza con el mismo NO-terminal

$$A \to \beta R$$
$$R \to \alpha R \mid \varepsilon$$

Preguntas

① ¿Por qué es importante eliminar la rec x izq
② ¿Qué alternativas se plantean para producciones donde NO se desee eliminar la rec x izq?

¿Qué se hace cuando hay rec. x izq.

Se reemplaza esa producción compuesta: "$A \to A\alpha \mid \beta$" por un conjunto de producciones nuevas $\begin{cases} A \to \beta R \\ R \to \alpha R \mid \varepsilon \end{cases}$ (son equivalentes = aceptan la misma entrada; cuando se reemplaza por R queda β)

¿Por qué es importante eliminar la Rec. x izq?

[Si tengo $A \to A\alpha \mid \beta$, se reemplaza por: $\begin{cases} A \to \beta R \\ R \to \alpha R \mid \varepsilon \end{cases}$]

Ej: $S \to S(S)S \mid \varepsilon$ $\quad A = S \quad \alpha = (S)S \quad \beta = \varepsilon$

$S \to S\overbrace{(S)S}^{\alpha} \mid \overbrace{\varepsilon}^{\beta}$

$S \to \beta R$, pero β es ε, entonces:

$S \to R$
$R \to (S)SR \mid \varepsilon$] $= S \to S(S)S \mid \varepsilon$

↳ Esta gramática NO tiene recursividad por la izq.

$A \to A\alpha \mid \beta$, en la rec izq le estoy buscando el lado compuesto

$A \to A\alpha$ ↳ No hay rec x izq para este caso porque NO tiene un caso base, está mal escrita

$A \to A\alpha$
$A \to \beta$ ↳ caso base

Ejemplo (Enunciado Anterior)

secuencia → secuencia + dígito

secuencia → secuencia − dígito

secuencia → dígito → β.

dígito → 0|1|2|3|4|5|6|7|8|9 B

secuencia → secuencia + dígito | secuencia − dígito | dígito

Eliminando la Rec x izq queda:

→ secuencia → secuencia + dígito | secuencia − dígito | dígito

A α1 A α2 β

secuencia → dígito → var. sintáctica específica (terminal)
└ solo puede derivar en los números

La gramática convertida:

secuencia → βR

R → + dígito R | − dígito R | ε → se pone por la fórmula

dígito → 1|2|...|9 → Es un NO terminal, una VAR. Sintáctica

El árbol es más profundo por la izq. ya que 'secuencia' es un NO-terminal y continuará creciendo por la izq.

¿La gram. es ambigüa?
└ Ya NO es ambigüa

Siempre que tengamos una gramática, debemos eliminar la rec. por izq.

La ambigüedad es manejable.

¿cómo saber si el árbol es profundo hacia la izq?

Es rec. x izq, porque existe al menos un lado derecho que empieza con el símbolo que está en la izq.

Debo eliminar la Rec x izq. de 'secuencia'

A → Aα | β

[A → βR
R → αR | ε]

2. Factor común por la izq.

Ej: S → 0S1 | 01

factor común

S → 0R

R → S1 | 1

→ 0 es un factor común, eso también crea ambigüedad. Porque NO sabremos con cuál iniciaremos (aplicaremos)

El factor común puede ser de terminales, NO-terminales o una combinación.

Si Hay Factor común ES ambigüa, se conoce como ambigüedad sintáctica.

Si tenemos recursión x izq y Factor común sobre el mismo símbolo, se aconseja eliminar la recursión por la izq. y luego se eliminan los dos

Scanned with CamScanner

Casos de Ambigüedad

① Ambigüedad Sintáctica (Factor común por la izq.)

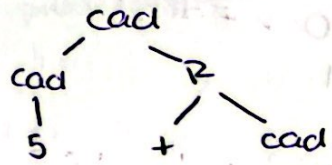
cad → cad + cad | cad - cad |0|1|2...|9

cad → cad R |0|1|2|...|9

R → + cad | - cad

cad → cad R
cad → 0
cad → 1
⋮
cad → 9
R → + cad
R → - cad

#R: variable Aux. Sintáctica



② Ambigüedad semántica (Factor común x la izq. indirecto)

Función Conjunto Primero → Es el 1º terminal al que puede llegar (al llegar a un terminal ya no es necesario recorrer)

P(cad) = P(cad R) ∪ {0} ∪ {1} ... ∪ {9}

P(R) = {+} ∪ {-} = {+, -}

③ Recursión por la izq

$$\begin{cases} A \to A\alpha \mid \beta \\ A \to \beta R \\ R \to \alpha R \mid \varepsilon \end{cases}$$

→ Se utiliza la fórmula para eliminar la ambigüedad y la indefinición.

cad → 0R' | 1R' | ... | 9R'

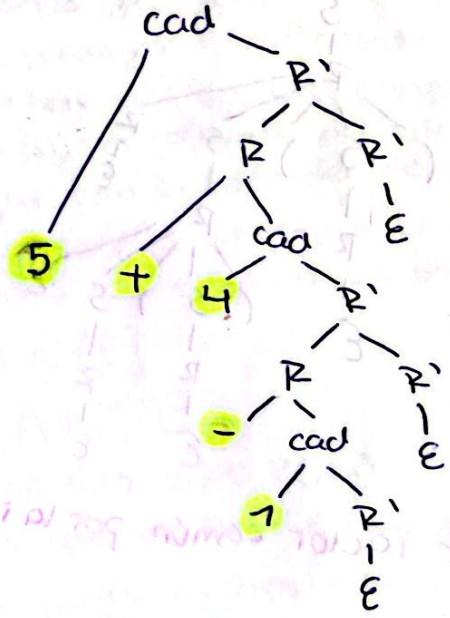
R' → RR' | ε

P(cad) = {0} ∪ {1} ... ∪ {9}

P(R') = {+, -} ∪ {ε}

P(R) = {+} ∪ {-}

Para hallar el conjunto primero ir de abajo hacia arriba es más fácil.



Scanned with CamScanner

[siempre eliminar primero la recursión por la izq]

cad → cad + cad

cad → cad - cad

cad → 0

cad → 1

⋮

cad → 9

cad → 0R | 1R | ... | 9R

R → + cad R | - cad R | ε

P(cad) = {0} ∪ {1} ∪ {9}

P(R) = {+} ∪ {-} ∪ {ε}

— Determinar si la gramática es ambigüa → Método Árbol sintáctico

R1 Eliminando Rec x izq. → EVITA LA REC. INFINITA.

A → A ∂ | β

A → βR

R → ∂R | ε

Aplicando la Fórmula

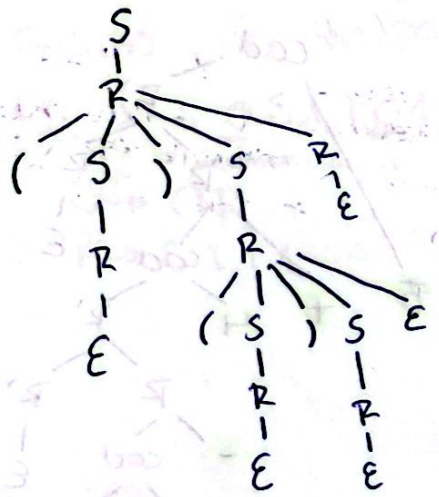
S → S(S)S | ε

β = ε

∂ = (S)S

⇒ S → R (tiene un solo lado derecho.)

R → (S)SR | ε



R2 Factor común por la izq. → No hay FC en el lado derecho

R3: Conjunto Primero

P(S) = {(, ε}

P(R) = {(} ∪ {ε}

* El código fuente de un traductor y un compilador en su fase sintáctica es capaz de detectar este tipo de cadenas de entrada

S → S(S)S | ε

→ Árbol de análisis sintáctico

son problemas
- Rec x izq.
- Factor común x izq

↳ Nos lleva a una situación de backtracking o ambigüedad en la cual no sé que producción, que lado derecho voy a utilizar, por lo gral ocurre cuando tenemos prod. compuestas

Decisiones
1°) TOP-DOW una palabra a la vez
2°) Recorrer la ent de izq a der
3°) La derivación que vamos a usar de izq a der

cuando tenemos una producción compuesta, un NO-terminal que puede derivar en varios lados derechos, entonces la elección de qué lado derecho voy a utilizar, se puede ver afectada por un factor común x izq.

DEFINICIÓN DEL CONJUNTO PRIMERO

Primero(x) (Terminal o NO-terminal)

① Si X es un terminal, entonces PRIMERO(X) = {X} ← terminal

② Si X es un NO-terminal y $X \to Y_1, Y_2 \dots Y_k$ es una producción para $k \geq 1$, entonces se agrega a en PRIMERO(X) si para cierta i, a está en PRIMERO(Y_i), y ε está en todas las funciones PRIMERO(Y_1)... PRIMERO(Y_{i-1})

③ Si ε está en todos los conjuntos PRIMERO(Y_1) a PRIMERO(Y_k), entonces se agrega ε en PRIMERO(X)

④ Si $X \to \varepsilon$ es una producción definida, entonces se agrega ε a PRIMERO(X)

② Debo irme a analizar el primero de su lado derecho, puede pasar dos cosas:
* Puede que Y_1 sea un terminal, entonces al 1° terminal que puede llegar X va a ser Y_1 * puede que Y_1 sea un NO-terminal, por lo tanto debo ir a buscar el 1° de Y_1 y voy a tener que trabajar con el lado der de Y_1

La única chance que tiene vacío de pertenecer a un conjunto PRIMERO de X, es que vacío esté en todos los elementos del lado derecho. El conjunto PRIMERO(ε) = {ε}

PRIMERO($X_1 X_2 \dots X_n$) → IMPORTANTE

1) Agregar a P($X_1 X_2 \dots X_n$) todos los símbolos que NO sean ε de P(X_1).
2) Si está ε en P(X_1) agregar todos los símbolos que NO sean ε de P(X_2)
3) Si ε está en P(X_1) y P(X_2), entonces agregar todos los símbolos que no sean ε de ambos, y así sucesivamente...
4) Si ε está en todos los conjuntos P(X_2) a P(X_n) entonces agregar ε al conjunto P($X_1 X_2 \dots X_n$)

Scanned with CamScanner

Ej: usamos la gramática

secuencia → dígito R1

R1 → RR1 | ε

R → + dígito | - dígito

dígito → 0|1|2|... |9

El vacío ε no es malo, nos obliga a tratar de buscar si existe algún caracter que pueda venir, nos indica que ese NO -terminal puede desaparecer

EL CONJUNTO PRIMERO de esta gramática es:

P(secuencia) = P(dígito R1) = {0,1,...,9}

↳ {ε}

El conjunto primero del terminal es el terminal mismo.

P(R1) = P(RR1) ∪ P(ε) = ... ∪ {ε} = {+, -, ε}

P(R) = P(+dígito) ∪ P(-dígito) = {+} ∪ {-} = {+, -}

↳ Escribir primero por separado

P(dígito) = {0} ∪ {1} ∪ {2} ... ∪ {9} = {0, 1, ..., 9}

La razón de usar el conjunto primero es la manera que yo tengo de ver si las producciones van a poder reproducir la entrada.

Lo que NO tiene que ocurrir en el conjunto PRIMERO:

- Si la producción es compuesta, cualquiera de los lados derechos me lleve a un mismo terminal. Si ocurre yo no voy a saber que lado derecho tengo que usar.

La condición que se debe buscar siempre:

- Si la producción es compuesta tiene que ser disjuntos los conjuntos primeros de cada lado derecho, es decir, que el conjunto que obtuve con el lado derecho debe ser disjunto al conjunto que obtenga, trabajando con ese lado derecho.

NO puede haber un terminal o vacío que se repita en dos o más conjuntos primeros porque si eso ocurre tengo un caso de FC.

no evidente

P → A | a P(P) = P(A) ∪ P(a) = ... ∪ {a}

A → B P(A) = P(B) = {a}

B → a P(B) = {a}

La funcionalidad del conjunto Primero es mirar anticipadamente qué terminales vas a derivar y poder establecer si existen o no conflictos

Problemas con el ε en conjunto PRIMERO

↳ continúa →

Problemas con el conjunto primero

$S \to ABC$

$A \to aA \mid \varepsilon$

$B \to bB \mid \varepsilon$

$C \to cC \mid \varepsilon$ (c terminal)

Para trabajar con el conjunto primero es recomendable trabajar de abajo hacia arriba. En un BNF, los de abajo estará más cerca de los terminales

- $P(S) = P(ABC)$ → todo el lado derecho ya que es una prod. simple

$P(ABC)$ es igual al $P(A)$, excepto ε unión $P(B)$, excepto ε, todo lo que NO es vacío se va agregando. Si encontramos vacío en cada uno de los no-terminales, entonces quiere decir que puede desaparecer, por tanto, agregamos también el ε como parte del conjunto

→ Todo el lado derecho

Ejemplo

$P(S) = P(ABC) = \{a, b, c, \varepsilon\}$

$P(A) = P(aA) \cup P(\varepsilon) = \{a\} \cup \{\varepsilon\} = \{a, \varepsilon\}$

$P(B) = P(bB) \cup P(\varepsilon) = \{b\} \cup \{\varepsilon\} = \{c, \varepsilon\}$

$P(C) = P(cC) \cup P(\varepsilon) = \{c\} \cup \{\varepsilon\} = \{c, \varepsilon\}$

En esta gramática debemos comparar las producciones compuestas (simple NO) si ✓ ✓ son disjuntos, eso se repite, si cumple, podemos decir que la gramática NO tiene ambigüedad semántica, tampoco tiene recursión por la izq, tampoco factor común, está bien construida.

$A \to A\alpha \mid \beta$

$A \to \beta R$

$R \to \alpha R \mid \varepsilon$

$A \to A\alpha \mid \beta$, No puede calcularse el CP ya que la gramática está mal construida, NO cumple con $A \to A\alpha \mid \beta$. La rec. x izq EXIGE que siempre sea compuesta.

$\beta = \varepsilon$
$\alpha = b$

$R \to bR \mid \varepsilon$

Ejemplo: Hallar el conjunto PRIMERO (CP)

$S \to AB \mid BA$

rec. izq $A \to A\alpha \mid \varepsilon$

rec izq $B \to B\beta$ → Debemos eliminar rec x izq.

→ $P(B)$ NO se puede calcular porque está mal construida la gramática

Otro Ejemplo (Bien escrito)

$S \to AB \mid BA$

$A \to Aa \mid \varepsilon$

$B \to Bb$

ATRÁS →

En el conjunto primero, vemos su impacto en el if-else. No puedo tener igualdad de expresiones que evalúe.

Ejemplo

$S \to AB \mid BA$

rec x izq. $\begin{cases} A \to Aa \mid \varepsilon \\ B \to Bb \mid \varepsilon \neq Bb \end{cases}$

$A \to A\alpha \mid \beta$
$A \to \beta R$
$R \to \alpha R \mid \varepsilon$

$B \to Bb$ → No se puede eliminar la rec x izq porque está mal escrito, no tiene caso base.

• Primero eliminamos la rec. x izq. para poder calcular el CP.

- Las entradas siempre se procesan de izq a derecha
- El Árbol vamos a recorrer de izq a der
- Hacemos un enfoque Top Down

① Eliminamos la rec x izq de:

- $A \to Aa \mid \varepsilon$ ($\alpha = a$, $\beta = \varepsilon$) $\quad B \to Bb \mid \varepsilon$

$\begin{cases} A \to R \\ R \to aR \mid \varepsilon \end{cases}$

$\begin{cases} B \to R_1 \\ R_1 \to bR_1 \mid \varepsilon \end{cases}$

② El nuevo sistema de Producciones

$S \to AB \mid BA$

$A \to R$

$R \to aR \mid \varepsilon$

$B \to R_1$

$R_1 \to bR_1 \mid \varepsilon$

↳ conjunto PRIMERO

$P(S) = P(AB) \cup P(BA) = \{a, b, \varepsilon\} \cup \{b, a, \varepsilon\}$

No son disjuntos, eso indica que existe una ambigüedad semántica, debo modif. la gramática

$P(A) = P(R) = \{a, \varepsilon\}$

$P(R) = P(aR) \cup P(\varepsilon) = \{a\} \cup \{\varepsilon\} = \{a, \varepsilon\}$

$P(B) = P(R_1) = \{b, \varepsilon\}$

$P(R_1) = P(bR_1) \cup P(\varepsilon) = \{b\} \cup \{\varepsilon\} = \{b, \varepsilon\}$

Los conjuntos primeros deben ser disjuntos entre sí, si no lo son es porque hay un caso de FC indirecto.

Ej: $P \to A \mid a$
$A \to B$
$B \to a$

Debo cambiar la gramática por una equivalente, pero es riesgoso

Para qué se hace Todo esto?

Una **Gramática Predictiva** es buena. → con el 1° terminal puedo predecir qué lado derecho puedo usar sin tener temor a equivocarme o sin temor a generar backtracking

- No tiene rec. x izq.
- No tiene ambigüedad sintáctica (No hay factor común x izq.)
- No tiene ambigüedad semántica (Los CP de prod compuestas son todos disjuntos entre sí)

↳ Es recomendable porque es sencilla de codificar
↳ vale la pena de hacer el esfuerzo para eliminar la rec. x izq, eliminar factor común por la izq. y tratar de corregir y eliminar cualquier ambigüedad semántica detectada por los conjuntos primeros

condiciones: Análisis sintáctico Predictivo (ASP)

La gramática debe estar libre de:

1°) Recursión por la izq. → No contiene producciones de la forma $A \to A\alpha \mid \beta$
2) Factor común directo (No tiene? Ambigüedad sintáctica) → Para un NO-terminal con varios lados der, no existe factor común por la izq. entre ellos.
3) Factor común indirecto. (No tiene? Ambigüedad semántica) → Acordarse del vacío
   → un NO-terminal con varias prod. no derive al mismo terminal por la izq. Se verifica teniendo un conjunto primero de cada prod. y verificando que sean disjuntos entre sí.

No hay una relación directa, donde se ve en la prod el problema.

Scanned with CamScanner

Ej

Escribir el código fuente del compilador simple:

Escribir el compilado
a partir de allí ir carge
la entrada y el compi
comporta con buen vali
dor de la entrada

$$\begin{cases} \text{cadena} \to \text{cadena} + \underbrace{\text{digito}}_{\alpha_1} \mid \text{cadena} - \underbrace{\text{digito}}_{\alpha_2} \mid \underbrace{\text{digito}}_{\beta} \\ \text{digito} \to 0 \mid 1 \mid 2 \mid \dots \mid 9 \end{cases}$$

Para escribir el código fuente, primero hay que validar que la gramática sea predictiva.

β = digito
α_1 = + digito
α_2 = - digito

$A \to A\alpha \mid \beta$
$A \to \beta R$
$R \to \alpha R \mid \varepsilon$

a) Eliminar la rec x la izq.

① cadena → digito R ▷ Es simple, NO puede haber factor común.
② R → + digito R | - digito R | ε ▷ NO tiene factorc. por la izq.
③ digito → 0 | 1 | ... | 9

Las uniones nos permite evaluar si tengo alguna repetición o NO

b) Factor común por izq.
NO tiene.

Necesariamente la gramática debe ser predictiva para evitar el backtracking y ambigüedad en la selección.

c) Ambigüedad semántica (conjunto Primero)

$P(\text{cadena}) = \{0,1,2\dots,9\}$

$P(R) = P(+\text{digito}R) \cup P(-\text{digito}R) \cup P(\varepsilon) = \{+\} \cup \{-\} \cup \{\varepsilon\}$

$P(\text{digito}) = \{0\} \cup \{1\} \cup \{2\} \cup \dots \cup \{9\}$

En la Est. condicional se ve reflejada la influencia del conjunto Primero

LA GRAMÁTICA ES PREDICTIVA

Si es Predictiva voy a escribir el código fuente utilizando una técnica:

- Para cada NO-terminal, asigno una función → var. sintáctica
- Utilizo una estructura condicional basada en el conjunto Primero

→ cada NO-terminal es una función

NO-terminales
- cadena
- R
- digito

→ Si tiene el lado der. compuesto, se usa la est. condicional. ▷ Es siempre sobre el conjunto Primero de la producción.

```
void cadena(){
    digito(); R();
}
```

```
void R(){
  if(input=='+'){
    match('+'); digito(); R();
  }else if(input=='-'){
    match('-'); digito(); R();
  }else{
    ;
  }
}
```

tiene 3 lados der. Por eso 3 ifs

match de la entrada

```
void digito(){
  if(input=='0'){
    match('0');
  }else if(input=='1'){
    match('1');
  }else if(input=='2'){
    match('2');
    .
    .
    .
  }else{
    match('9');
  }
}
```

tiene 9 lados der. Por eso 9 ifs.

```
main(){
    terminal input;
    input = getnextsymbol();
    cadena();
}
```

▷ Puede hacer lo que sea, por ej.. obtener el valor de input y llamar al símbolo de entrada.

cuerpo del match

```
match(terminal t){
  if(input==t){
    input = getnextsymbol(); // Avanza a la sgte entrada
  else
  }  error();
}
```

La rec. x izq es molestosa porque la función se llamará recursivamente.

El lado izq NO puede coincidir con el lado derecho que está se va a llamar infinitamente.

El **match**: su verdadera función es la vidente. Por Ej: si me estoy yendo al lado derecho, le digo que debe haber un match si hay 't', si está el 't', ya está avanza, si NO, error.

↳ Es el **manejador de errores, gestor de errores**

↳ asume que si tengo 1 o 2..., eso tiene que haber en la entrada

Si nuestra Est. condicional es **if**, siempre debe tener el **else**

↳ **switch**, siempre debe tener el **default**. Porque ahí se maneja el error.

La escritura del código fuente me va ayudar si es ambigüa o NO

↳ Es un **validador de ambigüedades**.

¿Por qué es importante escribir una gramática predictiva?

↳ Es importante para que actúe como un validador de ambigüedades.

- Escribir el código fuente que procesa la entrada definida por la sgte gramática.

expr → expr + expr | expr - expr | term

term → term * term | term / term | digito

digito → 0|1|...|9

TDS

clase 5 V

#Habiendo aceptado la entrada, implementar acciones sobre la entrada para producir un efecto de salida.

La herramienta propia del Analizador sintáctico son los árboles y las derivaciones. como entrada, esperamos BNF

Reglas semánticas

Se realiza uniendo reglas o fragmentos de un programa a las producciones de una gramática.
- Tendrá su propio conjunto de acciones o reglas que permitirán realizar acciones en la medida que avanzamos con esa derivación

ATRIBUTOS

Son variables que le podemos asociar a los NO terminales
son variables que le asocio a las variables sintácticas.

Cada NO-terminal puede tener asociado una variable, un atributo o instancia.

- Tipos
  - Sintetizados (más sencillos): son aquellos cuyo valor se obtiene por la evaluación de sus hijos en un árbol sintáctico.
    - Se calculan a partir de los atributos de los hijos en el árbol sintáctico y se propagan hacia arriba... J: chatgpt
    - Recorrido ascendente (descubrir)
  - Heredados: Son aquellos cuyo valor se puede obtener a partir del padre o de sus hermanos en un árbol sintáctico.
    - Recibe el valor de su padre

Escribir un TDS que reciba cadenas de forma infija y retorna en formato postfijo la misma cadena de entrada.

Entrada: 5+4-2

Salida: 5 4 + 2 -

cadena → digito R

R → + digito R | - digito R | ε

digito → 0 | 1 | ... | 9

En el caso de usar reglas semánticas, se procede de la sgte manera:

a) Divide y vencerás, Tratando cada producción de manera simple y definio las reglas adecuadas (separar las producciones)

BNF

cadena → digito R

R → + digito R

R → - digito R

R → ε

digito → 0

digito → 1

⋮

digito → 9

Scanned with CamScanner

BNF

cadena → digito R

R → + digito R

R → - digito R
(sirve para inicializar)

R → ε

digito → 0

digito → 1

⋮

digito → 9

Reglas semánticas (salida)

cadena.X = digito.X || R.X ;

R.X = digito.X || '+' || R.X

lado izq: R.X = digito.X || '-' || R.X
(R.X: valor de retorno)

→ Todo esto es generación de código

El símbolo '||' en la R.Sem. es el operador de concatenación

digito.X = '0'
digito.X = '1'
⋮
(variable asociada)

digito.X = 'a' → indica el valor de retorno.

El BNF y las Reglas semánticas están relacionados
El BNF escribe la función
Las R. semánticas. lo traduce en las acciones que van a generar la salida

cadena + digito
cadena - digito
digito

cadena.X || digito.X || +
cadena.X || digito.X || −
R = digito.X

entrada
ad de información mínima
iene sentido por sí sola

Debemos escribir BNF y reglas

Debe llamarle a cadena ← main( ){

A veces la recomendación es que antes de eliminar la rec xizq escribir las reglas.

```
char digito(){
    char x;
    if(input=='0'){
        match('0')
        x='0';
        return(x);
    }else if(input=='1'){
        match('1');
        x='1';
        return(x);
    }
    ⋮
    else
        match('a'); x='a';
        return(x);
    }
}
```

La función principal del match es manejar los errores
- validar

(el que está a la izq de la regla)

```
R(){
    char x;
    if(input=='+'){
        match('+'); x=digito() || '+' || R(); return(x);
    }else if(input=='-'){
        match('-'); digito(); R();
    }else{
        return(NULL); //NULL, retorna vacío
    }
}
```

↳ R deriva al vacío, desaparece.
Eso significa que la cadena se acabó

cada vez que una var sintáctica (un lado izquierdo) deriva en su lado derecho, se aplican reglas semánticas cuando empiece a recorrer el árbol, derive todo los NO-terminales de su lado derecho

El vacío es la puerta de salida

Scanned with CamScanner

BNF

cadena → digito R

R → + digito R

R → - digito R
(sirve para inicializar)

R → ε

digito → 0

digito → 1

⋮

digito → 9

Reglas semánticas (salida)

cadena.x = digito.x || R.x ;

R.x = digito.x || '+' || R.x

lado izq (R).x = digito.x || '-' || R.x
(valor de retorno)

▷ Todo esto es generado de código

El símbolo '||' en la R.Sem. es el operador de concatenación

---

digito.x = '0'
digito.x = '1'
⋮
(variable asociada)
⋮
digito.x = '9' ▷ indica el valor de retorno.

A veces la recomendación es que antes de eliminar la rec xizq escribir las reglas.

```
char digito (){
    char x;
    if (input == '0'){
        match ('0')
        x='0';
        return (x);
    }else if (input == '1'){
        match ('1');
        x='1';
        return (x);
    }.
    .
    .
    else
        match ('9'); x='9';
        return (x);
    }
}
```

▷ El que está a la izq de la regla.

```
R ( ){
    char x;
    if(input == '+'){
        match ('+'); x=digito () || '+' || R(); return (x);
    }else if (input == '-'){
        match ('-'); digito (); R ();
    }else{
        return(NULL); //NULL, retorna vacío
    }
}
```

↳ R deriva al vacío, desaparece.
Eso significa que la cadena se acabó

El vacío es la puerta de salida

El BNF y las Reglas semánticas están relacionados

El BNF escribe la función
Las R. semánticas. lo traduce en las acciones que van a generar la salida

cadena → cadena + digito
cadena → cadena - digito
cadena → digito

cadena.x = cadena.x || digito.x || +
cadena.x = cadena.x || digito.x || -
cadena.x = digito.x

Unidad de información mínima que tiene sentido por sí sola
(la entrada)

Debemos escribir BNF y reglas

Debe llamarse a cadena ← main ( ){

cada vez que una var sintáctica (un lado izquierdo) deriva en su lado derecho, se aplican reglas semánticas cuando empiece a recorrer el árbol, derive todo los NO-terminales de su lado derecho

Scanned with CamScanner

S → R
R → inst R | var D R | ε
D → int R2 | float R2
R2 → ident R3 | ε
R3 → E R2 | sep R2
E → S proc.

P(S) = {inst, var, ε}
P(R) = {inst} ∪ {var} ∪ {ε} = {inst, var, ε}
P(D) = {int} ∪ {float} = {int, float}
P(R2) = {ident} ∪ {ε} = {ident, ε}
P(R3) = P(E R2) ∪ {sep} = {inst, var, proc, sep}
P(E) = P(S proc) = {inst, var, proc}
↳ Atender!

#la gramática NO tiene Rec x izq, FC x izq ni ambigüedad semántica, por tanto, podemos decir que es predictiva

Escribir el código fuente del An. Sintáctico para validar cadenas de entrada

1) Obtener la gram en forma predictiva.
2) Escribir el código fuente utilizando la estrategia de cn:
   no_terminal = una_Función ↳ para hacer el cód fuente se debe tener en cuenta

```
S(){
  R();
}
```

```
D(){
  if(input='int'){
     match('int');
     R2();
  }else{
     match('float');
     R2();
  }
}
```

```
R3(){
  if(input='inst'||input='var'||input='proc'){
     E(); R2(); // Es un no-terminal por eso no se pone match
  }else if(input=='sep'){
     match('sep'); R2();
  }
}
```

```
R(){
  if(input='inst'){
     match('inst');
     R();
  }else if(input=='var'){
     match('var');
     D();
     R();
  }else{
     ;
  }
}
```

```
R2(){
  if(input=='ident'){
     match('ident');
     R3();
  }else{
     ;
  }
}
```

```
E(){
  S(); match('proc');
}
```

```
match(terminal t){
  if(input==t){
     input=getnextSymbol();
  else
     error();
  }
}
```

Escribir el código fuente que valida cadenas de entrada que pertenecen a la misma.

FC S → A = = B | A < B | A > B
A → not A | a A | ε
B → b B | ε

Hacer el seguimiento para not a < b ?

S → A R
R → = = B | < B | > B
A → not A | a A | ε
B → b B | ε
↳ NO es un prod compuesta

P(S) = P(A R) = {not, a} ∪ {=, <, >} = {not, a, =, <, >} (A obtiene ε directo)
P(R) = {=} ∪ {<} ∪ {>} = {=, <, >}
P(A) = {not} ∪ {a} ∪ {ε} = {not, a, ε}
P(B) = {b} ∪ {ε} = {b, ε}

```
S(){
  iA(), R();
}
```

```
R(){
  if(input=='='){
     match('='); B();
  }else if(input=='<'){
     match('<'); B();
  }else{
     match('>'); B();
  }
}
```

```
A(){
  if(input='not'){
     match('not'); A();
  }else if(input=='a'){
     match('a'); A();
  }else{
     ;
  }
}
```
↳ forzamos para que capture fuera del if en casos distintos a estos

```
B(){
  if(input=='b'){
     match('b'); B();
  }else{
     ;
  }
}
```

Scanned with CamScanner

Ejercicio

Clase 6 V

Escribir un traductor dirigido por la sintaxis que acepte un lenguaje formado por los números romanos del 1 al 10 y retorna el correspondiente número arábigo.

Ej: Entrada X, salida 10. → E: VIII S: 8

En el árbol sintáctico fluye los valores

Hago todo por caracter o por palabra, solo de una forma.

Paso 1

Escribir el BNF que acepta la gramática de entrada.

Una propuesta es la gramática (1):

NUMERO → DECENA | UNIDAD

DECENA → X | ε

UNIDAD → UNIDADMENOR | IV | V UNIDADMENOR | IX

UNIDADMENOR → I | II | III | ε

Paso 2

Agregarle las reglas semánticas → Para validar si la gramática tiene las condiciones para considerarla predictiva o tratar el FC, Rec.x izq o elci

Divide y vencerás

BNF	Reglas semánticas (salida)
NUMERO → DECENA	NUMERO.x = DECENA.x
NUMERO → UNIDAD	NUMERO.x = UNIDAD.x
DECENA → X	DECENA.x = 10
DECENA → ε	DECENA.x = 0
UNIDAD → UNIDADMENOR	UNIDAD.x = UNIDADMENOR.x
UNIDAD → IV	UNIDAD.x = 4
UNIDAD → V UNIDADMENOR (V - VIII)	UNIDAD.x = 5 + UNIDADMENOR.x
UNIDAD → IX	UNIDAD.x = '9'
UNIDADMENOR → I	UNIDADMENOR.x = '1'
UNIDADMENOR → II	UNIDADMENOR.x = '2'
UNIDADMENOR → III	UNIDADMENOR.x = '3'
UNIDADMENOR → ε	UNIDADMENOR.x = 0

↓ lado izq

Paso 3: validar que la Gramática sea Predictivo

- ¿Tiene rec. x izq?
  No tiene
- ¿Tiene factor común x izq? Si tiene

- Para el caso de UNIDAD, se elimina el FC x izq

UNIDAD → UM | VUM | IX | IV

UNIDAD → UM | V UM | IR → Eliminamos FC. x izq.
R → X | V

BNF	Regla Semántica
UNIDAD → UNIDADMENOR	UNIDAD.x = UNIDADMENOR.x
UNIDAD → V UNIDADMENOR	UNIDAD.x = 5 + UNIDADMENOR.x
UNIDAD → IR	UNIDAD.x = R.x − 1
R → X	R.x = 10
R → V	R.x = 5

- Para el caso de unidad menor, se debe eliminar el FC x izq.

UNIDADMENOR → I | II | III | ε

UNIDADMENOR → I Y_1 | ε (FC)

Y_1 → I Y_2 | ε (FC)

Y_2 → I | ε

- con el D y V.

UNIDADMENOR → I Y_1	UNIDADMENOR.x = 1 + Y_1.x
UNIDADMENOR → ε	UNIDADMENOR.x = 0
Y_1 → I Y_2	Y_1.x = 1 + Y_2.x
Y_1 → ε	Y_1.x = 0
Y_2 → I	Y_2.x = 1
Y_2 → ε	Y_2.x = 0

Paso 1: Escribir el BNF que acepta la gram. de entrada
Paso 2: Agregarle reglas semánticas. D y V
Paso 3: Validar que la G. sea Predictiva. ← Rec. x izq, FC x izq, Amb. Semántica conjunto P.

Scanned with CamScanner

Resultado final

BNF	Reglas semánticas	
NUMERO → DECENA	Numero.x = DECENA.x	P() = {x, ε}
NUMERO → UNIDAD	Numero.x = UNIDAD.x	P() = {I, ε, V, I}
DECENA → X	Decena.x = 10	P() = {x}
DECENA → ε	Decena.x = 0	P() = {ε}
UNIDAD → UNIDADMENOR	UNIDAD.x = UNIDADMENOR.x	P() = {I, ε} → No es disjunto
UNIDAD → V UNIDADMENOR	UNIDAD.x = 5 + UNIDADMENOR.x	P() = {v} → Tenemos amb. semántica.
UNIDAD → IR	UNIDAD.x = R.x - 1	P() = {I}
R → X	R.x = 10 → (Anteriormente era 9, por eso se resta -1)	P() = {x} ✓
R → V	R.x = 5	P() = {v}
UNIDADMENOR → I Y1	UNIDADMENOR.x = 1 + Y1.x	P() = {I}
UNIDADMENOR → ε	UNIDADMENOR.x = 0	P() = {ε}
Y1 → I Y2	Y1.x = 1 + Y2.x	P() = {I} ≠ V
Y1 → ε	Y1.x = 0	P() = {ε}
Y2 → I	Y2.x = 1	P() = {I}
Y2 → ε	Y2.x = 0	P() = {ε}
Hallamos el conjunto primero → De abajo Arriba		# Los que están en bloques tienen que ser disjuntos entre sí.

Cuando la ambigüedad es semántica, solo resta modificar las reglas. Debemos modificar las reglas tratando de crear un sistema de producción equivalente. La ambigüedad semántica ocurre cuando hay factor común indirecto, para resolver ese problema tenemos que empezar a derivar hasta que estén en un solo lado derecho todos los problemas y ahí factorear

cuando se modifica la Gramática, se debe evaluar las reglas de vuelta todas las que están alcanzadas.

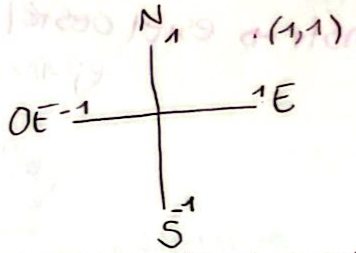
Para las reglas semánticas es recomendable utilizar el árbol para evitar problemas.

Scanned with CamScanner

Ejercicio

Predictivo ▷ sin rec x izq ▷ sin FC ▷ si FC IND.

Escribir un TDS que permita dar una secuencia de instrucciones a un robot que se puede mover al este, oeste, Norte o sur y que retorna su posición final.



Se propone la gramática sgte:

sec → sec instr | inicio

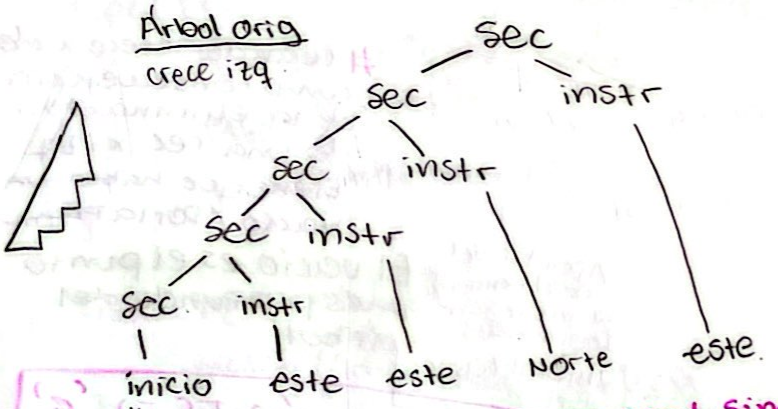
instr → este | oeste | Norte | sur.

ej: inicio este norte, retorna (1,1)

Entrada: inicio este este norte este

Hacer el árbol sintáctico podremos visualizar (explicar) como va a fluir las derivaciones en los lados derechos

Árbol orig crece izq



a) se puede plantear verificar el árbol sintácitico de la entrada pretendida

Para este caso hay rec. x izq en 'sec → sec instr | inicio

• Antes de eliminar la Rec x izq Hacer las Reglas Semánticas. RECOMENDABLE, luego debemos encontrar la Gramática Equivalente Predictiva

Separar las producciones en prod. simples

BNF	Reglas semánticas
sec → sec1 instr	sec.x = sec1.x + inst.dx ; sec.y = sec1.y + instr.dy
sec → inicio	sec.x = 0 ; sec.y = 0
instr → este	instr.dx = 1 ; instr.dy = 0
instr → oeste	instr.dx = −1 ; instr.dy = 0
instr → Norte	instr.dx = 0 ; instr.dy = 1
instr → sur	instr.dx = 0 ; instr.dy = −1

La gramática NO es predictiva ya que existe rec. x izq

A → Aα | β
A → βZ
R → αR | ε

dx incremental q se va a mover

• Eliminar la rec. x izq.

sec → sec instr | inicio
α β

sec → inicio R
R → instr R | ε
instr → este | oeste | Norte | sur

• No tiene factor común
• ver el conjunto primero

P(sec) = P(inicio) = {inicio}

P(R) = P(instr) ∪ P(ε) = {este, oeste, norte, sur} ∪ {ε}
= {este, oeste, Norte, sur}

P(instr) = P(este) ∪ P(oeste) ∪ P(Norte) ∪ P(sur)
= {este} ∪ {oeste} ∪ {Norte} ∪ {sur} = {este, oeste, Norte, sur}

Scanned with CamScanner

BNF

sec → inicio R

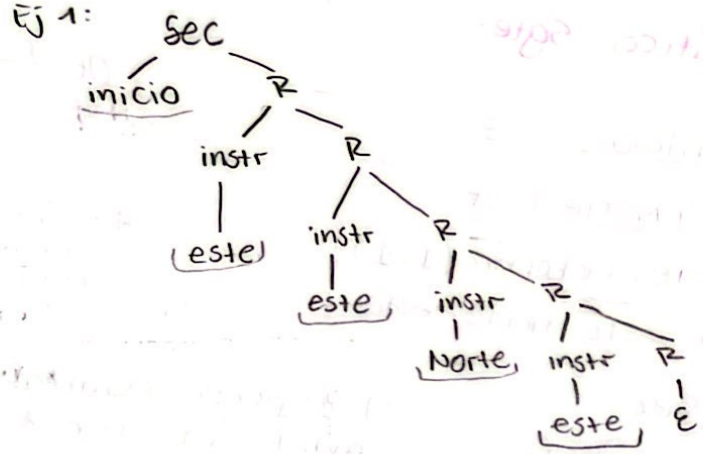
R → instr R | ε

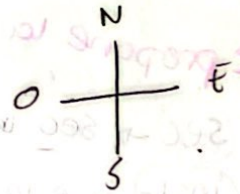
instr → este | oeste | norte | sur

Ej 1: inicio este este norte este

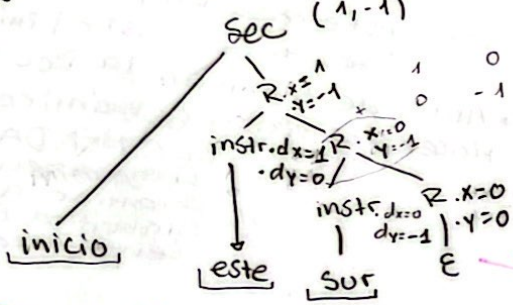
#si el árbol NO crece por la izq NO tiene rec x izq.

Ej 1:





Ej 2: inicio este sur (1,-1)



Resultado final.

Cuando crece x der como consecuencia de la eliminación de una rec x izq tiene que haber un vacío (por la fórmula!)

A partir del cual comienzo a procesar los nodos.

El vacío es el punto más profundo del árbol.

inicializamos en ε

LA INICIALIZACIÓN ES EL 'ε'

BNF	Reglas Semánticas
sec → inicio R	print(R.x ; R.y)
R → instr R_1	R.x = R_1.x + instr.dx ; R.y = R_1.y + instr.dy
R → ε	R.x = 0 ; R.y = 0
instr → este	instr.dx = 1 ; instr.dy = 0
instr → oeste	instr.dx = -1 ; instr.dy = 0
instr → norte	instr.dx = 0 ; instr.dy = 1
instr → sur	instr.dx = 0 ; instr.dy = -1

incremental

Cód fuente

```
void sec(){
    int x, y
    match('comienza');
    R(x,y)
    print(x,y)
}
```

```
void R (int *x, int *y){
    int dx, dy
    if(input=='norte' || input=='sur'....){
        instr(dx,dy)
        R(x,y)
        *x = *x + dx
        *y = *y + dy
    }else{
        *x=0
        *y=0
    }
}
```

```
void instr (int *dx, int *dy){
    if(input=='norte'){
        match('norte');
        *dx=0;
        *dy=1;
    }else if
    :
    }else{
        match('oeste');
        *dx=-1;
        *dy=0;
    }
}
```

Scanned with CamScanner

EJERCICIO PRÁCTICO

Dado el sgte programa que se presenta a continuación, se pide:

a) Obtener el TDS de origen.

b) Describir en lenguaje Natural la funcionalidad del mismo

c) La gramática de origen, es predictiva? JUSTIFICAR

```
declaración(){
    tipo(); print(lista());
}

tipo(){
    if(input == "int"){
        match("int");
    else
        match('float');
    }
}

lista(){
    match('id'); return(1+R());
}

R(){
    if(input==","){
        match(","); match("id"); return(1+R());
    }else{
        return(0);
    }
}

match(char*t){
    if(input==t){
        input=obtenerSgTerminal();
    else
        reportar("Error de sintaxis");
    }
}
```

No terminal

Posible BNF

declaración → tipo lista

tipo → int | float

lista → id R

R → , id R | ε

Tiene forma de que se eliminó por izq

Para función

AAT

No hay terminal y No hay match es un vacío 'ε'

return(0); → Es asignar valor al atributo de la izquierda

P/ BNF

declaración() es un NO terminal será lado izq, todo lo que hay dentro será el lado derecho. Si hay una est. condicional, será compuesto (lado der.) Si no hay, entonces será simple.

• Si hay terminales habría match.

Si hay return.... se concatena

• Puede ser match o un No terminal (llamada de función.) o un ';'.
Si no aparece entonces es 'ε' por más que haya una operación allí

Scanned with CamScanner

BNF
código que quedó

declaración → tipo lista
tipo → int | float
lista → id R
R → , id R | ε

variables sintetizadas.

1°) DyV

BNF	
declaración → tipo lista	print(lista)
tipo → int	—
tipo → float	—
lista → id R_1	lista.x = 1 + R_1.x
R → ; id R_1	R.x = 1 + R_1.x
R → ε	R.x = 0 (valor de retorno)

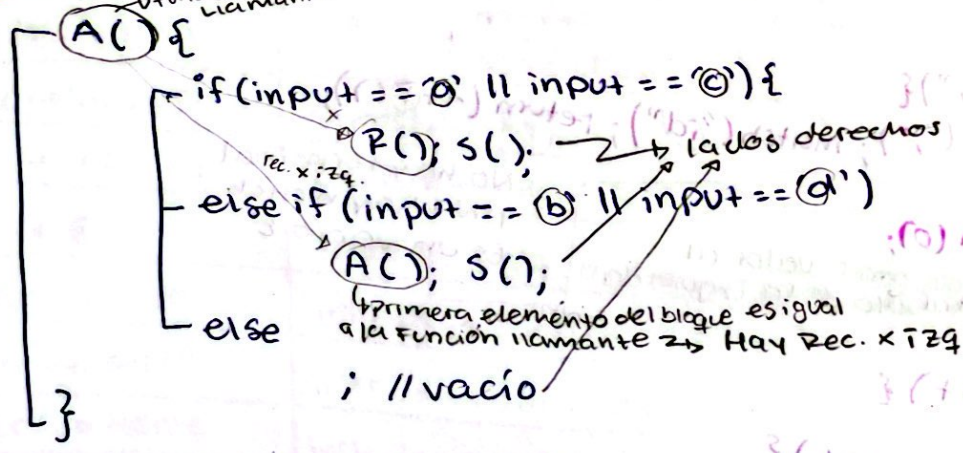
TAA(V)

¿Es predictiva la sgte gramática? Justificar

lenguaje de entrada.

↳ Dependerá de todas las funciones

↳ Para saber si la gramática es predictiva, debo evaluar cada una de las funciones. Puede que una función NO sea predictiva.



#Para este tipo de ejercicio

↳ Rápidamente debemos mirar

↳ las condiciones de los ifs NO deben ser repetidas
a, c, b, d son disjuntos, no hay problema ✌ → No falló el conjunto primero

↳ Mirar los bloques que están bajo las condiciones
R() S() y A() S() → mirar si hay Rec x izq, ¿cómo hay? El primer elemento de cada bloque sea una llamada a función de la función llamante.
→ buscar FC o Rec x izq.

Scanned with CamScanner

TAA V1

Escribir el código fuente de un TDS predictivo, en el que se recibe un lenguaje definido por el sgte BNF y retorna el valor que se menciona:

Lenguaje de entrada (BNF):

$S \rightarrow cAd$

$A \rightarrow ab \mid a$

$A \rightarrow a$

$A \rightarrow ab$

$\Sigma = \{a, b, c\}$

Lenguaje de salida:

Retorna TRUE, si la cantidad de a es igual a la cantidad de b.

False, en otro caso.

Ejemplo: → Entrada: cabd salida: TRUE

1°) Rec izq

- NO tiene

2°) FC x izq ✓

BNF	Reglas
$S \rightarrow CAd$	Print (A.x)
$A \rightarrow aR$	A.x = R.x
$R \rightarrow b$	R.x = TRUE
$R \rightarrow \varepsilon$	R.x = FALSE

```
S ( ){
    match('c'); A(); match('d')
}
```

```
A(){
    match('a'); R();
}
```

```
R (){
    if(input = 'b'){
        match (b)
        Print (TRUE),
    }else{
        print (False);
    }
}
```

```
match (terminal t){
    if (input == t){
        input = sgteTerminal()
    }else{
        reportar ('error')
    }
}
```

Scanned with CamScanner

Reglas semánticas (RS)

clase 8 V

Producción	Reglas Semánticas
expr → expr₁ + term	expr.t = expr₁.t ‖ term.t ‖ '+'
expr → expr₁ - term	expr.t = expr₁.t ‖ term.t ‖ '-'
expr → term	expr.t = term.t
term → 0	term.t = '0'
term → 1	term.t = '1'
⋮	⋮
term → 9	term.t = '9'

return de lo que está a la derecha

‖ → símbolo de concatenar

Esquema de Traducción
↳ es una notación p/ unir fragmentos de un programa a las producciones de una gramática

Diferencia Fundamental entre Acciones y Reglas Semánticas

→ son más cortas

Las acciones son exactamente lo mismo que las reglas NO hay una dif. fundamental. LA ÚNICA dif. es que ES UNA REGLA que se inserta en la producción.

 # LAS ACCIONES SEMÁNTICAS (AS)
Definen lo que sería un Esquema de Traducción
↳ Frag. de código a ejecutar durante el análisis de la cadena de entrada.

LAS REGLAS SEMÁNTICAS
↳ Definen un par: Producción – Reglas
es más fácil su manejo

En las RS existe la concatenación
En las AS NO existe concatenación

	ACCIONES
expr → expr₁ + term	{print('+')}
expr → expr₁ - term	{print('-')}
expr → term	
term → 0	{print('0')}
term → 1	{print('1')}
⋮	
term → 9	{print('9')}

A → Aα | β
A → βR
R → αR | ε

ESQUEMA DE TRADUCCION

↳ NO utiliza atributos ... .t, .x, las RS sí utilizan.

ACCIÓN PRINCIPAL de las RS
↳ la concatenación

ACCION PRINCIPAL del Esq. Trad
↳ El print se usa la salida como variable.

- Para escribir el cód. fuente, se necesita contar con un Esquema de Traducción Predictivo → Elim. rec x izq.

expr → expr₁ + term | expr₁ - term | term
(α₁ = + term, α₂ = - term, β = term)

expr → term R

R → + term {print('+')} R
R → - term {print('-')} R
R → ε
term → 0 {print('0')}
⋮
term → 9 {print('9')}

código fuente

```
expr(){
    term(), R();
}

R(){
  if(input == '+'){
    match('+') term(); print('+'), R();

  else if(input == '-')
    match('-'), term(); print('-'), R();

  else
  }   ;
  }

term(){
  if(input == '0'){
     match(0)
  else if ..
  :
  else
```

Scanned with CamScanner

#La ventaja de una ACCIÓN es que forma parte de la producción

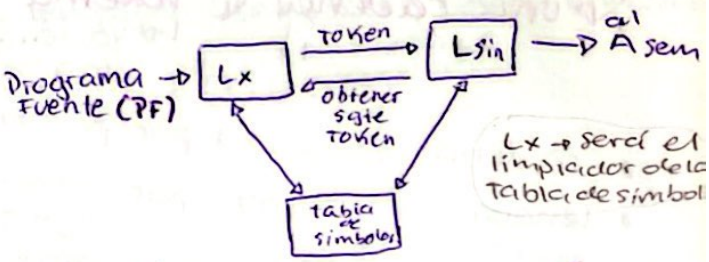
- Están insertadas dentro de la producción, cualquier fórmula que aplique a las producciones las reglas se van dentro.
  - Eliminar rec. x izq, la acción y aseva dentro

cap 3

Análisis Léxico

LIN → [Lx]

En un compilador, el Léxico actúa directamente con el PF. El sintáctico NO pero sí en el Traductor



→ A Lx: lo que hace es procesar palabras. Verifica que los caracteres sean parte del alfabeto (interactúa con el alfabeto y evalúa la pertenencia a una clase)

Definiciones

- **Lexema**: Es la secuencia de caracteres del PF (programa fuente), que coincide con un patrón
  - ↳ Es una instancia del token
- **Token**: Es un nombre que define una clase o tipo de patrón. Puede ser identificado por el nombre o como un par ordenado compuesto del nombre del token y del valor mismo.
- **Patrón**: Es una descripción, generalmente en lenguaje natural, de que lexemas pertenecen a un token. Es la definición del token. Puede utilizarse además esquemas de definición por extensión o comprensión.

clave Token	descripción de una clase Patrón	Lexema. Palabra de entrada
número	cualquier cte numérica	3,14159 | 0 | 25 | ...
comparación	< o > o <= o >= o == o !=	==, !=
ELSE	La letra e seguida de la l, la s y la e.	else

Lexema: unidad de entrada que tiene sentido por sí misma

Token: clase

Patrón: Dice a qué clase pertenece un lexema.
↳ Descripción de la condición que tienen que cumplir las palabras para pertenecer a un token clase dada

→ Los patrones NO deben ser ambiguos, es decir, las descripciones no deben chocar.

Para resolver ambigüedad de los patrones de un lenguaje natural, se aplican las ER (Expresiones Regulares)

ER → A. Lx

BNF → A. Sn

¿Por qué ER y NO BNF?
Porque lo que hace el BNF es definir los roles de las palabras.

Un lexema tiene que pertenecer ÚNICA y exclusivamente a un token, una clase

- **Alfabeto**: conjunto finito de símbolos. Ej: {0,1}, letras, dígitos, código ASCII
- **Cadena sobre un alfabeto**: secuencia finita de símbolos obtenidos a partir de un Alfabeto. Su longitud es la cantidad de símbolos. La cadena "universidad" tiene longitud = 11, el ε = 0.
- **Lenguaje**: Cualquier conjunto contable de cadenas definidas sobre un alfabeto. Pueden ser definidas por extensión o comprensión.

Scanned with CamScanner

La función match tiene (getnextsimbol). Esa función lo hace el analizador sintáctico?

↳ una cadena y lexema NO son lo mismo
Puede ser un lexema. ↳ ya tengo certeza que va ser una palabra

- cadena reemplazo por palabra
- lenguaje reemplazo por (token)
  sinonimo de palabra cuando satisface el patrón

Operación sobre lenguajes

Operación	Definición y NOTACIÓN
Union de L y M	$L \cup M = \{s \mid s \text{ está en } L \text{ o } s \text{ está en } M\}$
concatenación de L y M	$LM = \{st \mid s \text{ está en } L \text{ y } t \text{ está en } M\}$
cerradura de Kleene de L	$L^* = \bigcup_{i=0}^{\infty} L^i$
Cerradura positivo de L	$L^+ = \bigcup_{i=1}^{\infty} L^i$

Sea L el conjunto de letras = {a,b,c,...z} y D el conjunto de digitos = {0,1,...,9} entonces:

$L \cup D = \{a, b, c, \ldots, 0, 1, 2 \ldots\}$

$LD = \{a0, a1, a2, c8, c4, d0\}$

$L^4 = \{aaaa, aaab, aaac, \ldots, azzz, bbbb \ldots\}$

D^+:

$L(L \cup D)$

EL BNF Y ER No validan, solo el árbol valida la frase.

UNA ER (expreg.) → es una NOTACIÓN (Pasivas → No validan, No tienen retorno)
↳ Describe qué condiciones debe satisfacer la entrada para convertirse en un lexema válido de pertenecer a una clase, a un token.

BASE: Hay dos reglas que forman la base:

1) ε es una ER, y $L(\varepsilon)$ es $\{\varepsilon\}$; es decir, el lenguaje cuyo único miembro es la cadena vacía.

2) Si a es un simbolo en Σ, entonces a es una ER, y $L(a) = \{a\}$, es decir, el lenguaje con una cadena, de longitud uno, con a en su única posición.

Scanned with CamScanner

INDUCCIÓN: Hay 4 partes que constituyen la inducción, mediante la cual las ER más grandes se construyen a partir de las más pequeñas. Suponga que r y s son ER que denotan a los lenguajes L(r) y L(s), respectivamente.

1) (r)|(s) es una ER. denota el lenguaje L(r) ∪ L(s)
2) (r)(s) denota L(r) L(s)
3) $(r)^*$ denota $(L(r))^*$
4) (r) es una ER que denota a L(r). Esta última dice que podemos agregar pares adicionales de paréntesis alrededor de las expresiones sin cambiar el lenguaje que denotan.

Clase 9

Extensiones de las ER.

→ Manera de describir las condiciones para que un lexema pertenezca a una clase.

- Una o más instancias. cerradura positiva de kleene. Representado con el operador +. Así, $r^+ = r r^*$, y $r^* = (r^+ | \varepsilon)$
- cero o una instancia. Representado con el operador? Así es posible establecer que $r? = r | \varepsilon$ → puede darse una o ninguna instancia de la base
- Clase de caracteres. Es una abreviación para representar una secuencia de símbolos. Ej: $(a|b|c \ldots |z) = [a-z]$

ER → Leyes algebraicas

Ley	Descripción
$r \mid s = s \mid r$	| es conmutativo
$r \mid (s \mid t) = (r \mid s) \mid t$	| es asociativo
$r(st) = (rs)t$	La concatenación asociativa
$r(s \mid t) = rs \mid rt$; $(s \mid t)r = sr \mid tr$	La concatenación se distribuye sobre
$\varepsilon r = r\varepsilon = r$	ε es la identidad p/ la concaten.
$r^* = (r \mid \varepsilon)^*$	ε se garantiza en una cerradura
$r^{**} = r^*$	* es idempotente

Scanned with CamScanner

Ejemplos de ER

1) Todas las cadenas de las letras minúsculas que comiencen y terminen con la letra 'a'.

$a([a-z]^*a)?$

- a: condición de que empiece la cadena
- $([a-z]^*a)?$: podemos aceptar las cadenas unitarias la 'a' solo cumple

2) Cadenas de a's y b's que no tengan tres b's consecutivas.

$$a^*b?(ab|ba)^*b?a^*$$

3) Cadenas de a's y b's que terminan en 'a', 'aa', o 'ab'.

$$(a|b)^*a(a|b|\varepsilon)$$

4) Cadenas de a's y b's que no poseen un número impar de a's consecutivas seguidas de un número impar de b's consecutivas

$$(aa|b)^*(a|bb)^*$$

Scanned with CamScanner

BNF
Def Reg

Definiciones Regulares

Ej. I → Forma de como voy a presentar la Notación que describe a los lexemas que puedo esperar en la entrada

El traductor NO tiene Analizador Léxico

BNF
A. Sin. Analizador Sintáctico → NO ve terminales ve Tokens

es token / es token

```
instr → if expr then instr
      | if expr then instr else instr
      | ε  → NO es terminal
expr → term oprel term
      | term
term → id
      | numero
```

Si no hay terminales, el lado derecho es un Tokens o ε.

BNF → Describe el lenguaje de entrada en términos de producciones?

El Analizador Sintáctico NO ve terminales, ve Tokens

Def Regular
A. Lex Analizador Léxico

```
digito → [0-9]
digitos → digito+
numero → digitos (.digitos)? ( E [+-]? digitos)?
letra → [A-Z a-z]
id → letra (letra | digi
```

clase: if → if → Exp Reg describe que lexema pertenece a ese token

↳ concatenación de i

```
then → then
else → else
oprel → < | > | <= | >= | = | <>
```

se describe por extensión cada lexema

↳ concatenación

→ Para cada Def Reg debo hacer un diagrama de estado.

Def Regular

digito → [0-9]

↳ Token (una clase) ↳ Expresión Regular

↳ se usa para definir, describir, utilizar una Notación que me permita entender a que clase pertenece un lexema.

En el BNF
↳ NO hay terminales
Si no hay terminales en su lado derecho, el analizador léxico tiene que hacerse cargo de esas pseudoterminales
El sintáctico NO ve terminales.

```
expr → expr op term | term → BNF: juego de Tokens
op → + | -
term → 0 | 1 | ... | 9   Def. Regular
```

```
expr → expr + term | expr - term | term
term → 0 | 1 | 2 | 3 | ... | 9
```

Si aparece explícitamente es trabajo p/ un Traductor (terminal)

TDS es un compilador simple

Esta gramática está preparada para ser procesada por un TDS, porque tiene producciones en donde se combinan terminales y NO terminales
↳ lexemas y Tokens

Si en una BNF aparece un terminal (lado der) el sintáctico está viendo el Prog. Fuente, entonces es un TDS. Si NO ve el Prog Fuente hay un léxico entre él y el programa Fuente, (puro Tokens)

Compilador (BNF y DR están separados)
Si tiene BNF y Def. Reg. es trabajo para un compilador
Todo tiene que ser Def. Regular Sin Terminales en el BNF para que sea procesado por un compilador Tradicional

Scanned with CamScanner

BNF + Def Reg

Definiciones Regulares

Ej. I → Forma de como voy a presentar la Notación que describe a los lexemas que puedo esperar en la entrada

El traductor No tiene Analizador Léxico

BNF → A. Sin. (Analizador Sintáctico) → No ve terminales ve Tokens

```
instr → if expr then instr          (des token, es token)
        | if expr then instr else instr
        | ε   → NO es terminal
expr → term oprel term
     | term
term → id
     | numero
```

Si no hay terminales, el lado derecho es un Tokens o ε.

BNF 2 → Describe el lenguaje de entrada en términos de producciones?

El Analizador Sintáctico NO ve terminales, ve TOKENS

Def Regular → A. Lex (Analizador Léxico)

```
digito → [0-9]
digitos → digito+
numero → digitos (.digitos)? ( E [+-]? digitos)?
letra → [A-z a-z]
id → letra (letra | digito)*        (clase)
if → if   → Exp Reg describe que lexema pertenece a ese token
          → concatenación de i y f
then → then
else → else
oprel → < | > | <= | >= | = | <>     (se describe por extensión cada lexema; concatenación)
```

→ Para cada Def Reg debo hacer un diagrama de estado.

El traductor es dirigido por la sintaxis por eso la Sn está en contacto con la entrada pero NO tiene una definición Regular.
→ Para usar un compilador buscar BNF y Def. Reg

egular

[0-9] → Expresión Regular

(clase)

→ se usa para definir, describir, utilizar una Notación que me permita entender a que clase pertenece un lexema.

En el BNF
↳ NO hay terminales
Si no hay terminales en su lado derecho, el analizador léxico tiene que hacerse cargo de esas pseudoterminales
El sintáctico NO ve terminales.

```
expr → expr op term | term → BNF: juego de tokens
op → + | -
term → 0|1|...|9        Def. Regular
```

```
expr → expr + term | expr - term | term
term → 0|1|2|3|...|9
```

+ y - → Si aparece explícitamente es trabajo p/ un traductor (interminal)

TDS es un compilador simple

Esta gramática está preparada para ser procesada por un TDS, porque tiene producciones en donde se combinan terminales y NO terminales
↳ lexemas y tokens

Si en una BNF aparece un terminal (lado der) el sintáctico está viendo el Prog. Fuente, entonces es un TDS. Si no ve el Prog Fuente hay un Léxico entre él y el programa Fuente, (puro Tokens)

Compilador (BNF y DR están separados)
Si tiene BNF y Def. Reg. es trabajo para un compilador
Todo tiene que ser Def. Regular Sin Terminales en el BNF para que sea procesado por un compilador Traductor

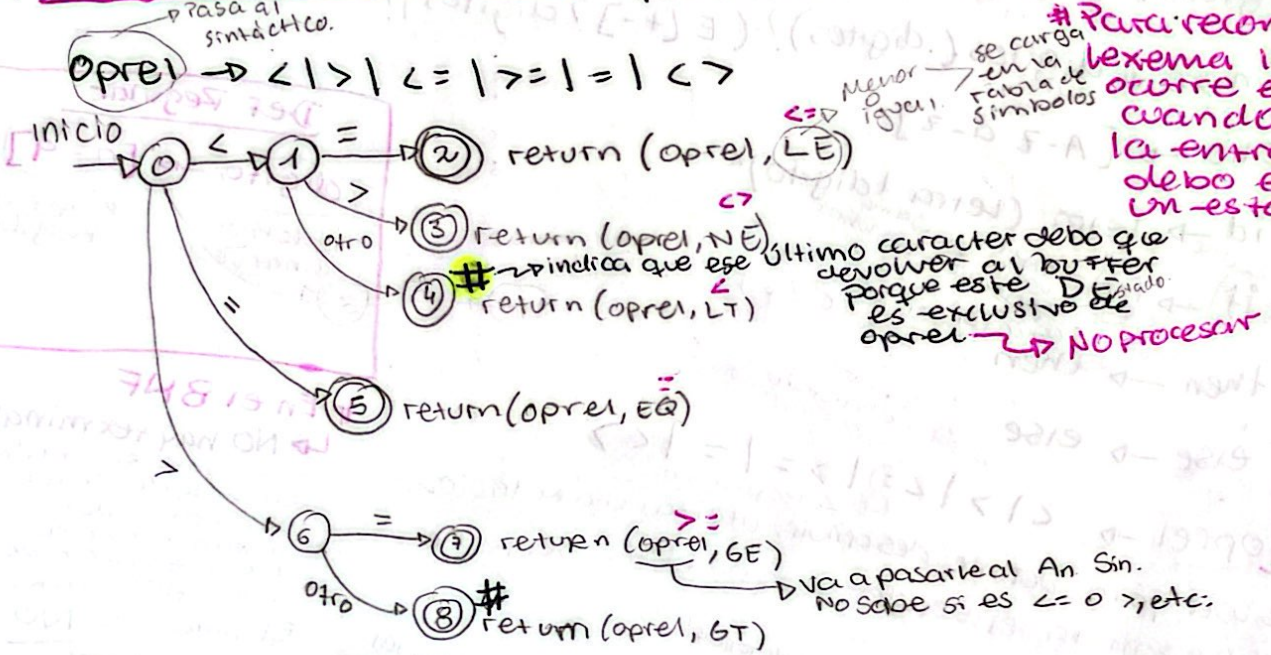
Scanned with CamScanner

El BNF NO ve el leng de entrada.
El léxico intercambia tokens con el sintáctico.

- Si quiero que un compilador tradicional procese un lenguaje de entrada, BNF NO debe tener terminales (Puede tener ε, porque no forma parte del leng de entrada)
- El BNF tiene combinación de lexemas y tokens (si

Autómatas → Herramienta para evaluar si un lexema pertenece o NO a un token.

(DE) **Diagramas de estado** → Permite evaluar cuando un lexema pertenece a un token.



→ Hay N diagramas de estado para una misma Exp. Reg.

→ Usamos Diagramas de Estado para ver como procesar la entrada.
↳ Leen la entrada y validan si pertenecen o NO a esas clases o token.

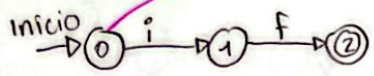
→ Tendremos un diagrama de estado por cada producción que tenga la definición regular

↳ Def. Reg → Es si el lado derecho es estrictamente una definición regular

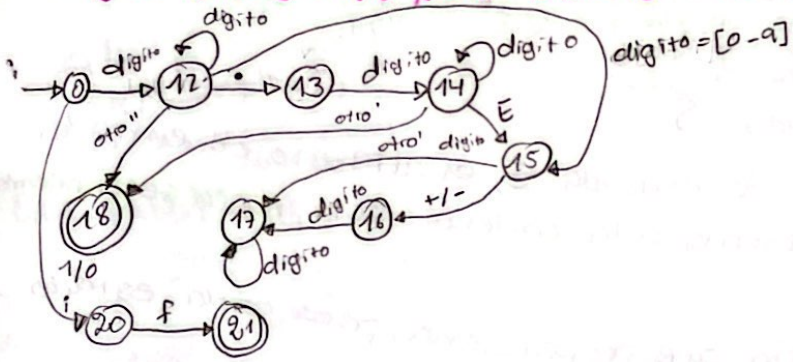
↳ voy a crear un diagrama de estado por producción

Diagrama de estado

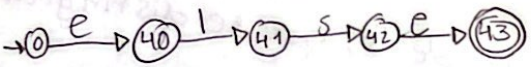
if → if. → Es el mismo para todas las Def. Reg.



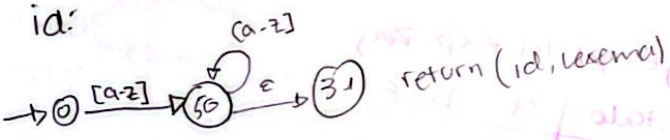
numero → digito (.digitos)?(E[+-]?digitos)?



else:
inicio



id:



Ej I

oprel → <|>|<=|>=|=|<>

↳ Puede ser sintáctico o léxico, puede ser sintáctico de un TDS puede ser un BNF para un traductor o puede ser una Def. Reg para un compilador → ¿Qué conviene que sea?

- Si es BNF p/ traductor → Debo hacer factor común... El factor común en el léxico NO me molesta

Ambigüedad en el léxico? El problema es entre dos tokens, un lexema podría pertenecer a más de un token (clase)

NO cuando el lexema tiene partes comunes con otro lexema de la misma clase → oprel → <|>|<=|>=|=|<> (1, 2, 3) 1,2,3 NO es factor común porque pertenecen a la misma clase

● Hay un solo estado inicial, se repite el '0' y NO los estados intermedios

Toda la Def. Reg está interconectada con un mismo estado inicial

#

#La ambigüedad que molesta en el léxico es cuando un lexema puede pertenecer a más de una clase, no cuando el lexema tiene partes comunes con otro lexema que pertenece a la misma clase.

Scanned with CamScanner

Def. Regulares

El diagrama de estado resuelve la ambigüedad

Ambigüedad: Nunca es en relación a su lado derecho, si no, es entre expresiones regulares. Dos Exp Reg pueden ser ambiguas entre sí

switch: cada case es un estado, pasa de acuerdo al valor de la Transición

AUTÓMATAS FINITOS NO Deterministicos (AFN)

Un AFN consiste en: (con un símbolo de Σ puedo irme a uno o más estados.)

1) Un conjunto finito de estados S

2) Un conjunto de símbolos de entrada Σ, el alfabeto de entrada, Suponemos que ε no que representa a la cadena vacía, nunca será miembro de Σ.

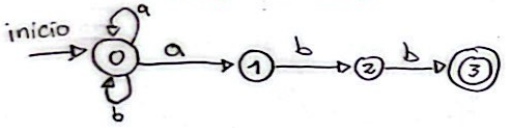
3) Una función de Transición que proporciona, para cada estado y el símbolo en Σ ∪ {ε}, un conjunto de estados sgtes.

4) Un estado s_0 de S, que se distingue como el estado inicial.

5) Un conjunto de estados F, un subconjunto de S, que se distinguen como los estados aceptantes (o estados finales)

Ej
AFN que reconoce el lenguaje p/ la exp reg

$(a|b)^* abb$



Estado	a	b	ε
0	{0,1}	{0}	∅
1	∅	{2}	∅
2	∅	{3}	∅
3	∅	∅	∅

→ La EReg. NO valida la entrada, que un lexema pertenezca al Token, una EReg. describe qué lexemas pueden pertenecer a un token pero NO hace la validación

→ Autómata: Herramienta Activa que valida, recorre la entrada y termina diciendo si pertenece o NO la entrada a una clase(token)

↳ Si termina en un estado final (pertenece) (3)

El código del autómata es el código del Analizador Léxico

Scanned with CamScanner

clase 11

Algoritmo de Thompson

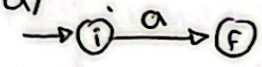
En el ámbito del Léxico.

CONSTRUCCIONES de THOMPSON

→ Aplicar las construcciones es para construir el Diagrama de Estados

↳ Permite obtener un AFN a partir de una EReg.

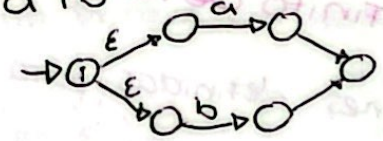
a)



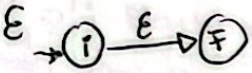
$a^+ = aa^*$

$a? = (a|\varepsilon)$

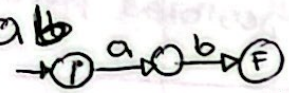
a|b



ε

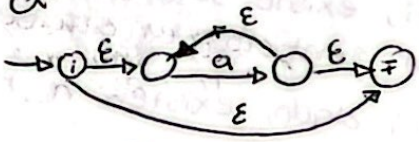


ab



↳ NO se agrega un estado más para unir

a^*



★ Detectar la ambigüedad

Thompson garantiza

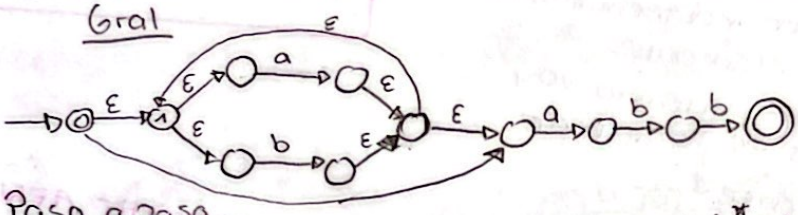
↳ Que si el AFN está bien construido, valida esa EReg.

En Thompson

Solamente sobre el vacío → NO hay indeterminismo sobre el alfabeto (una 'a' no puede ir de un estado a otras dos)

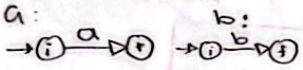
EReg: $(a|b)^*abb$

Gral



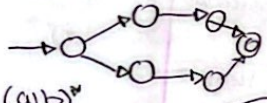
Paso a paso

a:

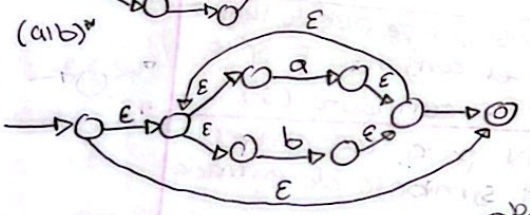


b:

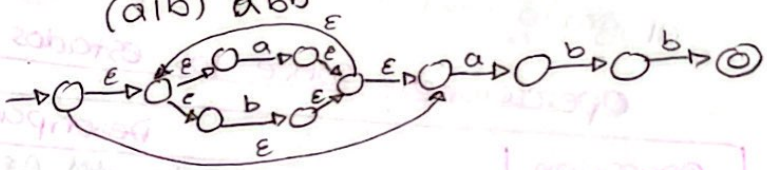
a|b



$(a|b)^*$



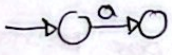
$(a|b)^*abb$



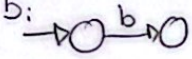
↳

Thompson de → $a^*\,b?\,(ab|ba)^*\,b?\,a^*$

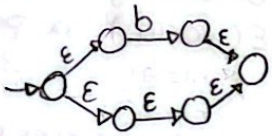
a:



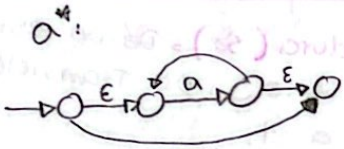
b:



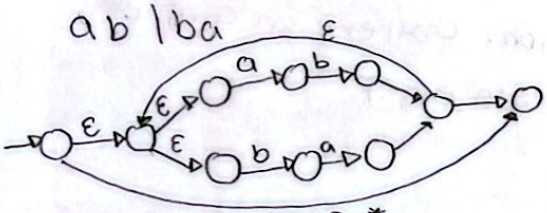
b?



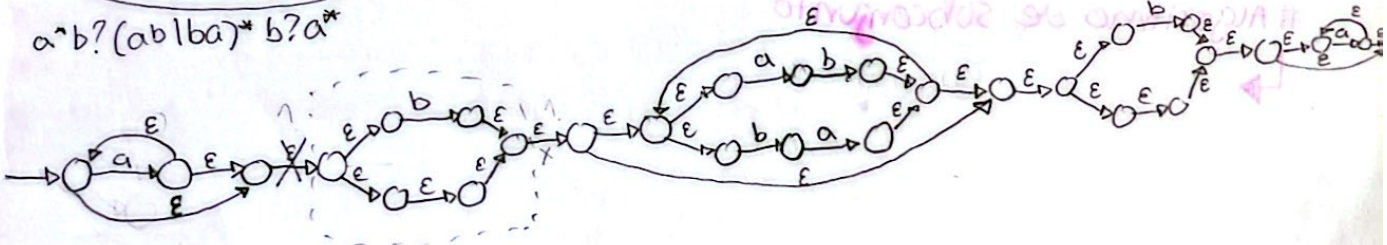
a^*:



ab|ba



$a^*b?(ab|ba)^*b?a^*$



El AFN tiene el indeterminismo sobre el vacío (problema) clase 11 V

AFD → Autómata Finito determinista. → No hay indeterminismo

a) No tiene Transiciones definidas para el 'ε'

b) Dado un estado i, existe sólo una Transición que sale de i a un estado sgte para un símbolo del alfabeto a. → No puede haber 2 T para un mismo símbolo desde un estado

c) Para un estado i dado, existen n transiciones posibles para n símbolos diferentes del alfabeto.

AFD es mucho más predictivo que un AFN

#se debe eliminar las Transiciones vacías
se debe eliminar cualquier indeterminismo que pudiera haber sobre elementos del Σ.
↳ Si se elimina el indeterminismo voy a estar en condiciones de programar el grafo muy fácilmente

Operaciones sobre los estados del AFN

convertir AFN a AFD

Operación	Descripción.
ε-cerradura (s)	conjunto de estados del AFN a los que se puede llegar desde el estado s del AFN, solo en Transiciones
ε-cerradura (T)	conjunto de estados del AFN a los que se puede llegar desde cierto estado s de AFN en el conjunto T, sólo con las transiciones ε; = Us en T ε-cerradura (s)
mover (T,a)	conjunto de estados del AFN para los cuales hay una Transición sobre el símbolo de entrada a, a partir de cierto estado s en T.

1) ε-cerradura (So) = De un estado obtiene/genera un conjunto de estados accesible sólo por Transiciones ε (vacías) a partir de ese estado incluyendole a él. Todos los estados alcanzables

2) ε-cerradura - (T): un conjunto de estados accesibles sólo por Transiciones ε para cada estado de T.

3) mover (T, a): Es la función de Transición. Obtiene un conj de estados accesibles por a ε Σ, para cada estado en T
↳ consume la entrada.

Algoritmo de subconjunto
↳

Algoritmo de Subconjunto → [redacted]

Inicializa Destados con ε-cerradura(s_0), (consumidor de ε; → Estado inicial)

```
while (hay estados sin marcar en Destados) {
    marcar (T)
    for (para cada a en Σ) {        // cada símbolo de entrada a
        U = ε-cerradura(mover(T,a));
        if (U no está en Destados)
            agregar U;
        Trans[T,a] = U;             // se agrega el estado
    }
}
```

El primer estado que puedo alcanzar con una T es el estado de identidad (Yo mismo)

↳ IMPORTANTE

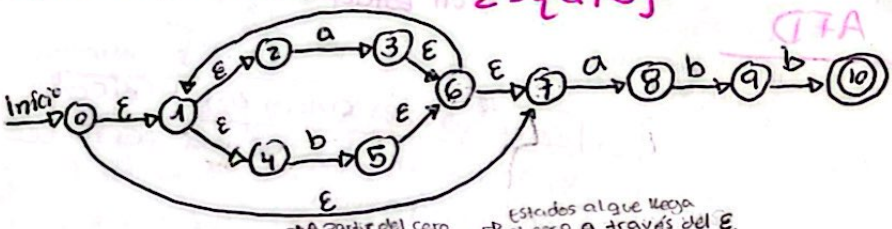
Inicializar Estados con c-ε(0) → genera → Destados =

↳ Todos los estados que alcanzo solamente con vacío, incluyendo el mismo estado

mover(T,a)
↳ Representa la tabla de Transiciones
Topología del AFN

mover(A,a) =
↳ A dónde me voy con 'a' a partir de cada uno de los estados de A.

Gráfico



Trans	a	b
→ A	B	C
B	B	D
C	B	C
D	B	E
(E)	B	C

Destados = { {0, 1, 2, 4, 7}, B, C, D, E }
A

A partir del cero / El mismo estado
Estados al que llega el cero a través del ε.

ε-C(s0) = C-ε(0) = {0, 1, 2, 4, 7} = A

Estados al que llega con ε, incluyéndole.
Estados que llego con ε
a partir de 3 y 8
Ellos mismos + ε
Ellos mismos

PARA A:
- con a: U = c-ε(mover(A,a)) = c-ε({3,8}) = {3, 8, 6, 7, 1, 2, 4} = {1, 2, 3, 4, 6, 7, 8} = B
- con b: U = c-ε(mover(A,b)) = c-ε({5}) = {5, 6, 7, 1, 2, 4} = {1, 2, 4, 5, 6, 7} = C

PARA B: {1, 2, 3, 4, 6, 7, 8}
- con a: U = c-ε(mover(B,a)) = c-ε({3,8}) = {1, 2, 3, 4, 6, 7, 8} = B
- con b: U = c-ε(mover(B,b)) = c-ε({5,9}) = {5, 9, 6, 7, 1, 4, 2} = {1, 2, 4, 5, 6, 7, 9} = D

PARA C: {1, 2, 4, 5, 6, 7}
- con a: U = c-ε(mover(C,a)) = c-ε({3,8}) = {1, 2, 3, 4, 6, 7, 8} = B
- con b: U = c-ε(mover(C,b)) = c-ε({5}) = {1, 2, 4, 5, 6, 7} = C

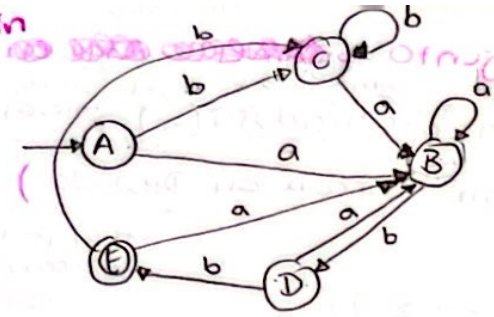
PARA D: {1, 2, 4, 5, 6, 7, 9}
- con a: U = c-ε(mover(D,a)) = c-ε({3,8}) = {1, 2, 3, 4, 6, 7, 8} = B
- con b: U = c-ε(mover(D,b)) = c-ε({5,10}) = {5, 10, 6, 7, 1, 2, 4} = {1, 2, 4, 5, 6, 7, 10} = E

PARA E: {1, 2, 4, 5, 6, 7, 10} =
- con a: U = c-ε(mover(E,a)) = c-ε({3,8}) = B
- con b: U = c-ε(mover(E,b)) = c-ε({5}) = {1, 2, 4, 5, 6, 7} = C

Scanned with CamScanner

Estados de Transición

Trans	a	b
→ A	B	C
B	B	D
C	B	C
D	B	E
• E	B	C



→ AFD resultante

Los estados finales del AFD son aquellos que contienen como elemento el estado final del AFN.

inicial

A = {0, 1, 2, 4 7}

E = {1, 2, 4, 5, 6, 7, 10}

final

En el AFD, A es inicial ya que contiene el '0', estado inicial del AFN.
E, es el estado final ya que contiene el '10' del estado final del AFN

Si tenemos una entrada, y ésta llega al estado final del AFD, podemos decir que la entrada es aceptada. (Aceptó el lexema)

Si la entrada termina en un estado final, es un LEXEMA

No cambia la topología, solo cambia la tabla

Algoritmo que está en el match. get next symbol

Simulación de un AFD

La entrada puede ser vacío.

Es el sgte símbolo del match de una Exp Reg

```
S = S0;
C = sig car();
While (c != eof) {
        S = mover(s, c);
        c = sig car();
}
if (s está en F) return "si";
else return "NO";
```

Es una rutina donde se escribe la tabla

Asumo que hay una tabla detrás

código del analizar léxico

detecta si el lexema pertenece o NO al token

La tabla de transición es LA ENTRADA

¿Para qué sirve Todo estos conceptos?

* P/ Escribir el código fuente del An Lex debemos copiar el código

get. next symbol

P/ Ejecutar el código fuente del An. Lex. → Acepta o rechaza lexemas

* Definir los patrones con exp Reg.
* Convertir a Thompson (AFN)
* convertir el AFN en AFD, mediante el algoritmo de subconjuntos
* Tenemos la tabla, usamos en mover, aplicamos mover cada vez que se le invoque, al final mirar si nos quedamos en un estado final o NO y ahí se determina si el lexema se acepta o NO

Sintáctico
- No terminal
- Función
- El BNF debe ser predictivo

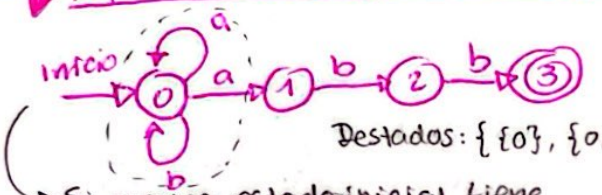
En el BNF NO debe haber terminales, debe haber tokens en vez de terminales que haya 'operador' para indicar '+, -, *,' La exp Reg. defina que pertenece a operador

Scanned with CamScanner

Preg

- En preanálisis nunca tengo un terminal, siempre tengo un TOKEN cuando se trata de un compilador.

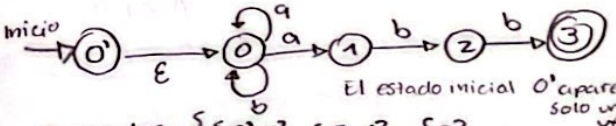
Aplicar el Alg de subconjunto al sgte AFN



Estado	a	b	ε
0	{0,1}	{0}	∅
1	∅	{2}	∅
2	∅	{3}	∅
3	∅	∅	∅

Destados: {{0}, {0,1}....

▷ Si nuestro estado inicial tiene una Transición, debemos transformar añadiendo un estado (0') con una Transición ε



El estado inicial 0' aparece solo una vez

Destados = {{0';0}, {0,1}, {0}....

* Vemos que el '0' aparece dos veces, como es un estado inicial, debe ser único, no puedo resolver cual es el estado inicial porque aparece en más de un estado.

Cuando NO hay cerradura ε, puedo dejar de usar porque no hay. Siempre devuelve la identidad de mover ellos mismos va a devolver

Thompson NO permite ningún caso en el que haga una Transición al estado inicial ← Garantiza que NO haya forma

Subconjunto: Obtiene un AFD a partir de un AFN creado por Thompson

Toda AFN tiene un AFD

#

clase 1

Minimización de un AFD

Método:

① Empezar con una partición inicial Π con dos grupos, F y S-F, los estados de aceptación (Finales) y de NO aceptación (No finales) de D. → Debe estar el estado inicial

② Aplicar el procedimiento (fig. 3.64) para construir una nueva partición Π_{nueva}.

al principio, dejar que $\Pi_{nueva} = \Pi$;
for (cada grupo G de Π) {
  particionar G en subgrupos, de forma que dos estados s y t se encuentren en el mismo subgrupo, si y sólo si para todos los símbolos de entrada a, los estados s y t tienen transiciones sobre 'a' hacia estados en el mismo grupo de Π;
  /* en el peor caso, un estado estará en un subgrupo por sí solo */
  sustituir G en Π_{nueva} por el conjunto de todos los subgrupos formados
}

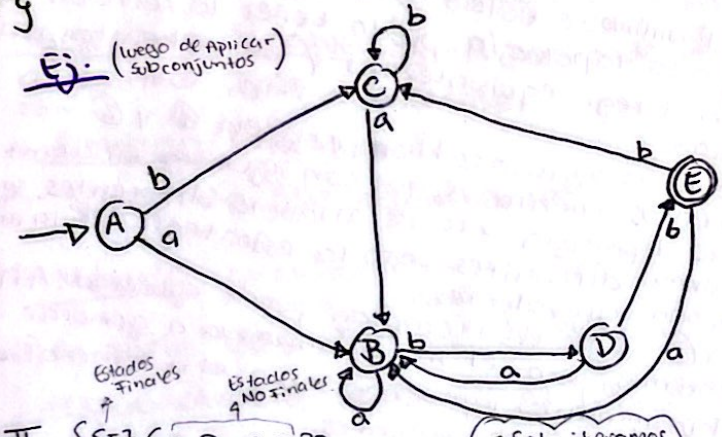
Validar que cada estado que está en cada grupo, siempre tenga una transición a estados de grupos similares

Pasos

① Crear partición Π, está formado por dos grupos: 1- todos los estados finales, 2- todos los demás que NO son finales.

Cada grupo es un estado. El AFD mínimo es el conjunto que resultó de la última iteración.
El Est. Inicial es donde se encuentra el EI original. Los EFinales son aquellos que contienen a los EF. originales

Ej. (luego de aplicar subconjuntos)



Estados finales | Estados NO finales

① $\Pi = \{\{E\}\{A,B,C,D\}\}$ (grupo 1) (grupo 2)
$\{\{E\}\{A,B,C,D\}\}$

• Solo iteramos los estados NO-unitarios

1ª iteración:
con a:

Estado	a	b
A	II	II
B	II	II
C	II	II
D	II	II
E	II	II

A con a se va a B, y B pertenece al Grupo II

→ Separar

No es necesario los grupos unitarios

2ª iteración

$\Pi_{nueva}: \{\{E\}\{A,B,C\}\{D\}\}$

Estado	a	b
A	II	II
B	II	III
C	II	II
D	II	I
E		

Solo ellos podrían separarse → Separar

Estados unitarios

Analizar E, NO tiene sentido porque pertenece a un grupo I que es unitario, no se va a dividir, NO es necesario analizar los grupos unitarios

El algoritmo deja de iterar cuando Π deja de cambiar

$\Pi_{nueva2}: \{\{E\}I, \{A,C\}II, \{D\}III, \{B\}IV\}$ (Estados unitarios)

Estados	a	b
A	IV	II
C	IV	II

No hay diferencia, no se separan.

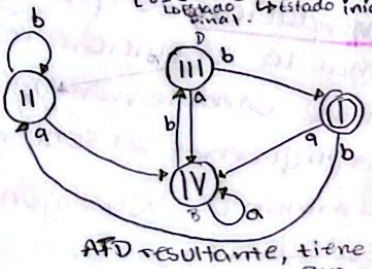
• Ya que NO hubo cambios en Π_{nueva2}, no hay separación, termina la ejecución del algoritmo

Es equivalente al original, reconoce los mismos lexemas, se produce la simplificación de un estado

Cada grupo es un estado

AFD mínimo

$\Pi: \{\{E\}I, \{A,C\}II, \{D\}III, \{B\}IV\}$
(Estado final) (Estado inicial)



Las transiciones deben ser de acuerdo al AFD original. La tabla $\Pi_{nueva\,2}$ corresponde a las transiciones de sus respectivos estados.

AFD resultante, tiene un estado menor que el original.

Scanned with CamScanner

Aplicando el Algoritmo de minimización

↳ se tiene la posibilidad de validar eventuales separaciones o NO de los grupos iniciales conforme a los grupos a los que van con cada símbolo del alfabeto, podemos establecer si hay o NO una reducción de estados.

Cuando Π ya no cambia, hay que tener cuidado, puede ocurrir la sgte situación:

Puede haber un estado muerto. (Salen T. pero no es alcanzable por ninguna Transición), es un estado impuro, se puede eliminar pero el algoritmo en sí, NO valida que se elimine. Hay que ver. (AFD mínimo), por lo menos todos los estados sean alcanzables. (Que aparezcan en la Tabla), Si NO aparece, es un estado impuro y se puede eliminar. El estado inicial es el único que está exento de esta situación. (NO eliminar)

↳ Si es el caso, debemos eliminar ese estado (NO tiene ninguna T. que le llegue)

#El algoritmo NO detecta ese caso.

- Acordarse (que nos pase)
  ① Estados inalcanzables se pueden eliminar

→ Paraqué se minimiza un DFA? (No detecta la ambigüedad de patrones) ↳ Detector de Ambigüedad de Patrones.

Para demostrar que 2 o más expresiones Regulares son equivalentes (solo basta con que el Alfabeto y la Topología final sean iguales) con la minimización se responden las sgtes preguntas:

- ¿Las EReg. describen el mismo lenguaje?
- ¿Las EReg son equivalentes?

Ej

① $(a|b)^*abb$

② $(01)100$

③ $(a|b|\varepsilon)(a|b)^*abb$

Si el AFD mínimo de éstas 3 EReg son iguales en cuanto a la Topología, puedo tener la certeza de que son EReg. equivalentes y describen los mismos lexemas.

cuando hagamos thompson en ① y ② nos daremos cuenta de que son iguales en cuanto a su topología, pero de alfabeto diferentes, serán lexemas diferentes pero la estrategia es la misma, NO son equivalentes

① y ③ al hacer thompson puede generar AFDs diferentes, pero el AFD mínimo va a generar AFDs mínimos iguales y sobre el mismo alfabeto entonces son equivalentes.

El AFD mínimo es un validador de EReg equivalentes

Se convierte en una estrategia para validar equivalencias de patrones de EReg.

Recomendación

EXReg, escribir los AFN con un inicio común

La ventaja de unir los tokens (Exp.Reg) en el AFN, es que el indeterminismo está permitido, si quiero unir, en un AFD no lo podré hacer.

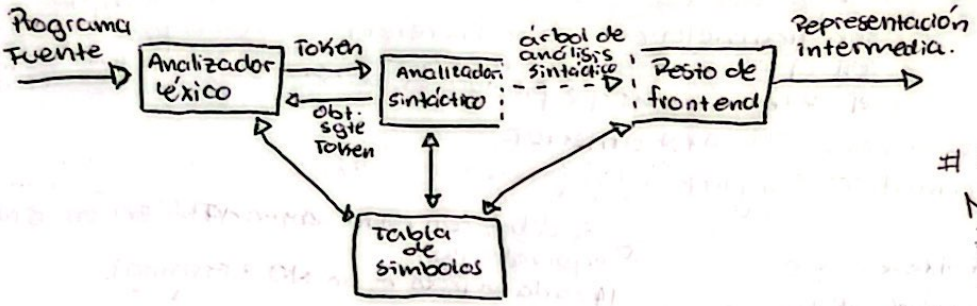
Generar un solo autómata donde esté todo el lenguaje

→ La recomendación, es que unamos la ERReg. para armar la definición Regular a nivel del AFN, tiene indeterminismo pero sobre el E. cuando apliquemos el subconjuntos, se eliminará el indeterminismo. Subconjunto eliminará la ambigüedad.

Se termina el análisis léxico.

Scanned with CamScanner

Analizador Sintáctico



#Ejemplos de An. sintáct en el cual aparezcan terminales, NO deberían ocurrir en un compilador Trad.

El compilador Trad. NO dejará al An. Sintáctico interactuar con los terminales tal cual cómo están

Derivaciones

- Derivación por izq (leftmost)
- Derivación por der. (rightmost)

↳ Una derivación puede derivar en cero o más pasos a una cadena de terminales y NO terminales.

Entonces:

1) $\alpha \Rightarrow^* \alpha$
2) Si $\alpha \Rightarrow^* \beta$, y $\beta \Rightarrow^* \partial$, entonces $\alpha \Rightarrow^* \partial$

Obs: Es la secuencia recorrida en un árbol sintáctico.

Relación BNF - Expresiones Regulares → conversión

Algoritmo de conversión: A partir de una ERreg. se puede encontrar una gramática BNF equivalente.

Entrada: Una Exp. Reg.

Salida: Un BNF equivalente.

BNF GLC: Notación más poderosa que las Exp. Reg.
↓
Gramáticas Libres de contexto

Método ↳ convertir un diagrama de estado en un BNF.

1) A partir de la Exp. Reg. de entrada, obtener el AFN equivalente. (diagrama de estado)
Opcionalmente se obtiene el AFD y/o el AFD mínimo equivalente.

(Creadas las sentencias de usuario)

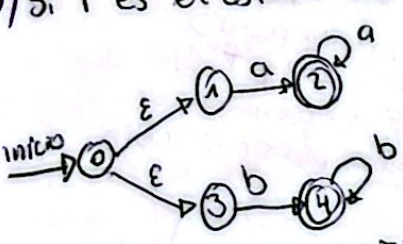
2) Para cada estado i del AFN, crear un NO-terminal A_i. (Estado 1 va a ser A_1) (cada estado se convierte en un terminal)

3) Si el estado i tienen una transición al estado j con la entrada a, agregar la producción $A_i \to aA_j$. (Para el diagr de estado (AFN o AFD)) Si el estado i pasa al estado j con la entrada ε, agregar la producción $A_i \to A_j$

(Genera la manera de como va a construir las producciones)

4) Si i es un estado de aceptación o final, agregar $A_i \to \varepsilon$

5) Si i es el estado inicial, hacer que A_i sea el símbolo inicial del BNF.



Un AFN que acepta $aa^*|bb^*$

(cada estado es un NO-terminal)

Gramática Original

$A_0 \to A_1 | A_3$
$A_1 \to aA_2$ (a: transición, A_2: estado siguiente)
$A_3 \to bA_4$
$A_2 \to aA_2 | \varepsilon$ (ε: pie agrega porq. es un estado final)
$A_4 \to bA_4 | \varepsilon$

Todos los estados finales pueden tener como lado der. 'ε' (Por definición)

Gramática simplificada

$A_0 \to aA_2 | bA_4$
$A_2 \to aA_2 | \varepsilon$
$A_4 \to bA_4 | \varepsilon$

$A_1 \to aA_2$ (a: Transición, A_2: estado sgte.)

Gramática simplificada

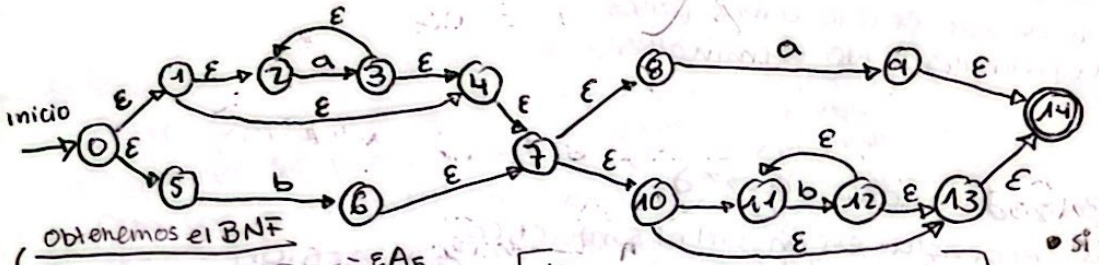
Si sólo tenemos uno o varios NO-terminales compuestos, entonces podemos evitar utilizar esos simples como ($A_1 \to aA_2$, $A_3 \to bA_4$)

- Son transiciones que se hicieron con el ε, por eso son simples, se elimina el determinismo que produce el vacío.
- Se eliminó el vacío, por eso son simples, siempre va a estar solo. $\varepsilon \to$ NO forma parte del alfabeto.

#Si obtengo el AFD mínimo del AFN anterior, se llega a la gramática simplificada. (mucho más fácil)

Si empiezo con un AFN, tengo que recurrir a una simplificación de la gramática. Si empiezo con un AFD mínimo, NO hace falta simplif. la gramática

Si debo convertir este AFN en un BNF equivalente:
1) cada estado es un NO-terminal.
2) Obtenemos el BNF
3) obtenemos la gramática simplificada.



Obtenemos el BNF

$A_0 \to A_1 \mid A_5$ (= εA_5)
$A_1 \to A_2 \mid A_4$
$A_2 \to aA_3$
$A_3 \to A_2 \mid A_4$
$A_4 \to A_7$
$A_5 \to bA_6$
$A_6 \to A_7$
$A_7 \to A_8 \mid A_{10}$
$A_8 \to aA_9$
$A_9 \to A_{14}$
$A_{10} \to A_{11} \mid A_{13}$
$A_{11} \to bA_2$
$A_{12} \to A_{11} \mid A_{13}$
$A_{13} \to A_{14}$
$A_{14} \to \varepsilon$ → No tiene saliente pero es EFinal.

#Recomendación: Convertir un AFN a AFD para eliminar todas las transiciones vacías, estás generan estos problemas. (-rec x izq -Factor común -Amb. semántica)

- Si tenemos una def Reg. sería más complejo el proceso de simplificación
- "ε" en el BNF significa que es un Estado Final y NO que es una transición

En un BNF (mínimo) generado a partir de un AFD, puede existir ambigüedad semántica?
- Es predictivo, NO puede existir

Para qué hacemos todo esto?

Porqué convertimos la Exp. Reg en un BNF?
↳ Para obtener el código fuente de la Exp. Reg.

```
S = S0;
c = sigCar();
while (c != eof) {
    S = mover(s, c);
    c = sigCar();
}
if (s está en F) return 'Si';
else return 'no';
```

→ código fuente de la Exp. Reg

La def regular estamos convirtiendo en BNF.

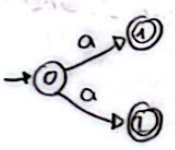
Hay un solo formalismo que va poder ejecutar todo el compilador

Que es nuestra estrategia de tener un BNF predictivo y escribir una función por cada NO terminal

Ej

$B_0 \to aB_1 \mid aB_2$
$B_1 \to \varepsilon$
$B_2 \to \varepsilon$

EFinales



Hay indeterminismo sobre 'a'
- cuando hay factor común es porque hay un indeterminismo.

Puede haber rec x izq y también ambigüedad semántica.

→ NOS conviene convertir a AFD para evitar, la rec x izq, F.C.
Si elegimos el AFD, obtendremos un BNF que vamos a poder escribir código fuente.

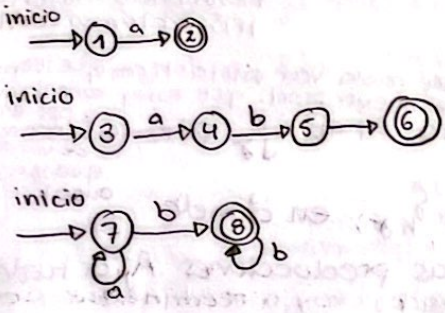
Scanned with CamScanner

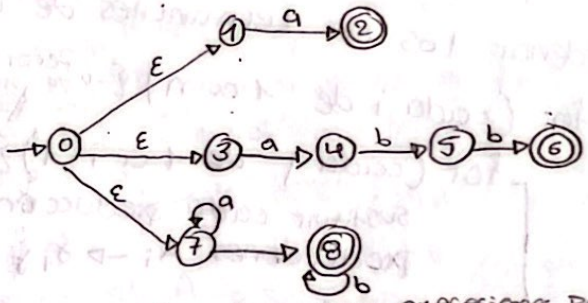
Para ejecutar el compilador: Primero tengo que ejecutar con aquel algoritmo de simulación de AFD; una vez q que el algoritmo capturó el lexema, tendrá que pasarle el control al código fuente del BNF, porque ¿Dónde se produce la captura del lexema?

→ Convertir la definición regular en un AFD o AFD mínimo, luego

En el algoritmo.

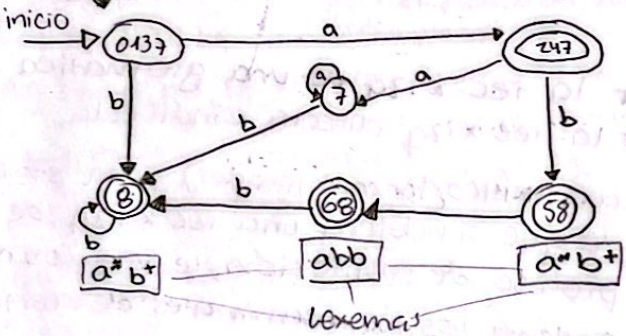
AFNs para las Exp. Reg.
a, abb y a*b+





AFN único para expresiones regulares
a, abb y a*b+

AFD para las ERegulares a, abb y a*b+



→ Este AFD voy a ejecutar con ese algoritmo, si el AFD representa una definición regular, es un capturador de lexemas, el Algoritmo va a retornar, 'Si' o 'No', va a retornar qué token es y va a grabar en la tabla de símbolos, dónde retorna el control?

Es recomendable obtener el AFD y convertir el AFD, entonces estoy haciendo a la vez la eliminación de la rec x izq, el factor común y de la amb. semántica.

Si convierto el AFN directo, voy a tener que aplicar esos 3 procesos, después.

NO recomendable

#Dónde se llama el código fuente de AFD que va a determinar los lexemas?
→ En el MATCH

#La parte léxica es lo que yo puedo convertir a BNF.

El BNF va a ser mínimo, si nosotros utilizamos un AFD mínimo, No es necesario que el AFD mínimo. La minimización es opcional.

Scanned with CamScanner

Existen casos de recursión x izq: (Todavía NO hemos evaluado)
↳ casos de rec. x izq de forma indirecta.

Se plantea este algoritmo de eliminación de rec x izq.
↳ Contiene al algoritmo de eliminación por la izq.

Eliminación de rec x izq. (Recursión indirecta) ↳ y directa.
↳ Haciendo una sustitución puedo llegar a profundidad a encontrar un caso de rec x izq.

Entrada: La gramática G sin ciclos ni producciones ε

Salida: Una ~~cadena~~ gramática equivalente sin recursividad x izq.

Método: Aplicar el sgte algoritmo. Se observa que la gramática NO recursiva por la izq. resultante puede tener producciones ε.

1. Ordenar los NO terminales de cierta forma $A_1, A_2, \dots, A_n$ ↳ De acuerdo a una conveniencia como resultado de una inspección visual
2. for (cada i de 1 a n) { ↳ Recorrer todos los sistemas de producción
3. for (cada j de 1 a i-1) { ↳ Reemplaza cada var sintáctica que tengas ahora en la nueva prod. que estoy analizando
4. sustituir cada producción de la forma $A_i \to A_j \gamma$ por las producciones $A_i \to \delta_1\gamma \mid \delta_2\gamma \mid \dots \mid \delta_k\gamma$, en donde $A_j \to \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ sean todas las producciones A_j actuales (Todos los NO terminales ya procesados voy a reemplazar por su lado derecha) por el lado derecho de var sintáctica que ya analice.
5. }
6. eliminar la recursividad inmediata por la izq. entre las producciones A_i
7. }

Algoritmo para eliminar la rec x izq de una gramática ↳ legal que permite eliminar la rec x izq directa e indirecta.

El algoritmo aplica un proceso cuadrático (for concatenados) para sustituir los lados derechos de tal manera de que si hubiera una rec x izq de forma indirecta en algún momento del proceso de sustitución, le vamos a encontrar.

Aplica dos ciclos, pero antes ordena los NO terminales de cierta forma. (la manera que quiera) (romper el esquema jerárquico)

#Recomendación p/ acortar el proceso

- Hacer una inspección visual de toda la gramática y trato de buscar visualmente si NO detecto una rec x izq indirecta, si detecto entonces empezar como producción inicial, empezar con aquella producción que va desencadenar la rec. x izq. indirecta. Ese orden solo vale para el algoritmo, finalizado el algoritmo, cada producción vuelve a ocupar su posición en el estado jerárquico.

El orden por default, puede salir un algoritmo mucho más largo, el problema simplemente es de longitud, le va a encontrar igual pero con más esfuerzo.

Scanned with CamScanner

Ejemplo:

original →
$S \to Aa \mid b$
$A \to Ac \mid Sd \mid \varepsilon$

F.C.? (Aplicamos la elim. directa)

$A \to Ac \mid Aad \mid bd \mid \varepsilon$

$A \to bdR \mid R$
$R \to cR \mid adR \mid \varepsilon$

No cambió → $S \to Aa \mid b$

La primera vez, NO entra en el 2do for, y trata de eliminar la recdirect en S. (No hay prod. Anteriores)

La segunda vez, estoy en A y entro al segundo for. Hay producciones anteriores a A? Sí, S.
En caso que S esté en el lado der A, expandir S con su lado derecho

$A \to Ac \mid Aad \mid bd \mid \varepsilon$

S → reemplaza por su lado derecho.

→ Al ordenar las producciones, poner primero una producción que a simple vista no tenga Rec x izq directa.

1) $S \to Aa \mid b$
2) $A \to Ac \mid Sd \mid \varepsilon$

Este orden NO importa. No necesariamente se debe seguir ese orden que plantea el ejercicio.

i=1
j=0

$S \to Aa \mid b$ (NO tiene Rec x izq) Procesado

i=2
j=1

$A \to Ac \mid Sd \mid \varepsilon$

$A \to Ac \mid (Aa \mid b)d \mid \varepsilon$

$A \to Ac \mid Aad \mid bd \mid \varepsilon$ → Se debe eliminar la Rec x izq directa →

$A \to bdR \mid R$
$R \to (c \mid ad)R \mid \varepsilon$ (sin rec)
$R \to cR \mid adR \mid \varepsilon$

* La pregunta que debo hacer es: ¿cual de los NO-terminales ya procesé?
Todas las producciones que están antes que no tienen rec x izq en general.
→ Si hay producción arriba, reemplazar.

Procesado

$S \to Aa \mid b$
$A \to bdR \mid R$
$R \to cR \mid adR \mid \varepsilon$

La gramática Resultante.

De esta manera nos damos cuenta de la importancia de ordenar las producciones.

\# Si elegía primero la A y luego las S

$A \to Ac \mid Sd \mid \varepsilon$, la primera vez NO hay Prod Anterior

$A \to SdR \mid R$
$R \to cR \mid \varepsilon$ } Rec x izq

Reemplazando la A en S

$S \to SdRa \mid Ra \mid b$
$S \to RaR' \mid bR'$
$R' \to dRaR' \mid \varepsilon$
$A \to SdR \mid R$
$R \to cR \mid \varepsilon$

4 prod, mientras el otro es 3 prod siempre va a elim. la rec x izq solo que será más largo.

\# ES importante la inspección visual.

$A \to A\alpha \mid \beta$
$A \to \beta \mid R$
$R \to \alpha R \mid \varepsilon$
NO olvidar.

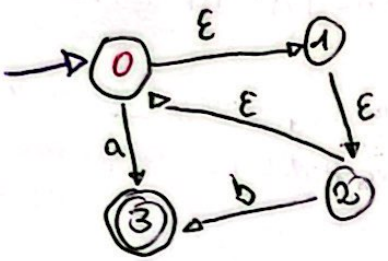
clase 14

Escribir el código fuente del compilador descrito por el sgte esquema:

BNF:

S → regexp S | regexp.

regexp:



Obs: 1) Utilizar el AFN, no convertir a AFD

2) No realizar simplificaciones. Solo aplicar algoritmos vistos en clases.

En vez de darnos la EReg. nos dió el diagrama de estado → AFN o AFD, usa solo para confundir

Si tengo la def Reg, tengo el código fuente

¿Cómo se escribe el código fuente de un compilador a partir de su Def Reg?

Como primera opción... convertir a un AFD, agarrar la tabla de Transiciones y mostrar el algoritmo de SIMULACIÓN de un AFD

Simulación de un AFD:

```
S = So
c = sigCar();
while (c != eof){
        s = mover(s,c);
        c = sigCar();
}
if (s está en F) return "si";
else return "no";
```

→ Obtengo el AFD o AFD mínimo (solo si hay tiempo), agarro la tabla, pongo al lado de este algoritmo y éste algoritmo es llamado en el match.

cód. fuente → Versión 1

Versión 2 (Si no se puede convertir a AFD)

↳ una salida sería usar el código muy largo de los 'case' pero demanda mucho tiempo

Tengo que salir del mundo léxico y convertir todo en sintáctico

Otra opción (Versión 3)

(convertir) el AFN a BNF

Pasar lo que está en la parte de la def regular (expresado en un diagrama de estado) pasar eso a BNF y controlo la rec, FC y amb semántica.

Si aplicamos otro algoritmo que NO sea el de conversión ER a BNF NO tenemos certeza de que ese BNF acepte el mismo lenguaje que el AFN

BNF (convertimos el diag. de estado a BNF)

$A_0 \to A_1 \mid aA_3$

$A_1 \to A_2$

$A_2 \to A_0 \mid bA_3$

$A_3 \to \varepsilon$

Podemos escribir el código fuente utilizando la estrategia del traductor, un NO terminal, una función pero debe ser predictivo.

CONTINÚA.

Scanned with CamScanner

BNF (Anterior)

BNF que debo escribir en el código fuente

- $A_0 \rightarrow A_1 \mid a A_3$
- $A_1 \rightarrow A_2$
- $A_2 \rightarrow A_0 \mid b A_3$
- $A_3 \rightarrow \varepsilon$ (estado de aceptación)

$S \rightarrow regexp\ S \mid regexp$

$S \rightarrow A_0\ S \mid A_0$ → se reemplaza porque eso es el nombre del token, está representando a ese autómata que se convirtió en BNF, debo reemplazar por el estado inicial

→ Por eso se reemplaza regexp por A_0.

Cómo se transfiere el indeterminismo de un AFN a un BNF?

La conversión es entre el modelo léxico al sintáctico, la base de la conversión, puede ser un AFN, AFD o AFD mínimo, cualquiera de estos 3 sirve para obtener un BNF, es mejor que sea un AFD porque nos eliminaba mos problemas.

En el AFN dado (Diagrama) tiene incidencia en el estado inicial, hay un ciclo de ε .....

↳ Thompson nunca creará problemas con el estado inicial en el algoritmo de subconjuntos, tampoco con la Recursiva izq. (No te mete) cada parte de Thompson garantiza que NO tendré problemas.

#

Scanned with CamScanner

V Clase 15

Dada la sgte gramática, se pide obtener una gramática equivalente que sea predictiva.

$S \to A = B \mid A < B \mid A > B$

$A \to not\ A \mid a$

$B \to b\,B \mid \varepsilon$

Rec x izq: NO

F. C: Sí, sobre A

Amb. Semántica:

Aceptemos que debemos hallar el conjunto SGTE

Antes de hacer el C. Primero y SGTE debemos solucionar el FC y Rec x izq.

$S \to A\ R$

$R \to = B \mid < \ldots$

$A \to not\ A \mid a\ A$

$B \to b\,B \mid \varepsilon$

$P(S) = P(A) = \{not, a, \varepsilon\}$

$P(R) = \{=\} \cup \{<\} \cup \{>\} = \{=, <, >\}$

$P(A) = \{not\} \cup \{a\} \cup \{\varepsilon\} = \{not, a, \varepsilon\}$

$P(B) = \{b\} \cup \{\varepsilon\} = \{b, \varepsilon\}$

Conjunto SGTE

① $S(S) = \{\$\}$ → eof → Termina ahí porque S no aparece en ningún lado derecho de las prod.

$X \to \alpha S \beta$

$S(R) = \{\$\}$ → No aparece R en ningún otro lado derecho.

$S(A) = \{=, <, >\}$

$S(B) = \{\$\}$

PARA $S(A)$: A_1 aparece en S, tenemos en el lado der $A\ R_{1\,up}$ → Si no hay α pero sí β, entonces todo lo que hay en PRIMERO de β agrego en SGTE(B), exceptuando el vacío (Aplica la regla 2), Agrego en el SGTE(A) todo lo que hay en PRIMERO(R). Luego... Veo otro lado derecho que contenga A, aparece en $A \to not\ A \mid a\ A \mid \varepsilon$, en ambos casos NO hay β, veo el caso 3, si no hay β, entonces pongo en SGTE(A) Todo lo que hay en SGTE(A) en este caso. En el caso de SGTE NO hay problema que se repita, pero se puede poner una sola vez los que se repiten y agregar los nuevos. Si ya no hay A en otro lado derecho, cierro el conj SGTE(A).

CONJUNTO SGTE

Para calcular el conjunto sgte tengo que ir a buscar los lados derechos.

seguimos 3 pasos:

① colocamos un símbolo '$' en SGTE(S) donde S es el símbolo inicial de la gramática y el símbolo que voy a colocar, va a representar el fin de la cadena de entrada
Ese símbolo NO está en el alfabeto (Si el $ está en el alfabeto, usamos otro símbolo)

agregamos un dólar, más todo lo que encuentre después

② Tengo que buscar un lado derecho en el cual tenga cualquier cosa (x)

$X \to \alpha S \beta$, ①

Buscar que S esté en un lado der. para poder calcular nuevos valores al conjunto sgte. Si NO aparece, se acabó la búsqueda.

El paso ① agregar el $ se hace una sola vez sobre el símbolo inicial

② Buscar un lado derecho que sea $X \to \alpha R \beta$, ó RB ó αR?, está R en el lado derecho?
hay en $S \to A\ R$, (No hay otra en ningún lado derecho.)

Pero NO tiene β, no puedo aplicar el paso 2, entonces pasamos a 3

③ Para cualquier prod que tenga en donde $A \to \alpha B$ ó $A \to \alpha B \beta$, $P(\beta)$ tiene ε entonces todo lo que hay en SGTE(A) está en SGTE(B)

Ponemos en SGTE(B) todo lo que hay en SGTE(A) → Llamante

Para que S se acabe, se tiene que acabar la R, es lógico pensar que si no hay nada después de la R, entonces ponemos en SGTE de R lo que hay en sgte de S, del llamante.

Scanned with CamScanner

la sgte gramática, se pide obtener una gramática e que sea predictiva.

Rec x izq: NO

F.C: Sí, sobre A

=B | A < B | A > B

not A | a A | ε

b B | ε

→ A R

→ = B | < B | > B

→ not A | a A | ε

B → b B | ε

= P(A) = {not, a, ε}

= {=} ∪ {<} ∪ {>} = {=, <, >}

) = {not} ∪ {a} ∪ {ε} = {not, a, ε}

.) = {b} ∪ {ε} = {b, ε}

onjunto SGTE.

) = {$} → eof → Termina ahí porque S no aparece en ningún lado derecho de las prod.

) = {$} → No aparece R en ningún otro lado derecho.

) = {=, <, >}

) = {$}

Buscamos el conj SGTE porque hay un caso de amb semántica en el que podría equivocarse el conjunto primero o NO podría detectar el conjunto primero.

Ese caso ocurre cuando hay ε en P. Hacemos una evaluación del conjunto Sgte solamente porque detecto que el vacío pertenece a algún conjunto primero

Si NO hay ε en P, ni siquiera es necesario hacer el SGTE. Ayuda a validar la existencia o NO de un problema semántico

SGTE

el conjunto sgte

car los lados

pasos:

un símbolo '$' en

S es el símbolo inicial

a y el símbolo que

va a representar el

fin de la cadena de entrada

Ese símbolo NO está en el alfabeto (si el $ está en el alfabeto, usamos otro símbolo)

agregamos un dólar, más todo lo que encuentre después

② Tengo que buscar un lado derecho en el cual tenga cualquier cosa (x)

X → α S β ①

Buscar que S esté en un lado der. para poder calcular nuevos valores al conjunto sgte. Si NO aparece, se acabó la búsqueda.

El paso ① agregar el $ se hace una sola vez sobre el símbolo inicial

② Buscar un lado derecho que sea X → α R β, ó R β ó α R ?, está R en el lado derecho?

hay en S → A R (No hay otra en ningún lado derecho.)

pero NO tiene β, no puedo aplicar el paso 2, entonces pasamos a 3

③ Para cualquier prod que tenga en donde A → α B ó A → α B β, P(β) tiene ε

parece en S, tenemos en el lado der

hay α pero sí β, entonces todo lo

PRIMERO de β agrego en SGTE(B),

el vacío (Aplica la regla 2), Agrego en el

lo que hay en PRIMERO(R). Luego...

que contenga A, aparece

dos casos NO hay β,

SIGUIENTE

Aplicar a las sgtes reglas hasta que NO sea posible agregar ningún simbolo terminal nuevo a cualquier conjunto sgte:

(1) Colocar $ en SIGUIENTE (S), en donde S es el simbolo inicial y $ es el delimitador derecho de entrada (fin de cadena).

(2) Para cada producción de la forma $A \to \alpha B \beta$, agregar a SGTE (B) todos los simbolos terminales a excepción de ε que se encuentren en PRIMERO (β)

(3) Para toda producción de la forma:

$A \to \alpha B$, o,

$A \to \alpha B \beta$, en donde PRIMERO (β) contiene a ε, entonces todo lo que hay en SIGTE (A) está en SGTE(B)

PARA S(B) del anterior: B aparece en $R \to = = B \mid < B \mid > B$, vemos que en el caso 3, $A \to \alpha B \Rightarrow$ todo lo que hay en SGTE(R) estará en el sgte (B), que se repita NO es problema para SGTE, pero lo agrego una sola vez en SGTE(B) todo lo que hay en SGTE(R) que NO sea el ε. B aparece en el $B \to bB \mid \varepsilon$, como tampoco hay β, será el SGTE(B), él mismo NO hay rec si agregamos SGTE(B) al mismo, hay rec x derecha pero NO afecta

CONJUNTO SGTE RESULTANTE

$S(S) = \{\$\}$

$S(R) = \{\$\}$

$S(A) = \{=, <, >\}$

$S(B) = \{\$\}$

En caso de que aparezca ε en el conjunto SGTE. → Si el conjunto primero tuviera el 'ε' yo NO le agrego al SGTE, agrego todo lo demás (caso 2), en en caso 3, si hay ε en el PRIMERO, agregamos el SGTE del llamante, NO hay forma de que el ε entre al conjunto SGTE, NO puede estar en el SGTE nunca.

La única forma de que ε estuviera en SGTE, es que desaparezca la producción. si pasa eso viene la sgte producción .... sucesivamente.

Scanned with CamScanner

→ Algoritmo que nos establece un mecanismo para tener el conjunto SGTE → conceptualmente significa... lo primero que viene después de ese NO-terminal

↳ La amb. semántica que detecta solamente se produce si hay ε en el PRIMERO y además los conjuntos SGTE y PRIMERO de ese NO-terminal, cuando hay ε en el PRIMERO, NO son disjuntos.

Cuando hay ε en PRIMERO, decimos que hay amb. semántica si el PRIMERO y SGTE tienen caracteres en común, no se validan que sean conjuntos disjuntos.

→ UNA vez que se halla el SGTE, tengo que verificar entre sus valores pero hago solo el chequeo cuando está el vacío

$P(S) = \{not, a, \varepsilon\}$

$P(R) = \{=, <, >\}$ No es necesario hacer chequeo porque No hay ε

$P(A) = \{not, a, \varepsilon\}$

$P(B) = \{b, \varepsilon\}$

$S(S) = \{\$\}$

$S(R) = \{\$\}$ → Es adorno si NO aparece ε en $P(R)$

$S(A) = \{=, <, >\}$

$S(B) = \{\$\}$

$S(S)$ es disjunto al $P(S)$?, digo eso porque hay 'ε' en el $P(S)$, efectivamente se cumple que son disjuntos, No hay ningún símbolo común.

$S(A)$ es disjunto al $P(A)$?, son disjuntos.

$S(B)$ es disjunto de $P(B)$?, Sí, son disjuntos.

→ Entonces la gramática es predictiva.

$S(P)$ NO es disjunto $P(P)$ ocurre la ambigüedad.
Si ε aparece en $P()$ tengo que chequear siempre el conjunto SGTE y tiene que ser disjuntos, en este caso X está en $S(P)$ y $P(P)$, sólo el ε puede ser común entre ellos.

La verificación se hace porque.

$X \to a\,R\,b$ $\quad P(X) = \{a\}$

$R \to b \mid \varepsilon$ $\quad P(R) = \{b\} \cup \{\varepsilon\}$ → si aparece ε, puede desaparecer la P.

Entrada: a b → No sabemos si hace match en X o R

↑ hace match en X

Hallar el PRIMERO y SGTE de la gramática.
Logrando las producciones son compuestas sean disjuntos

$S \to Px$

$P \to Aa \mid (\varepsilon)$ → indica que P puede desaparecer

$A \to X \mid Y$

caso 2 BB

$S \to Px$

$S(P) \leftarrow Term\,P(x)$

$P \to A\,a$
$\quad\;\; B\,B$

$S(A) \leftarrow Term\,P(a)$

[$P(P)$ y $P(S)$ NO son disjuntos, NO es predictivo.

CP

$P(S) = P(P) = \{x\} \cup \{y\} \cup \{\varepsilon\}$

$P(P) = P(A) \cup \{\varepsilon\} = \{x\} \cup \{y\} \cup \{\varepsilon\}$

$P(A) = \{x\} \cup \{y\}$

CSGTE

$S(S) = \{\$\}$

$S(P) \{x\}$

$S(A) = \{a\}$

Entrada: x a

$S \Rightarrow Px \Rightarrow Aax \Rightarrow xax$ → consumimos la entrada

Scanned with CamScanner

CALCULAR SGTE(X)

Conj PRIMERO de un terminal es un terminal.

$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid id$

$P(E) = P(T) = \{(\} \cup \{id\}$

$P(E') = \{+\} \cup \{\varepsilon\}$

$P(T) = P(F) = \{(\} \cup \{id\}$

$P(T') = \{*\} \cup \{\varepsilon\}$

$P(F) = \{(\} \cup \{id\}$

↳ $P(F) =$

$P(\beta) \to P(\,)) = \{)\}$

$S(E) = \{\$,)\}$

$S(E') = \{\$,)\}$

$S(T) = \{+, \$,)\}$

$S(T') = \{+, \$,)\}$

$S(F) = \{*, +, \$,)\}$

$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid id$

$P(E) = P(TE') = \{(\} \cup \{id\}$

$P(E') = \{+\} \cup \{\varepsilon\}$

$P(T) = P(FT') = \{(\} \cup \{id\}$

$P(T') = \{*\} \cup \{\varepsilon\}$

$P(F) = \{(\} \cup \{id\}$

$S(E) = \{\$,)\}$

$S(E') = \{\$,)\}$

$S(T) = \{\$,), +\}$

$S(T') = \{\$,), +\}$

$S(F) = \{*, \$,), +\}$ //

$E \to TE'$

$A \to \alpha B$

$S(E') \leftarrow S(E)$

$E' \to +TE'$ (α)

$A \to \alpha B$

$S(T) \leftarrow S(E)$

$S(E) \leftarrow S(E')$

$E \to TE'$

$A \to BB$ (Aplico 2)

$S(T) \leftarrow P(E') - \varepsilon$

P/ S(T')

$T \to FT'$

$A \to \alpha B$

$S(T') \leftarrow S(T)$

$T' \to *FT'$ (α)

$A \to \alpha B$

$S(F) \leftarrow S(T')$

P/ S(F)

$T \to FT'$ (P regla 2)

$A \to B\beta$

$S(F) \leftarrow P(T') - \varepsilon$

$T' \to *FT'$

$A \to \alpha B\beta$

$S(F) \leftarrow P(T') - \varepsilon$

$S(F) = \{*, +, \$,)\}$

↳ Se añade el $P(T')$ que NO sea vacío y como hay vacío en el $P(T')$ entonces agregamos el SGTE(T) y SGTE(T') por que son sus llamantes.

↳ Se aplica el caso 2 y caso 3.2

P/E

$F \to (E)$ (B)

$S(E) \leftarrow$

P/E'

$E \to TE'$ (αB)

$S(E') \leftarrow S(E)$

$E' \to +TE'$ (α, B)

$S(E') \leftarrow S(E')$

P/T

$E \to TE'$ (αB)

$S(T) \leftarrow S(E)$

$T \to FT'$

$S(T') \leftarrow S(T)$

P/ F

$T \to FT'$ ($B\beta$)

$S(F) \leftarrow$

$E' \to +TE'$ (α, B)

$S(E') \leftarrow S(E')$

$T' \to *FT'$ ($\alpha B\beta$)

$S(F) \leftarrow P(T') - \varepsilon$

P/T

$E \to TE'$ ($B\beta$) $P(E') =$

$E' \to +TE'$ (αB)

$S(E') \leftarrow S(E)$

$T \to FE'$ (αB)

$S(T') \leftarrow S(T)$

$T \to FT'$ ($B\beta$) $P(T') = \{*\}$ $+S(T) = \$,), +$

$T' \to *FT'$ ($\alpha B\beta$) $P(T') = \{*\}$ $+S(T') =$

Análisis Sintáctico Predictivo

Gramáticas LL(1)

Toda gramática predictiva es LL(1)
↳ Si se cumple, toda la gramática es predictiva

LL(1)
- Left
- Left
- (1) → un carácter
  - → utilizar un símbolo de la cadena de entrada para realizar la selección de la acción sintáctica que corresponda (elegir la sgte producción)
  - → A partir del símbolo inicial, se produce una derivación por la izq. para la verif. de pertenencia de la cadena a la gramática
- → Se refiere al análisis de la cadena de entrada de izq. a derecha hasta encontrar el símbolo $

#Necesito un lexema solo de la entrada para determinar qué lado derecho voy a utilizar, derivar, expandir (cero backtraking)

Si la gramática es LL2, permitiría un grado de ambigüedad
↳ No hay manera para escribir programas de una LL2. porque Backtraking es difícil programar

> #Si pide si la gramática es LL1 o si la gramática es predictiva está diciendo la misma cosa

Una gramática G con su Tabla de Análisis Sintáctico sin entradas múltiples se define como (LL(1)) y cumple con las sgtes condiciones:
↳ Si NO es rec x izq
↳ " FC
↳ Amb. semántica.

a) NO es recursiva por la izquierda.

b) No hay ambigüedad:
- ↳ sintáctica: es decir, está factorizada x izq. en caso de ser necesario.
- ↳ semántica: No hay amb. en PRIMERO ni SGTE

O bien, en otras palabras: Para toda $A \to \alpha \mid \beta$, se debe cumplir:

- Solo α ó β a la vez, puede derivar en ε:
- PRIMERO(α) es disjunto al PRIMERO(β) ↳ NO hay FC, ni amb semántica ↳ se verif la ambigüedad.
- Si β deriva en ε ($\beta \Rightarrow^* \varepsilon$), entonces ∄ NO deriva en ninguna cadena que comience con un terminal en SGTE(A)
  ↳ Si está 'ε' en P(β), entonces debe ser disjunto el PRIMERO(β) y SGTE(β)

Amb semántica
Puede darse la ambigüedad porque no habla de eso

Si una gramática NO cumple con alguna de estas condiciones hay que tratar de convertirla ya sea aplicando algoritmos de eliminación de rec x izq, factorizando o eliminando la ambigüedad.

NO todas las gramáticas pueden ser convertidas en gramáticas LL(1)

Scanned with CamScanner

Ejemplo

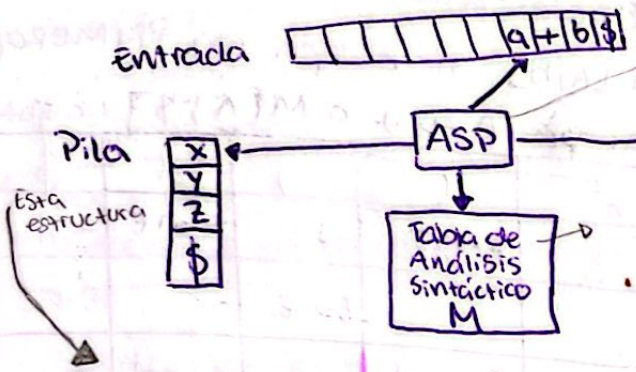
si la prod es simple, NO tiene que ser rec. x der ó?

Resumen de las partes que tiene un Analizador sintáctico Predictivo

↳ Puedo utilizar esta estructura para implementar un analizador Sintáctico Predictivo (ASP) en vez de escribir el código Fuente como venimos haciendo.

En vez de utilizar una función para cada NO-terminal, puedo escribir el código fuente de un ASP, describiendo esta estructura,

El cód fuente en esta estructura NO tiene los NO-terminales, la gramática NO está en duro.



▸ Programa de ASP, preescrito. acepta un conjunto de entradas específicos por cada problema, utiliza una tabla M donde está la gramática (equivalente a la tabla de Transiciones que usábamos en el An. Léxico. Arma una var interna (Pila) estructura interna, No nos afecta, y se genera una salida.

Reemplaza una función por cada NO-terminal

En el An léxico podíamos implementar el diagrama de estado con ifs o con el algoritmo de simulación. La topología del diagrama de estado NO está, la topología entra como una tabla de Transiciones en la función mover.

Una gramática predictiva puede simularse, ejecutarse, implementarse con un algoritmo que maneja esta estructura de datos

Tabla M es equivalente a la tabla de Transiciones que usábamos en el Analizador léxico.

Scanned with CamScanner

Algoritmo

construcción de una tabla de ASP

Entrada: La gramática G

Salida: La tabla de análisis M

Método: Para cada producción $A \to \alpha$ de la gramática, hacer lo sgte:

1) Para cada terminal a en PRIMERO(A), agregar $A \to \alpha$ a M [A, a] (No-terminal: A; Matriz: M; Terminal: a)

2) Si ε está en PRIMERO(α), entonces para cada Terminal b en SGTE(A), se agrega $A \to \alpha$ a M [A, b]. Si ε está en PRIMERO(α) y \$ se encuentra en SGTE(A), se agrega $A \to \alpha$ a M [A, \$] también

Solamente si está vacío, voy agregar para cada terminal que está en SGTE, la misma producción

El caso de \$, se trata de un caso especial solo para decir que NO es un terminal

- $E \to TE'$
- $E' \to +TE' \mid \varepsilon$
- $T \to FT'$
- $T' \to *FT' \mid \varepsilon$
- $F \to (E) \mid id$

M	+	*	(	)	id	$
E			E→TE'		E→TE'	
E'	E'→+TE'			E'→ε		E'→ε
T			T→FT'		T→FT'	
T'	T'→ε	T'→*FT'		T'→ε		T'→ε
F			F→(E)		F→id	

Tendrá como columnas los símbolos del alfabeto + \$

Tendrá como filas los NO-terminales

↳ cargamos las prod. simples, por eso PRIMERO está en U para ver por cuál lado procesamos

La tabla de ASP de esta gramática, tiene terminales (columnas) y NO-terminales (filas), el vacío NO puede ir en la columna porque NO forma parte del alfabeto

Para completar la tabla, debemos tener calculado el PRIMERO y SGTE de los NO-terminales

$P(E) = \{(, id\}$ → NO uso el SGTE porque no hay ε

$P(E') = \{+\} \cup \{\varepsilon\}$ → uso SGTE porque hay ε

$P(T) = \{(, id\}$

$P(T') = \{*\} \cup \{\varepsilon\}$ → uso SGTE

$P(F) = \{(\} \cup \{id\}$

comenzar ↑

Todo lo que está en E', menos el vacío

$S(E) = \{\$,)\}$ ← aparece en F

$S(E') = \{\$,)\}$

$S(T) = \{\$,), +\}$

$S(T') = \{\$,), +\}$

$S(F) = \{\$,), +, *\}$

comenzar ↓

- PARA hallar S(E), vemos que E aparece en el lado der. de $F \to (E) \mid id$, estamos en el caso 2 $A \to \alpha B \beta$, entonces agregamos a S(E) todos los terminales menos ε que se encuentran en P(β), β es ')' en este caso
- S(E'), E' aparece en $E \to TE'$ y $E' \to +TE' \mid \varepsilon$
- S(T), T aparece en $E \to TE'$ ($A \to \alpha B$) y $E' \to +TE'$ ($A \to \alpha B \beta$)
- S(T'), T' aparece en $T \to FT'$ ($A \to \alpha B$) y $T' \to *FT'$ ($A \to \alpha B$)
- S(F), F aparece en $T \to FT'$ ($A \to B\alpha$) y $T' \to *FT'$ ($A \to \alpha B \beta$)

Pasos para hallar M.

① Usamos cada terminal que está en el conjunto primero de ese NO-terminal, para cada terminal que está en ese conjunto primero, voy a agregar la producción en la tabla

Si en el conjunto primero aparece vacío 'ε', entonces hago lo mismo para los terminales que están en el SGTE, uso el SGTE, solamente si aparece 'ε' en el conjunto PRIMERO, si ε, no hay en el PRIMERO no voy a usar SGTE para ese NO terminal

Scanned with CamScanner

En la tabla M:

- Para cada terminal, se carga las producciones simples, por eso es importante que el PRIMERO esté en U para identificar cada símbolo de cada lado de la prod. compuesta.

La tabla sirve para: es como una tabla de transiciones pero en lo sintáctico.... → como, operador de ambigüedad, pero NO de predicción, si hay rec x izq. la tabla no dirá.

→ El BNF que nos da antes, no necesariamente es predictivo

Puede ser que la gramática NO sea predictiva pero si NO es predictiva es probable que tenga entradas múltiples, si hay entradas múltiples,

id
F → id
F → (E)

→ Entrada múltiple Ejemplo.

P(F) = {(, id} ∪ {id}

Amb. semántica

La tabla te dice que la gramática NO es predictiva

La tabla M es un validador, un operador
→ Permite entender si la gramática es o NO ambigüa.

Dada la sgte gramática, X, es predictiva? → La tabla NO dice si es predictiva, solo si es ambigüa.

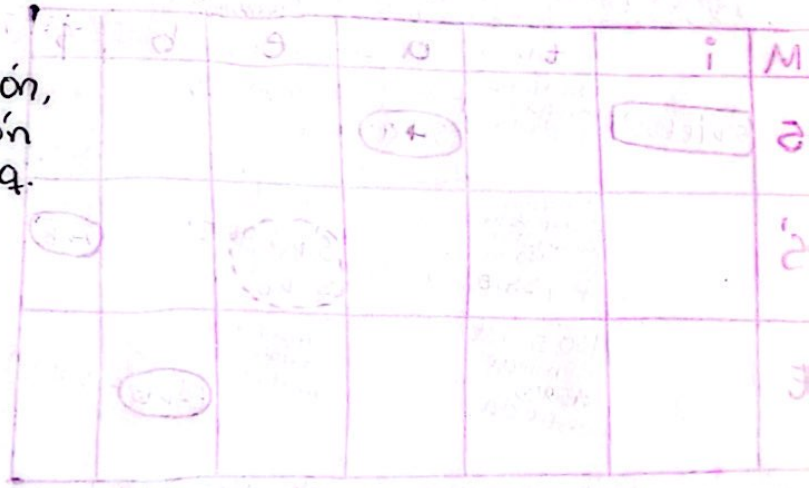
La tabla solo satisface la ambigüedad sintáctica y semántica, a eso le debo agregar un análisis externo de la Rec x izq, si todo sale bien, ambas cosas va a determinar que la gramática es predictiva.

→ Si queremos validar si es ambigüo o NO, hallamos la tabla y ya esta no hace falta la Rec x izq.

PREDICCION = control (Amb. + Rec x izq)

La entrada multiple también se da cuando hay Rec x izq.

Es posible usar la tabla para detectar que la gramática es predictiva? NO, detecta la ambigüedad, no la predicción, para que detecte la predicción hay que agregarle la Rec x izq.



clase

Dada la sgte gramática, se pide evaluar si es una gramática ambigüa analizando su tabla de análisis sintáctico M.

Método: Para cada producción $A \to \alpha$ de la gramática, hacer lo sgte:

① Para cada terminal a en $P(A)$, agregar $A \to \alpha$ a $M[A,a]$.

② Si ε está en PRIMERO(α), entonces para cada terminal b en $S(A)$, se agrega $A \to \alpha$ a $M[A,b]$. Si ε está en $P(\alpha)$ y \$ se encuentra en $S(A)$, se agrega $A \to \alpha$ a $M[A,\$]$ también.

$S \to i\,E\,t\,S\,S' \mid a$

$S' \to e\,S' \mid \varepsilon$

$E \to b$

$P(S) = \{i\} \cup \{a\}$

$P(S') = \{e\} \cup \{\varepsilon\}$ ← uso SGTE

$P(E) = \{b\}$

$S(S) = \{\$, e\}$

$S(S') = \{\$, e\}$

$S(E) = t$

Se define la tabla M como una tabla compuesta de tantas filas como NO terminales tenga la gramática y tantas columnas como terminales tenga la gramática + \$.

M	i	t	a	e	b	\$
S	$S \to iEtSS'$	No tiene ninguna entrada	$S \to a$			
S'		t forma parte de la gramática, pero NO del P y SGTE		$S' \to eS$ $S' \to \varepsilon$		$S' \to \varepsilon$
E		No se puede eliminar de la entrada.			$E \to b$	

Medio gráfico representa el resultado del análisis.

DESambigüo porque presenta una entrada múltiple.

Presenta entrada múltiple.

Un terminal como ('t') en este caso, NO puedo eliminar simplemente porque no tenga entradas en la tabla. (No creará problemas ni en P ni SGTE).

Si una Fila (No terminal) No tiene entradas

P/ S(S)

$S \to i\,E\,t\,S\,S'$

$A \to \alpha B \beta$ (caso 2)

$S(S) \leftarrow P(S') - \varepsilon$ ← como P(S') tiene ε, aplicamos el caso 3

$S(S) \leftarrow S(S)$

P/ S(S')

$S \to i\,E\,t\,S\,S'$

$A \to \alpha B$ (caso 3)

$S(S') \leftarrow S(S)$

$S' \to e\,S'$

$A \to \alpha B$ (caso 3)

$S(S') \leftarrow S(S')$

P/ S(E)

$S \to i\,E\,t\,S\,S'$

$A \to \alpha B \beta$ (caso 2)

$S(E) \leftarrow P(t)$ ← Por eso se usa el SGTE

Si P y SGTE son disjuntos, cuando hay vacío en PRIMERO, NO hay ambigüedad. Pero si NO son disjuntos, entonces sí hay ambigüedad.

$P(S') = \{e\} \cup \{\varepsilon\}$ $\quad S(S') = \{\$, e\}$ (No son disjuntos)

Scanned with CamScanner

Si una Fila (NO-terminal) NO tiene entradas

↳ puede ser un símbolo impuro pero fundamentalmente, que la Fila NO esté vacía también puede ser impuro (NO tiene SGTE)

Por Ejemplo

S → it SS'
E → b

} E NO aparece en ningún lado derecho, Nadie le invoca, tendrá un SGTE indefinido.

¿Qué producción generaría una Fila vacía?

↳ Un caso puede ser... uno que derive en vacío y que NO aparezca en ningún lado derecho.

Otro caso: cuando hay rec x izq, es una producción simple con rec izq

↳ A → Aia — No podrá resolverse, No hay conjunto primero definido

El SGTE(A) = {i} pero no hay conjunto Primero.

[La tabla puede ser minimizador NO detecta la rec izq. pero detecta símbolos que se pueden eliminar, cuando son producciones simples con rec x izq.]

La Tabla puede ser minimizador eliminando símbolos impuros

→ # La entrada múltiple siempre se va a dar cuando se tenga FC x izq o bien se tenga un ε en el conj. PRIMERO y el PRIMERO no sea disjunto al conj. SGTE.

→ # Que la columna 't' esté vacía NO significa que se deba eliminar, ya que es parte de la gramática (si se quita, cambiaría totalmente la gramática)

→ # Si se tiene una Fila vacía es porque ese NO-terminal no tiene Conj. PRIMERO (la prod. puede ser T → ε y esto definitivamente NO tiene conj PRIMERO. Entonces, si T NO aparece en la tabla es porque NO aparece en ningún lado derecho.

Esa Fila puede eliminarse, tanto en la tabla como de la gramática, T se conoce como símbolo muerto o símbolo impuro.

Scanned with CamScanner

Algoritmo para generar un ASP

Método

Al principio, el analizador sintáctico se encuentra en una configuración con (w)$ en el búfer de entrada, y el símbolo inicial S de G en la parte superior de la pila, por encima de $.
El programa en la figura 4.20 utiliza la tabla de análisis sintáctico predictivo M para producir un An. sintáctico predictivo para la entrada.

```
establecer ip ← primer simbolo 'a' en 'w'      // ip: input; Para que apunte al primer simbolo de (w); primer token o lexema
establecer X ← top() push(S0)                  // con el simbolo de la parte superior de la pila
while (X ≠ $) {                                // Tope de la Pila; Mientras la pila NO está vacía.
    if (X es a) {pop(); ip ← getnext();}       // terminal; sacar de la pila y avanzar ip
    else if (X es un terminal) error();
    else if (M[X,a] es una entrada de error) error();   // Pizarra: else if (M[x,a] está vacía) error();
    else if (M[X,a] = X → Y1 Y2 ... Yn) {
        Print(X → Y1 Y2 ... Yn);               // imprimir: enviar de salida la producción X → Y1 Y2 ... Yn;
        pop()                                  // sacar de la pila
        push(Yn ... Y2 Y1);                    // y reemplazo por su lado derecho; meter Yn, Yn-1 en la pila, con Y1 en la parte superior.
    }
    X ← top() // establecer X con el simbolo de la parte superior de la pila
}
```

El match se hace solo entre terminales

son equivalentes al Match

cuando No hay terminal en el tope de la pila

Debemos cargar de forma invertida la Pila, lo que está a la derecha, cargo primero para que me quede en el tope de la pila, lo que está a la izq. Porque el algoritmo genera una derivación por la izq. Por eso molesta el FC x izq y la Rec x izq.

voy a ciclar siempre y cuando la pila NO esté vacía, cuando la pila se vacía? cuando saque todos los elementos y me quede en el tope de la pila el $ que pusimos inicialmente en la pila

1° if? a es un terminal y coincide con lo que está apuntado por ip, coincide con lo que tengo actualmente en la entrada.

1° else if (x es un terminal) significa que ya NO está apuntado por la entrada porque o sino ya hubiera entrado en el caso anterior. Establece que sea un terminal NO apuntado por la entrada, error.

El 1° if = Si X es el terminal que está en la entrada actualmente, entonces saco ese terminal de la pila y voy avanzar la entrada. Si X es terminal y no está en la entrada, ERROR.

Casos
- cuando en el tope de la pila es un terminal
  - si coincide con la entrada, saco y avanzo
  - Si NO coincide con la entrada, error.

Dónde entra el analizador léxico, el simulador del AFD?
↳ En avanzar ip y establecer ip que apunte al 1° simbolo de w.

cuando en el tope de la pila hay un NO terminal, pasamos al 3° o 4° caso

Scanned with CamScanner

Los dos últimos ifs, corresponden al momento en el que en el tope de la pila tengo NO-terminales, al ser un NO-terminal, planteamos lo sgte:

→ **3° if**: ¿Qué hay en la tabla? En la posición tope de la pila, terminal apuntado por ip, puede ser que no haya nada, puede ser que haya un espacio vacío, si hay un espacio vacío, ERROR, porque quiere decir que 'a' NO está en el conjunto primero(X), ni tampoco en su conjunto SGTE, si el conjunto Primero(X) estuviera vacío, entonces desde X no llegaremos a 'a'

→ **4° if**: Establece que la entrada en la tabla, NO es vacía. El tope de la pila y lo que actualmente tengo en la entrada tiene una posición NO vacía en la tabla? Sí, tiene que ser una producción

Por eso NO puede ser una tabla con entradas múltiples, ambiguas porque NO sabríamos que producción usar. Por eso la ambigüedad es mala para el ASP, porque NO podrá hacer backtraking después. Tiene que haber UNA SOLA ENTRADA

NO hace falta que el algoritmo valide que no hayan dos o más entradas porque no habrán, nuestra **tabla M YA ES PREDICTIVA**

→ Puede que NO tenga entrada
→ A lo más tengo solo UNA entrada, en esa posición

En una LL1 la derivación es de izq a derecha, se carga de forma invertida para que en el tope de la pila, me quede lo que está más a la izquierda.

Pila es LIFO → Last in, first out, lo último que carguemos en la pila, es lo que va a quedar en su tope.

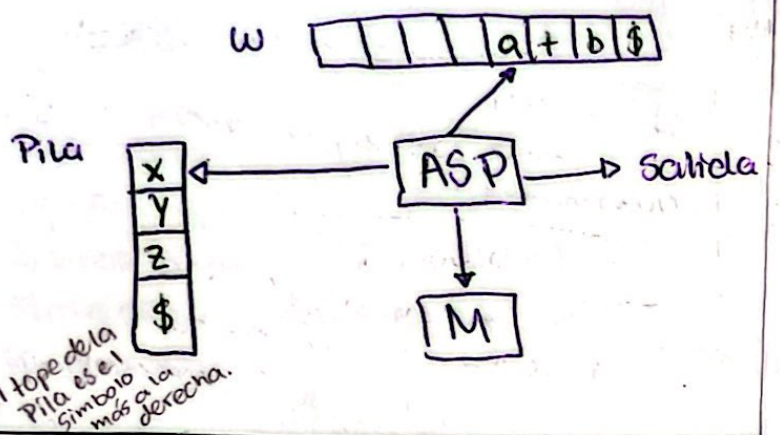
 # La pila NO es toda la producción, hay terminal o NO terminal, la producción voy cargando por componente.

 # W es sólo terminales porque apunta a la entrada

 # X puede ser terminales y NO-t- el tope de la pila puede ser terminales y NO terminales.

- la primera producción que cargo en la pila, es la producción inicial

Scanned with CamScanner



Algoritmo

- La pila tiene terminales y NO-term
- w tiene terminales

→ El tope de la pila, es un terminal?
Si, coincide con la entrada a?
Si, match, sacar el tope de la pila porque es un terminal (NO tiene lado derecho) y avanzar la entrada, pasamos a '+'
Tendría que buscar que '+' esté en el tope de la pila en cualquier momento.

→ Luego pregunto, 'Y' es +?, NO.
'Y' es un terminal? NO, entonces sacamos 'Y' y reemplazamos por su lado derecho. Para hacer eso Y.+ tiene que tener una entrada en la tabla porque si no tiene, NO sabríamos cuál es el lado derecho de 'Y'.

Pila	Entrada	Salida (imprimir)
$E (Símbolo inicial)	id + id $	E → TE'
$E'T (Lado derecho invertido salida ant)	id + id $	T → FT'
$E'T'F	id + id $	F → id
$E'T'id	id + id $ (match)	
$E'T'	+ id $	T' → ε
$E'	+ id $	E' → +TE'
$E'T+	+ id $ (match)	
$E'T	id $	T → FT'
$E'T'F	id $	F → id
$E'T'id	id $ (match)	
$E'T'	$	T' → ε
$E'	$	E' → ε
$	$	

- La pila inicia con $E, porque E es el So.
- En la entrada tengo id + id $
- ip (input) apunta a id, la entrada
- ¿Qué hago cuando tengo en la pila el So y en la entrada id + id $, el primer símbolo es id y a ese le apunta ip?
  - Comparo y digo, el tope de la pila, es un terminal y coincide con la entrada?
  - 1° if, NO. El tope es un terminal? NO. El tope de la pila es un NO-terminal que con la entrada me lleva a la tabla en algún lugar NO-vacío? [E, id] en la tabla tiene una producción, imprimo la producción, eso se uso, saco E' del tope de la pila y cargo su lado derecho en la pila, de forma invertida, lo que está más a la ~~izq~~ derecha queda en el tope de la pila.

continúa →

E → TE'
E' → +TE' | ε
T → FT'
T' → *FT' | ε
F → (E) | id

	id	(	)	*	+	$
E	E → TE'	E → TE'				
E'			E' → ε		E' → +TE'	E' → ε
T	T → FT'	T → FT'				
T'			T' → ε	T' → *FT'	T' → ε	T' → ε
F	F → id	F → (E)				

Scanned with CamScanner

continuación (Fila 2 pila) → símbolo más a la derecha.

- El tope de la pila, es un terminal? NO
  El tope de la pila es un NO-terminal? Sí
  ↳ El tope con la entrada tienen entrada en la tabla? M[T, id], imprimo, saco T y reemplazo T por su lado derecho, en la pila queda $E'T'F (reemplazo a T)

Fila 3 → tope: F

- El tope de la pila es un terminal?, NO.
- El tope es un NO-terminal?, Sí
  ↳ tiene con la entrada, una entrada en la tabla?, Si, M[F, id]; imprimo, saco F y reemplazo por su lado derecho invertido.

Fila 4 → tope = id

- id es un terminal?, Si
  ↳ coincide con la entrada?, Si.
  Sacamos el tope de la pila y avanzamos la entrada. Se hace un match. NO se imprime Nada

Fila 5 → tope: T'

- T' es un terminal?, NO
- T' es un NO-terminal? Sí
  ↳ Tiene con la entrada, una entrada en la tabla? M[T', +], Sí, con ε.
  imprimo la producción.
  T' → ε, Si tengo ese caso, donde el lado derecho es vacío, saco el tope de la pila, NO cargamos vacío en el tope porque ε NO forma parte del alfabeto. T' desaparece y la pila queda $E'

Fila 6 → tope = E'

- E' es un terminal? NO
- E' es un NO-terminal? Sí
  ↳ M[E', +] está definido en la tabla?
  Sí, imprimo, saco E' y reemplazo por el lado derecho invertido

Fila 7 → tope = +

- + es un terminal? Sí
  ↳ coincide con la entrada? Sí
  Saccamos el tope de la pila y avanzamos la entrada.
  se hace match.

Los ifs del algoritmos

- 2 ifs para cuando el tope de la pila es terminal.
- 2 ifs para cuando el tope de la pila NO es terminal.

Fila 8 → tope: T

- T es un terminal? NO
- T es NO-terminal?, Si
  ↳ M[T, id] está definido en la tabla? Sí, imprimo, saco T de la pila y reemplazo por el lado derecho invertido

Fila 9 → Tope = F

- F es un terminal? NO
  ↳ un NO-terminal? Sí.
  ↳ M[F, id] está definido en la tabla? Sí, imprimo, saco del tope de la pila y remplazo por su lado derecho invertido [F, id]

Fila 10 → tope: id

- id es un terminal?, Si
  ↳ coincide con la entrada? Si
  Sacamos el tope de la pila y avanzamos la entrada, se hace match, NO se imprime Nada.

Fila 11 → Tope: T'

- T' es terminal?, NO
  T' es NO-terminal?, Sí
  ↳ M[T', $] está definido en la tabla? Sí, imprimo, sacamos T' y como deriva a vacío ε, T' desaparece y NO cargamos nada

Fila 12 → tope = E'

- E' es terminal? NO
- E' es un NO-terminal?, Sí.
  ↳ M[E', $] está definido en la tabla?, Si, imprimo y E' desaparece porque deriva al vacío.

Pregunta

Es terminal?
- NO → Es un NO-terminal?
  - Sí → tiene en M entrada NO-vacía
    - Sí, imprimo, saco del tope y reemplazo por su der. invertid
- Sí → coincide con la entrada?
  - Si, sacamos del tope y avanzamos

Scanned with CamScanner

Casos de error que NO se tienen en consideración

↳ Que el tope de la pila sea $ pero la entrada NO se ha acabado. Al salir del while (no está en el algoritmo), tenemos que controlar que la entrada también acabe.

Puede darse el caso que consumimos toda la pila, todo el BNF se aplicó bien pero quedaron valores en la entrada que NO se procesaron En el árbol sintáctico sería ... tenemos hojas pero no un padre al que podamos enlazarle hojas.

Clase 17 ♡

Algoritmo para ejecutar un ASP

↳ Si se pide que implementemos un analizador sintáctico, utilizaremos este algoritmo.

↳ con este algoritmo se puede simular la funcionalidad del análisis sintáctico

¿Como puedo utilizar este algoritmo (ASP) para que sea el algoritmo de un compilador tradicional?

¿Cómo hago para que este algoritmo me permita implementar la funcionalidad de un compilador?

Que agarre la entrada y realice las tareas que hasta ahora definimos del compilador?

En el algoritmo →
El analizador léxico esta en Establecer ip que apunta al primer ~~símbolo~~ token de 'w' y en su acción recurrente de avanzar ip.

↳ se le llama al analizador léxico y se le pide que detecte al primer simbolo

Si quiero hacer el cod. Fuente de un compilador, tengo que agarrar el código Fuente del Analizador léxico que va a ser una definición Reg. Tengo que agarrar el código Fuente de la definición regular e implementar ese código fuente, llamar y ejecutar ese código cuando tengo que leer la entrada, el resultado de leer la entrada es lo que se guarda en ip. (input pointer)

El léxico integrado en este código origina el código Fuente del compilador.

↳ Si al código del analizador sintáctico, le sumo el código que simulaba el grafo (An. léxico) tengo el código del compilador, metemos el cod Fuente del (An léxico) en 'establecer ip que apunta al primer simbolo' y en el primer if (avanzar ip) esos dos códigos trabajando juntos son el compilador.

cod. fuente (An. Sintáctico + An. léxico) = compilador

Scanned with CamScanner

En qué caso los terminales forman parte del an. sintáctico

Un compilador tradicional NO tiene terminales en su BNF, tiene tokens.

- $S \rightarrow S + S$, tiene pinta de que será usado en un Traductor porque '+' es un Terminal y está explícitamente en el lado derecho de un BNF.
- En un compilador
  ↳ $S \rightarrow S\ op\ S$, op es un token que es la clase, en una def Reg op puede ser +, −, *, / de tal manera que utilice una def Reg, una Exp. Reg. para definir la clase.

El algoritmo (ASP) nos dice que podemos convertir una definición Regular en un diagrama de estado que más te convenga (preferible AFD) y después migrar a BNF, todo el problema de compilador manejado con una Notación y una herramienta. Exp Reg y diagramas ya no necesitamos

- Si el algoritmo ASP asume que tenemos terminales adentro y es sintáctico, es porque el quiere controlar el 100% del proceso de compilación al final.

Técnicas de SINCRONIZACIÓN

Recuperación de Errores

▷ Estrategia para ayudar a recuperar errores cuando ocurra. Filtrar el error hasta cierto punto de una entrada que viene mal para volverla a ejecutar el algoritmo.

Modo Pánico

En caso de error, omitir símbolos hasta que aparezca un carácter de sincronización.

▷ Recomendable para indicar que hay un error, pero seguir adelante con el proceso.

Caso típico

Todos los símbolos en S(A) pertenecen al conjunto de sincronización de A

se plantea una estrategia para el manejo de errores que tiene que ver con introducir un caracter de sincronización en el SGTE

En SGTE, si la entrada está vacía, introduzco un caracter de sincronización

Recorro los conj SGTE y en cada lugar vacío, meto el caracter de sync.

	id	+	*	(	)	$
E	E→TE'			E→TE'	Sync	Sync
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'	Sync		T→FT'	Sync	Sync
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→id	Sync	Sync	F→(E)	Sync	Sync

▷ Ej: Si M[E,)] ó M[E',$] está vacío, agregamos el caracter de sincronización. (Sync)

$P(F) = P(T) = P(E) = \{(, id\}$

$P(E') = \{+, \varepsilon\}$, $P(T') = \{*, \varepsilon\}$

Si P() aparece vacío, NO hay sync.

Los más expuestos para resolver problemas son aquellos en el q ε aparece en P()

$S(E) = S(E') = \{), \$\}$

$S(T) = S(T') = \{+,), \$\}$

$S(F) = \{+, *,), \$\}$

Estrategia

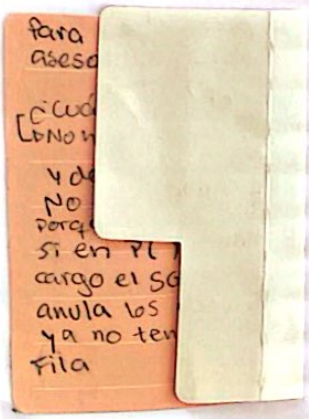
↳ Todo el conj SGTE estará ocupado

1) Si M[A,a] = no definido (está vacía la entrada), omitir 'a'.
2) Si M[A,a] = sync, se elimina A de la pila ▷ (A: tope de la pila) ▷ siempre y cuando en el tope de la pila No sea el símbolo inicial
3) Si top() es un terminal b, y b ≠ a, entonces eliminar b de la pila.

▷ M[E,)] = sync ▷ se aplica la estrategia 1 porque es)

Pila	Entrada	Comentario
E$	)id*+id$	error, omitir)
E$	id*+id$	id está en P(E)
TE'$	id*+id$	
FT'E'$	id*+id$	
idT'E'$	id*+id$	
T'E'$	*+id$	
*FT'E'$	*+id$	
FT'E'$	+id$	error, M[F,+] = sync
T'E'$	+id$	se sacó F
E'$	+id$	
+TE'$	+id$	
TE'$	id$	
FT'E'$	id$	
idT'E'$	id$	
T'E'$	$	
E'$	$	
$	$	

Si en el tope de la pila tenemos un NO terminal, entonces hay algo en la tabla o no hay nada en esa posición, si hay algo elimino el NO terminal del tope de la pila siempre y cuando NO sean símbolos iniciales. Si no hay nada, elimino y avanzo la cadena de entrada, si en el tope hay un terminal y NO coincide con la entrada, entonces elimino el terminal del tope de la pila. ▷ Pasos que hacemos cuando un error para poder seguir con el procesamiento y validar lo que viene después o identificar nuevos errores.



Scanned with CamScanner

→ Podemos recuperarnos de un error siempre y cuando lo que venga en la entrada este en el conj. SGTE, [por eso le agregamos 'sync' al conj SGTE de los NO-terminales.]

Pasos

Teniendo la tabla M, los conjuntos P( ) y S( )

Recorremos los conjuntos SGTES y en cada lugar vacío, ponemos el caracter de sincronización (sync)

Estrategia

1) Si M[A,a] = no definido (está vacía la entrada), omitir a → cuando no hay producción o sync, mantengo el tope de la pila, pero elimino la entrada, avanzo la entrada

→ 1° [E,)] = sync
como es inicial, no puedo eliminar de la pila, porque si hacemos, la pila estará vacía y no procesaremos nada

Pila	Entrada	Comentario
E$	)id *+id$	error, omitir

2) Si M[A,a] = sync, se elimina A de la pila. → Es válido cuando el tope de la pila no sea el símbolo inicial.

3) Si top() es un terminal b, y b≠a, entonces eliminar b de la pila → si en el tope de la pila hay un terminal que NO coincide con la entrada, eliminamos el terminal del tope de la pila

Si en el tope hay un NO-terminal → Hay algo (elimino el NO-terminal del tope)

→ O no hay nada en esa posición de la Tabla (Elimino y avanzo la cadena de entrada)

Si en el tope hay un terminal → NO coincide con la entrada (elimino el terminal del tope)

Obtener la tabla de Análisis Sintáctico (M) para la sgte gramática.

$S \to -FN \mid FN$

FC $FN \to DL \mid DL.DL$ (. terminal)

FC $DL \to D \mid D\,DL$

$D \to 0 \mid 1 \mid 2 \mid \dots \mid 9$

P/ tabla sin factorear para ver el impacto

(*) Para calcular la tabla, debemos llenar el PRIMERO y SGTE

$P(S) = \{-\} \cup P(FN) = \{-\} \cup \{0,1,2\dots9\}$

$P(FN) = \{0\} \cup \dots \{9\} \cup \{0\} \cup \dots \{9\}$ FComún (No son disjuntos.)

$P(DL) = \{0\} \cup \dots \{9\} \cup \{0\} \cup \dots \{9\}$ F. común.

$P(D) = \{0\} \cup \{1\} \dots \cup \{9\}$

Para.!!

La tabla queda:

	0	1	2	...	9	-	$
S	S→FN	S→FN	S→FN	S→FN	S→FN	S→-FN	
FN	FN→DL FN→DL.DL	FN→DL DL.DL	FN→DL DL.DL	FN→DL DL.DL	FN→DL DL.DL		
DL							
D							

$S(S) = \{\$\}$

$S(FN) = \{\$\}$

$S(DL) = \{\$, .\}$

$S(D) = \{0,1\dots9, \$, .\}$

La tabla tiene dos entradas, nos genera un problema de ambigüedad.

- Si FN está en el tope de la pila, NO sabré con cual entrada de la tabla voy a utilizar para cargar el lado derecho de forma invertida a la pila.

Para llenar la tabla, solo si hay el 'ε' en P, vamos a utilizar el conjunto SGTE, por eso en este caso NO calculamo, porque NO hay el vacío

El vacío en P, desencadena usar el SGTE

El factor común tiene un efecto de entrada múltiple en la tabla, no podremos resolver cuando el tope de la pila sea ese NO terminal

$FN \to DL \mid DL.DL$

¿Cómo resuelvo FC.?

① Resolviendo Factor Común

$FN \to DL \mid DL.DL$ queda

$FN \to DL\,R$

$R \to .DL \mid \varepsilon$ # tendremos que lidiar con el conj SGTE

$DL \to D \mid D\,DL$

$DL \to D\,R'$

$R' \to DL \mid \varepsilon$

② Actualizar el C PRIMERO

$P(S) = \{-\} \cup \{0\} \cup \dots \{9\}$

$P(FN) = P(DL) = \{0\} \cup \dots \{9\}$

$P(R) = \{.\} \cup \{\varepsilon\}$

$P(DL) = \{0\} \cup \dots \{9\}$

$P(R') = P(DL) \cup \{\varepsilon\} = \{0\} \cup \dots \{9\} \cup \{\varepsilon\}$

$P(D) = \{0\} \cup \{9\}$

Para obtener la tabla se debe eliminar el FC en DL y FN para completar la tabla

Scanned with CamScanner

Ej2 Obtener la tabla de análisis sintáctico M para la sgte gramática:

Resolver la rec x izq.

$S \to SNRO \mid \varepsilon$ Rec x izq.

$NRO \to 0 \mid 1 \mid 2 \mid \dots \mid 9$

▷ Aparece vacío, como NO forma parte del alfabeto, me voy al conjunto SGTE y cargar la prod. $S \to \varepsilon$

$S \to SNRO$

} Se viola la 3ra regla de LL1, hay una ambi. semántica.

$P(S) = P(SNRO) \cup \{\varepsilon\} = \{0,1,2..9\} \cup \{\varepsilon\}$

$P(NRO) = \{0\} \dots \{9\}$

$S(S) = \{\$, 0, 1, 2, \dots 9\}$

$S(NRO) = \{\$, 0, 1, 2 \dots 9\}$

	0	1	...	9	$
S	$S \to SNRO$ $S \to \varepsilon$	$S \to SNRO$ $S \to \varepsilon$	SNRO ε	SNRO $S \to \varepsilon$	$S \to \varepsilon$
NRO					

▷ se pone aquí, porque se analiza cada elemento del SGTE y añadir $S \to \varepsilon$

La tabla tiene entradas múltiples

↳ Esto se debe a la presencia del 'ε', genera un problema, porque no puedo saber cual producción elegir.

AL NO ser disjunto, NO es LL(1), hay una ambigüedad semántica, eso ocasiona que tenga entradas múltiples

Por ejemplo

① Si en la entrada se tiene un 0 más una cadena de dígitos que aún no se sabe cuales son, entonces:

$S \to SNRO \Rightarrow NRO \Rightarrow 0$

▷ S puede ser ε por eso desaparece.

▷ Hace match con 0 para consumir la entrada

② $S \Rightarrow SNRO \Rightarrow SNRONRO \Rightarrow NRONRO \Rightarrow 0NRO$.

AL no ser disjunto P y SGTE, porque hay vacío, entonces, se genera entradas múltiples

Como son LL(1) no puedo mirar más allá de la entrada, solamente un símbolo, que es el símbolo de la entrada. Para mal o bien, hay ambigüedad y no sabemos cual es la manera en que podemos consumir el símbolo de entrada.

Es importante entender que esta situación se debe a la presencia del ε en la producción inicial S.

Ej3

$A \to Aa \mid b$

▷ No hay definición para P(A), no tengo manera de aportar nada para el P(A), descarto

▷ Este si aporta algo para el P(A)

(No está def todavía)

¿Qué hago con la primera parte? P(Aa) NO está definido, entonces tengo que ver si puedo resolver, y si puedo, eso va a valer el P(A)

$P(A) = P(Aa) \cup P(b) = \{b\}$ o ~~No está definido~~ $\cup \{b\}$

▷ conclusión:

Podemos tomar que NO está definido o que $P(A) = \{b\}$, tiene validez, pero NO estoy en condiciones de dar un valor a este P(Aa) porque NO está definido

$S(A) = \{\$, a\}$

$A \Rightarrow Aa \Rightarrow Aaa \Rightarrow Aaaa \Rightarrow^* \dots Aa \dots aa \Rightarrow b \dots aaa$

	a	b	$
A		$A \to b$ $A \to Aa$	Si P(Aa) = b

▷ Si no declaramos que vemos está def.

Si había $A \to Aa$, la tabla NO está definida porque no hay un conjunto PRIMERO definido.

ocurre esto, cuando hay una derivación por izq que entra en recursión. Existe una Rec x izq que afecta la gramática.

P(A) = Aà|b.
Lo conclusión:

La tabla es una validadora de gramáticas LL1

Hallar la table es una manera de probar que una gramática es o NO LL1

La tabla, Finalmente, es una herramienta que valida una gramática si es LL(1) o Predictiva

Las reglas LL(1) NO analiza la Rec x izq. Si aplicamos la regla, necesariamente debemos controlar la Rec x izq Antes o después para decir que una gramática es o NO LL(1)

La tabla validaba las ambigüedades sintácticas y semánticas porque son las que generan entrada múltiple.
Pero vimos que la entrada multiple también se genera cuando hay Rec x izq.

Scanned with CamScanner

Análisis sintáctico Ascendente (Reducciones)

Construcción del árbol de An. sintáctico en sentido ascendente: Se comienza desde la cadena de entrada y se busca llegar al símbolo inicial de la gramática.

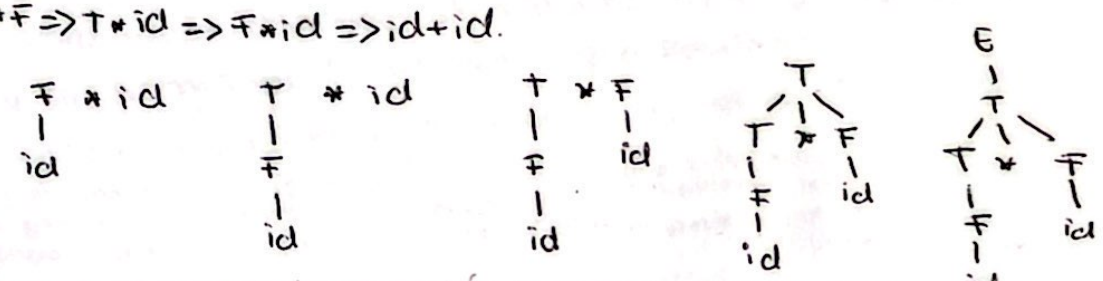
Reducción: Reducir la cadena de entrada al símbolo inicial.

Aquí cambia de paradigma, un paradigma invertido, cambia el sentido porque vamos a empezar con la cadena de entrada pero siempre mantenemos el contexto de la izq. Al cambiar el paradigma, comienzan los problemas, por eso se define el concepto de mangos

$id + id \Rightarrow F * id \Rightarrow T * id \Rightarrow T * F \Rightarrow T \Rightarrow E$

$E \Rightarrow T \Rightarrow T * F \Rightarrow T * id \Rightarrow F * id \Rightarrow id + id.$

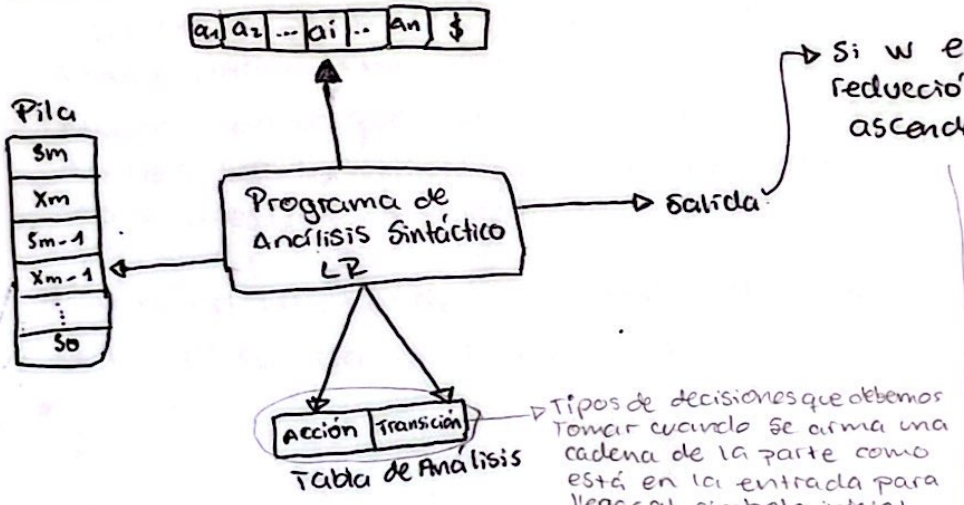
id = id



Mangos

Tabla compuesta

Como el mango de una sartén, encontrar todo lo que está más a la izquierda, que se pueda agrupar y reemplazar por un terminal.



Puede que una gramática tengo problemas por Rec x izq y Rec x der a la vez? Puede, pero es un caso muy raro.

Solo vamos a saber el primer elemento de la cadena de entrada.

!!! Saber conceptualmente ↓

Esta es la estrategia que se utiliza cuando NOS quedamos sin elementos en las LL, es importante saber porque existen algunas limitaciones en el análisis léxico con las Exp. Regulares ya que las Exp Reg NO pueden contar, es un formalismo que tiene sus limitaciones, Las LL tiene sus limitaciones pero nos escapamos recurriendo a la LR.

Scanned with CamScanner

V 17 (21)

Capitulo

Análisis semántico

Definición dirigida por la sintaxis

Una definición dirigida por la sintaxis, es una gramática libre de contex formada por atributos y reglas o acciones. Los atributos se asocian con simbolos gramaticales y las reglas con las producciones (pueden ser directas en su resultado y no necesitan pasar valores)

Si X es un simbolo y a es un atributo, escribimos X.a. (puede ser entero, vector, lo que sea)

Tipos de atributos (depende de quien le ayude a valorizar)

a) Sintetizados. cuyo valor depende de sus hijos y del Nodo mismo donde se encuentra. Mira por abajo (valor de retorno)

b) Heredados: cuyo valor depende de valores en el padre, en el nodo mismo y sus hermanos. (parámetro que recibe)

cuando la gramática me permite, puedo insertar acciones dentro de una gramática. (Posfijo: insertabamos print', es una acción porque básicamente ya genera un resultado. La acción simplifica la aplicación y el juego para generar valores. si no es suficiente, la acción puede incluir variables)

casos

La acción insertado en el BNF

Las reglas gramáticas separadas (D y V)

Ambas pueden tener variables, lo llamamos atributos o instancias

¿Quienes son los que van a tener esas variables?

↳ Los NO-terminales, a estos le agregamos las variables. garantizar los valores que va a retornar la Función.

- Un valor sintetizado es cuando tengo que calcular el valor de los hijos y el resultado se guarda en el padre
-

Recurso sintetizado para resolver problemas típicamente heredados

Que recursos de programación se utilizan para resolver problemas heredados en forma sinterizada? La Pila

↳ casi cualquier problema heredado se puede resolver con una sintetizada y una estructura adicional 'Pila'.

Scanned with CamScanner

Ej: Expresiones Aritméticas para los operadores + y *

Exp. Aritméticas p/ los operadores + y *

Producción	Reglas Semánticas
1) L → En	L.val = E.val
2) E → E1 + T	E.val = E1.val + T.val
3) E → T	E.val = T.val
4) T → T1 * F	T.val = T1.val x F.val
5) T → F	T.val = F.val
6) F → (E)	F.val = E.val
7) F → digito	F.val (sintetizado) = digito.lexval

va a tomar su valor siempre a partir del valor de sus hijos.

2º lexval es una instancia en el que asumimos tener el dígito
val es un atributo sintetizado porque va a tomar su valor siempre a partir del valor de sus descendientes, de sus hijos

V17 2018

cuando hacemos análisis sintáctico, tenemos dos caminos,

Reglas o acciones (Pueden tener las variables)

Las acciones tienden a ser más directas en su resultado y NO necesitar pasar valores (por algo son acciones) ▷ simplifica la aplicación para generar valores

cuando la gramática me permite, puedo insertar las acciones dentro de una gramática

Si NO es suficiente la acción, puede incluir variables

*Ambas pueden tener variables (Los NO-terminales le agregamos las variables)

Si X es un símbolo y a un atributo, escribimos X.a

Quiero generar el concepto que el NO-terminal se lleva el valor

21/10/24

Compiladores

$A \to A_1\ Y \quad \{A.a = g(A_1.a, Y.y)\}$

$A \to X \quad \{A.a = f(X.x)\}$

⇩

$A \to X \quad \{R.h = f(X.x)\}$ ← resultado del lado derecho

$R \quad \{A.a = R.s\}$

$R \to Y \quad \{R_1.h, Y.y\}$

$R_1 \quad \{R.s = R_1.s\}$

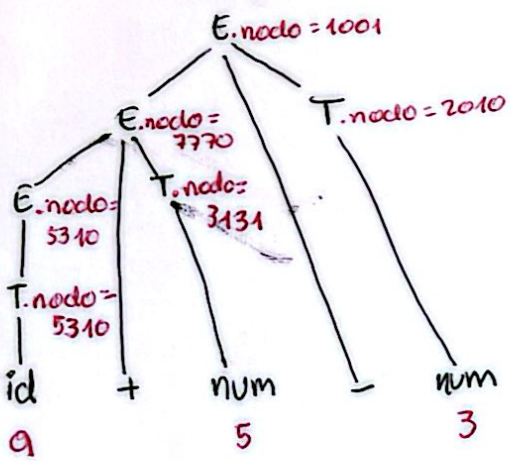
$R \to \varepsilon \quad \{R.s = R.h\}$

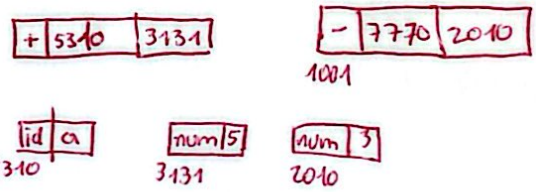
g ≡ agregar

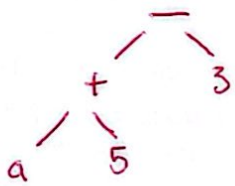
g ≡ L derecho (agr, input, h)

La aplicación de fórmula debe ser pensada.

BNF	Reglas
$E \to E_1 + T$	E.nodo = new ('+', E_1.nodo, T.nodo)
$E \to E_1 - T$	E.nodo = new ('-', E_1.nodo, T.nodo)
$E \to T$	E.nodo = T.nodo (función de P)
$T \to (E)$	T.nodo = E.nodo
$T \to id$	T.nodo = n Hoja (id, id.input)
$T \to num$	T.nodo = n Hoja (num, id.input)







Scanned with CamScanner

29/11

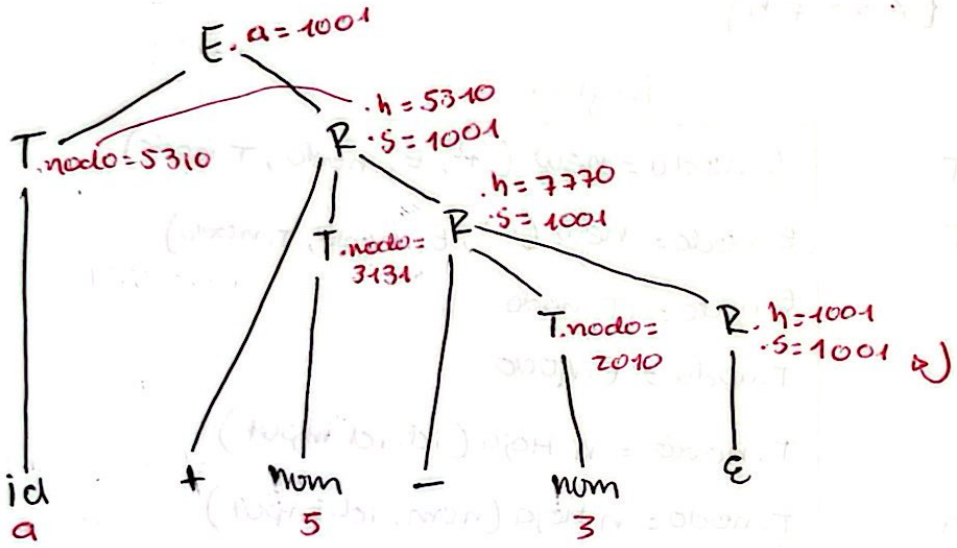
Eliminando la Rec x izq:

$E \to T$	$R.h = T.nodo$
R	$E.a = R.S$

$R \to +T$	$R_1.h = new$ [illegible] $('+', R.h, T.nodo)$
R_1	$R.S = R_1.S$

$R \to -T$	$R_1.h = new\ ('-', R.h, T.nodo)$
R_1	$R.S = R_1.S$

$R \to \varepsilon$	$R.S = R.h.$



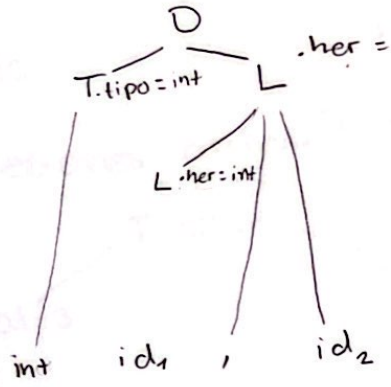
BNF	Reglas
$D \to T\ L$	$L.her = T.tipo$
$T \to int$	$T.tipo = int$
$T \to float$	$T.tipo = Float$
$L \to L_1, id$	$L_1.her = L.her$ agregarTipo(id.input, L.her)
$L \to id$	agregarTipo(id.input, L.her)

→ a = L.her
agregar(input(id), a))
g(x)

Lag( ) estado lo que hay acá

void L(her){ }

Esto es un parámetro de L

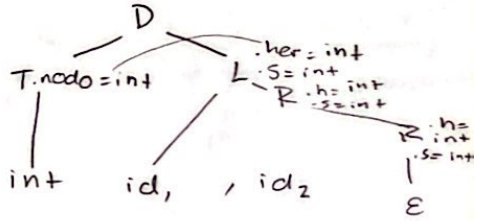


Eliminando la Rec x izq

Hay herencia (En este caso) **ojo**

$L \to id\ R_1$	agregarTipo(id.input, L.her) $R_1.h = L.her$ $L.s = R_1.s$
$R \to , id\ R_1$	$R.h = R.h$ agregar(id.input, R.h) $R.s = R_1.s$
$R \to \varepsilon$	$R.s = R.h$

id1, int



El lado derecho es la función
Si tenemos her a tender, la herencia está generando un resultado.

Si la Rec x izq tiene her

TP

Tabla de símbolo → Hash condiciones
→ se le agrega un bucket.
→ con Lista enlazada (más óptimo)
Usar ese concepto

Malo / Bueno

palabra	ponderación

Podemos limitar la tabla

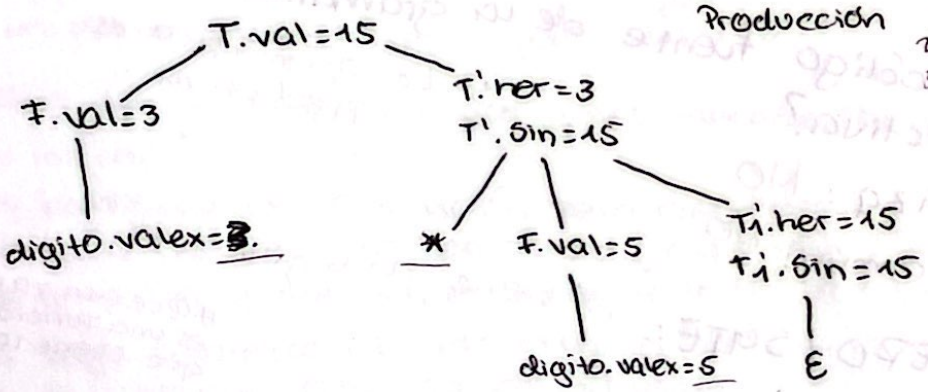
Scanned with CamScanner

Atributos sintetizados y heredados

Producción	Reglas semánticas
1) $T \to F\,T'$	$T'.her = F.val$ → acumular algún resultado. $T.val = T'.sin$
2) $T' \to *\,F\,T'_1$	$T'_1.her = T'.her \times F.val$ $T'.sin = T'_1.sin$
3) $T' \to \varepsilon$	$T'.sin = T'.her$
4) $F \to digito$	$F.val = digito.valex$

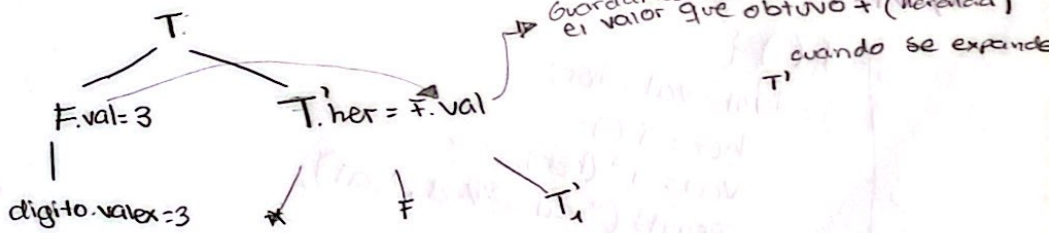
No hay Recursividad

Ej: Expresiones aritméticas para los operadores + y *



Producción
1) Derivamos T → FT'
2) Derivamos F
3) Derivamos T'

Entrada = 3 * 5



Cuando se expande T', F volverá a tener un valor sintetizado porque va a derivar, digito cargará el valor....

Las variables heredadas, son NO-terminales que están a la derecha pero en las reglas semánticas o relaciones semánticas, se pasan a la izq para tomar su valor de un asignación.

→ Cualquier variable sintetizada es una variable que se define dentro de la función y sólo tiene alcance dentro de la función
→ La var heredada es la que se pasa como parámetro porque dentro de la función está el hijo.

La var es heredada cuando..
el NO-terminal está a la derecha
izq → der
NO-terminal — conj. de terminales y NO-terminales
Cuando el NO-terminal de la derecha pasa a la izquierda y el de la izq. pasa a la derecha, se está pasando valor

Scanned with CamScanner

Prod.	Reglas semánticas
1) $T \to FT'$	$T'.her = F.val$ $T.val = T'.sin$
2) $T' \to *FT'_1$	$T'_1.her = T'.her \times F.val$ $T'.sin = T'_1.sin$
3) $T' \to \varepsilon$	$T'.sin = T'.her$
4) $F \to digito$	$F.val = digito.valor$

Lo que retorne F, voy a guardar en her

Está en el parámetro

return.

Se utiliza el CONJUNTO SGTE

Cuando está .sin alaiz devuelve el valor. Al estar a la derecha y ser .sin significa que alguna función está dentro tuyo devolviendo el valor

Escribir el código fuente de la gramática.

1) Es predictiva?

Rec x izq : NO

Factor común: NO

PRIMERO / SGTE: No tiene ambigüedades.

∴ La gramática es predictiva.

2) Divide y vencerás.

```
T(){
    int val, her;
    her = F();
    val = T'(her);
    printf("val = %d", val);
}
```

Llamante.

```
T'(int her){
    int val;
    if (input = '*'){
        match('*'); val = F(); T'(her*val);
    }else{
        return(her); // sin=her return(sin);
    }
}
```

Parte

Los hijos de T', he ven a her

Pueden ver el valor de F.

Poner las reglas en T'

$T' \to \varepsilon$ → Como

Que una var sea heredada es una función, es un rol que tiene la variable.

A T'1 pasar el valor acumulado en T' ¿?

cuando es heredado paso como parámetro cuando es sintetizado paso por el return.

Por parámetro: Porque quiero que todo lo que está dentro de la función de la que está recibiendo como parámetro pueda verle, dentro de la función está el hijo

Quiero que todo lo que haya en T'(her) lo pueda ver her, eso logramos, pasando por parámetro, esa es la manera de heredar el valor.

El concepto de sintetizado es cuando retornas el valor

Scanned with CamScanner

cuando hay variables heredadas ocurre el rebote, recibir lo que pasamos por parámetro, devuelvo.

Las variables sintácticas
↳ Son los NO-terminales → Pueden tener instancias → Sintetizadas: valor de los hijos
→ Heredadas: valor del padre

Grafos de referencia

Ponerle sentidos al árbol, define roles, quién será el papá hijo, etc.

Grafo de dependencia.

Define como fluyen los datos, definitivamente son grafos que van de abajo para arriba (en las hojas están los datos)

Cuando estoy en presencia de un Terminal, tengo resuelto el valor de un dato y debo tirar para arriba

Ayuda a ver si tenemos variables heredadas.

Al ver los reglas

Si soy llamante de la función pero aparezco a la derecha, Probablemente mi valor es heredado (porque tendría que aparecer siempre a la izq y obtener el valor de todo lo que está a mi derecha)
Si en la regla aparezco al lado derecho en una operación, quiere decir de que alguien está usando el valor que yo recibí

Scanned with CamScanner

Producción	Reglas semánticas
1) $E \to E_1 + T$	E.nodo = new Nodo('+', E1.nodo, T.nodo)
2) $E \to E_1 - T$	E.nodo = new Nodo('-', E1.nodo, T.nodo)
3) $E \to T$	E.nodo = T.nodo
4) $T \to (E)$	T.nodo = E.nodo
5) $T \to id$	T.nodo = new Hoja(id, id.entrada)
6) $T \to num$	T.nodo = new Hoja(num, num.val)

crea nodos de un árbol.

Es sintetizado!

Construir el árbol sintáctico para: a - 4 + c

↳ Si aparece en el lado derecho de la producción

Si el valor obtenido tengo que guardar en una instancia de ellos, entonces tienen que recibir eso como parámetro., E.nodo es el llamante, tiene que aparecer a la derecha para que sea heredada.

↳ Formula para convertir una gramática con Rec x izq. en una gramática sin rec x izq. con variables heredadas.

Transformación de Acciones Semánticas (Reglas semánticas)

El método gral para adaptar otros esquemas de Traducción dirigidas por la sintaxis es el sgte:

↳ gramáticas con acciones o reglas que tienen variables sintetizadas

a) Suponer que se cuenta con

Rec x izq

$A \to A_1 Y \quad \{A.a = g(A_1.a, Y.y)\}$

$A \to X \quad \{A.a = f(X.x)\}$

↳ Representan una función.

Scanned with CamScanner

b) El algoritmo para eliminar la rec x izq simple es:

$A \rightarrow X\ R$

$R \rightarrow Y\ R \mid \varepsilon$

c) Aplicar:

Parámetro que le debo pasar a la función R

$A \rightarrow X \quad \{R.h = f(X.x)\}$ → se refiere a T.nodo (return)

$R \quad \{A.a = R.s\}$ (valor final)

$R \rightarrow Y \quad \{R_1.h = g(R.h, Y.y)\}$ → new nodo; Hace referencia al valor que voy a recibir por T

$R_1 \quad \{R.s = R_1.s\}$

$R \rightarrow \varepsilon \quad \{R.s = R.h\}$ → Rebota lo que recibo de ese parámetro voy a retornar.

Reglas Semánticas

Prod.		
1) $E \rightarrow E_1 + T$	Rec x izq.	E.nodo = new Nodo('+', E1.nodo, T.nodo)
2) $E \rightarrow E_1 - T$	Rec x izq	E.nodo = new Nodo('-', E1.nodo, T.nodo)
3) $E \rightarrow T$	Rec x izq	E.nodo = T.nodo (Es valor asignado / valor heredado)
4) $T \rightarrow (E)$		T.nodo = E.nodo
5) $T \rightarrow id$		T.nodo = new Hoja(id, id.entrada)
6) $T \rightarrow num$		T.nodo = new Hoja(num, num.val).

$A \rightarrow A + \mid \beta$

$A \rightarrow \beta R$

$R \rightarrow + R \mid \varepsilon$

$E \rightarrow E_1 + T \mid E_1 - T \mid T$ (α_1, α_2, β)

$E \rightarrow TR$

$R \rightarrow (+T \mid -T)R \mid \varepsilon$

$R \rightarrow +TR \mid -TR \mid \varepsilon$

Aplicar la regla p/ eliminar la Rec x izq

Prod.	Reglas semánticas
$E \rightarrow TR$	R.h = T.nodo
$R \rightarrow +T\ R$ (α)	E.a = R.s
$R \rightarrow -T\ R$ (α)	R_1.h = new Nodo('+', R.h, T.nodo)
$R \rightarrow \varepsilon$	R_1.h = new Nodo('-', R.h, T.nodo)
	R.s = R.h.

$E \rightarrow TR$
$R \rightarrow +TR$
$R \rightarrow -TR$
$R \rightarrow \varepsilon$

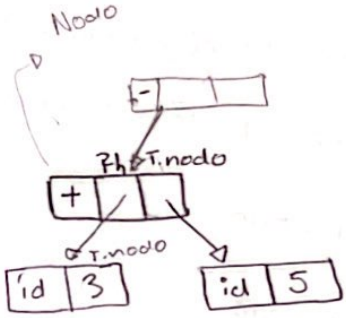
→ Adecuación de las Reglas semánticas eliminando la Rec x izq.

→ continúa

Scanned with CamScanner

BNF	Reglas semánticas
$E \rightarrow TR$	$R.h = T.nodo$ $E.a = R.S$
$R \rightarrow +TR_1$	$R_1.h = new\ Nodo('+', Rh, T.nodo)$ $RS = R_1.S$
$R \rightarrow -TR$	$R_1.h = new\ Nodo('-', Rh, T.nodo)$ $RS = R_1.S$
$R \rightarrow \varepsilon$	$R.S = R.h$

$R_1.h = new\ Nodo('+', Rh, T.nodo)$



BNF	Reglas
$num \rightarrow num_1\ num_2$	$num.x \rightarrow num.x + num.x$ $num.y = num.y + num.y$
$num \rightarrow num_1\ 0$	$num.x = num_1.x$ $num.y = num_1.y + 1$
$num \rightarrow 1001$	$num.x = 2;\ num.y = 2$
$num \rightarrow 11$	$num.x = 2$

Dec x izq:
IC:
Ambos:

$A \rightarrow x \ \{R.h = f(x.x)\}$
$\quad R \ \{A.a = R.s\}$

$R \rightarrow Y \ \{R_1.h = g(R.h, Y.y)\}$
$\quad R_1 \ \{R.s = R_1.s\}$

$R \rightarrow \varepsilon \quad \{R.s = R.h\}$

$num \rightarrow$ (1) 001 R | (11) R (IC)

$R \rightarrow num\ R \mid 0\ R \mid \varepsilon$

Divide y vencerás

	If (1 + S.x == S.y) TRUE else FALSE (sintetizada)
$num \rightarrow 1S$	
$S \rightarrow 001R$	$S.x = 1 + R.sx$ $S.y = 2 + R.sy$
$S \rightarrow 1R$	$S.x = 1 + R.sx$ $S.y = R.sy$
$R \rightarrow num\ R_1$	$R_1.hx = num_x + R.h_x;\ R_1.h_y = num.y + R.h_y$ $R.sx = R_1.sx$ $R.sy = R_1.sy$
$R \rightarrow 0R_1$	$R_1.hx = R.hx;\ R_1.h_y = R.h_y + 1$ $R.sx = R_1.sx$ $R.sy = R_1.sy$
$R \rightarrow \varepsilon$	$R.sx = R.hx$ $R.sy = R.hy$

Tabla de símbolos
Listas de listas
mejor opción

Scanned with CamScanner